

Hochschule Bremen
City University of Applied Sciences



Masterarbeit

Von statisch zu dynamisch:

Lernende Gegner als Designfaktor in interaktiven
Spielsystemen

Name: Berkan Bulut

Matrikelnummer: 5004459

Fakultät: Elektrotechnik und Informatik

Studiengang: Informatik M. Sc.

Betreuer 1: Jendrik Bulk

Betreuer 2: Andreas Teufel

Abgabedatum: [17.11.2025]

Eidesstattliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Masterarbeit mit dem Titel „*Von statisch zu dynamisch: Lernende Gegner als Designfaktor in interaktiven Spielsystemen*“ selbstständig und ohne unzulässige Hilfe angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Bremen, den 17.11.2025

A handwritten signature in black ink, appearing to read 'B. Bulut', with a long horizontal flourish extending to the right.

Berkan Bulut

Zusammenfassung

Diese Arbeit untersucht den Einsatz von Reinforcement Learning (RL) zur Entwicklung adaptiver nicht-spielbarer Charaktere (NPCs) in digitalen Spielen. Im Mittelpunkt steht die Frage, inwieweit RL-basierte Gegner ihr Verhalten dynamisch an individuelle Spielstile anpassen können, um sowohl die Immersion als auch die wahrgenommene Herausforderung zu steigern. Als methodische Grundlage dient ein experimenteller Prototyp in der Unity-Engine, der es ermöglicht, RL-gesteuerte NPCs systematisch mit klassischen, skriptbasierten Gegnern zu vergleichen. Die Evaluation kombiniert quantitative Analysen objektiver Leistungsmetriken mit qualitativen Einschätzungen durch Spieltests. Dadurch werden sowohl die technische Funktionsweise als auch die gestalterische Wirkung adaptiver Agenten auf das Spielerlebnis umfassend bewertet. Die Ergebnisse zeigen, dass RL-basierte NPCs ein höheres Maß an Reaktivität und Unvorhersehbarkeit aufweisen und damit zu einer intensiveren Spielerfahrung beitragen. Zugleich werden Grenzen und Herausforderungen des Ansatzes sichtbar, insbesondere in Bezug auf Trainingsaufwand und Steuerbarkeit des Verhaltens. Aus den Befunden lassen sich konkrete Designprinzipien für den gezielten Einsatz lernfähiger NPCs ableiten, die als Grundlage für zukünftige Entwicklungen adaptiver Spielsysteme dienen können.

Inhaltsverzeichnis

1	Einleitung	13
1.1	Motivation	13
1.2	Zielsetzung der Arbeit	14
1.3	Aufbau der Arbeit	15
2	Theoretische Grundlagen	17
2.1	Reinforcement Learning	17
2.2	Game AI und klassische NPC-Logiken	18
2.3	Adaptive Systeme und Player-Centered AI	19
2.4	Multiagenten-Lernen	20
3	Verwandte Arbeiten	22
3.1	Stand der Forschung	22
3.1.1	Künstliche Intelligenz und Reinforcement Learning	23
3.1.2	Game AI und klassische NPC-Logiken	23
3.1.3	Game Design und Player-Centered AI	24
3.2	Abgrenzung und Einordnung der Arbeit	24
4	Konzeption und Umsetzung des Prototyps	26
4.1	Designüberlegungen	26
4.2	ML-Agents Setup	27
4.2.1	Python-Umgebung und Paketinstallation	28
4.2.2	Unity Setup	29
4.2.3	YAML-Konfiguration	31
4.2.4	Training und Ablauf	33
4.3	Arena-Aufbau und Steuerung	34
4.4	Verhalten und Ziel der NPCs	37

4.5	Belohnungssystem	38
4.5.1	Collector-Agenten	39
4.5.2	Hunter-Agent	39
4.5.3	Zusammenfassung	40
4.6	Technische Umsetzung in Unity	40
4.6.1	Grundlegende Infrastruktur	40
4.6.2	Geskriptete Variante	40
4.6.3	Dynamische Variante (RL-Integration)	41
4.7	Integration von ML-Agents	41
4.8	Trainingsprozess und Visualisierung mit TensorBoard	42
4.8.1	Iteration 1: Basis-Setup mit einfachem Hunter	44
4.8.2	Iteration 2: Erweiterte Observations und aktiver Hunter	45
4.8.3	Iteration 3: Finale Optimierung mit stabilisiertem Lernverhalten	45
5	Methodik	47
5.1	Forschungsfragen und Hypothesen	47
5.2	Studiendesign und Datenerhebung	48
5.3	Fragebogen	49
5.4	Qualitative Beobachtung	49
5.5	Usability-Test und User Experience	50
5.6	Bewertungsmatrix	51
6	Evaluation	53
6.1	Auswertung der quantitativen Metriken	54
6.1.1	Interpretation der Leerlauf-Simulationen	54
6.1.2	Quantitative Visualisierung der Leerlauf-Simulationen	56
6.2	Auswertung der Live-Tests	57
6.2.1	Datengrundlage und Teilnehmerprofil	57
6.2.2	Verlauf der Spielerleistung (Scores)	60
6.2.3	Quantitative Ergebnisse der Fragebögen	62
6.3	Analyse der qualitativen Daten	71
6.3.1	Auswertung der Freitextantworten	71
6.3.2	Wahrgenommener Realismus der Varianten	73

6.3.3	Verbesserungsvorschläge zu den NPCs	75
6.3.4	Wahrgenommene Lernfähigkeit der RL-NPCs	77
6.3.5	Eindruck des dynamischen Verhaltens	79
6.3.6	Beobachtungen aus den Live-Tests	81
7	Diskussion	83
7.1	Zusammenfassung der zentralen Befunde	83
7.2	Interpretation im Kontext der Hypothesen	84
7.3	Limitationen und Herausforderungen	87
8	Fazit und Ausblick	88
8.1	Zusammenfassung	88
8.2	Zukünftige Arbeiten	89
A	Anhang	90
A.1	Fragebogen	90
A.2	Trainingsparameter	94
A.3	Code-Auszüge	95
A.4	TensorBoard-Statistiken	117

Abbildungsverzeichnis

4.1	Unity-Szenen-Struktur	30
4.2	Arena-Aufbau in Unity: Spielfeld mit Wänden, Boden und zufällig spawnenden Coins.	35
4.3	Spiel in Ego-Perspektive mit HUD-Timer am oberen Bildschirmrand. .	36
4.4	Scoreboard am Rundenende mit Coin-Zählung und Zeitangabe.	36
4.5	Schematische Darstellung des Collector-Verhaltens: Coins (gelb), Spieler (rot), NPC (blau) sowie Sicherheits- und Gefahrenradius.	37
4.6	Visualisierung der NPC-Ausweichlogik in Unity mittels Gizmos.	38
4.7	Zentrale Funktionen eines ML-Agents in Unity	42
6.1	Vergleich der durchschnittlich eingesammelten Coins in den Leerlauf-Simulationen.	56
6.2	Entwicklung der durchschnittlich eingesammelten Coins im Lernverlauf.	56
6.3	Altersverteilung der Testpersonen	58
6.4	Spielerfahrung der Testpersonen (in Jahren)	58
6.5	Selbsteingeschätzte Spielzeit pro Woche	59
6.6	Erfahrung der Testpersonen mit KI/NPCs	59
6.7	Verlauf der durchschnittlichen Rundenzeit über die Testdurchläufe . . .	61
6.8	Durchschnittliche von den Collector-NPCs gesammelte Coins	61
6.9	Verteilung der Antworten zu „NPCs wirkten glaubwürdig“ (1–5)	63
6.10	Verteilung der Antworten zu „NPCs reagierten situationsangemessen“ (1–5)	63
6.11	Bewertung: „Die Scripted Version war schwierig“ (1–5)	64
6.12	Bewertung: „Die Dynamic Iteration 1 war schwierig“ (1–5)	64
6.13	Bewertung: „Die Dynamic Iteration 2 war schwierig“ (1–5)	65
6.14	Bewertung: „Die Dynamic Iteration 3 war schwierig“ (1–5)	65
6.15	Bewertung: „Der Schwierigkeitsgrad war angemessen“ (1–5)	65

6.16	Bewertung: „ <i>Balance Sammeln/Ausweichen wirkte fair</i> “ (1–5)	66
6.17	Bewertung: „ <i>NPCs wirkten abwechslungsreicher</i> “ (1–5)	67
6.18	Bewertung: „ <i>Verhalten steigerte meine Immersion</i> “ (1–5)	68
6.19	Bewertung: „ <i>Ich hatte das Gefühl, gegen ‚echte‘ Gegner zu spielen</i> “ (1–5)	68
6.20	Bewertung: „ <i>Steuerung des Spiels war verständlich</i> “ (1–5)	69
6.21	Bewertung: „ <i>Spielziele waren klar und nachvollziehbar</i> “ (1–5)	69
6.22	Thematische Verteilung der Freitextantworten nach inhaltlichen Kategorien (in %)	73
6.23	Häufigkeit der Nennungen realistischer Varianten in den Freitextantworten (n=10)	75
6.24	Subjektiv wahrgenommene Lernentwicklung der RL-NPCs (Frequenz der Antworten)	78
6.25	Thematische Verteilung der Rückmeldungen zum Eindruck des dynamischen Verhaltens (in Verhältnis zu 100%)	80
A.1	Cumulative Reward – Iteration 1 (Collector und Hunter).	117
A.2	Policy Loss – Iteration 1 (Collector und Hunter).	117
A.3	Cumulative Reward – Iteration 2 (Collector und Hunter).	117
A.4	Policy Loss – Iteration 2 (Collector und Hunter).	118
A.5	Cumulative Reward – Iteration 3 (Collector und Hunter).	118
A.6	Policy Loss – Iteration 3 (Collector und Hunter).	118
A.7	Cumulative Reward (Collector und Hunter).	119
A.8	Policy Loss (Collector und Hunter).	119

Tabellenverzeichnis

5.1	Bewertungsmatrix zur Evaluation	52
6.1	Übersicht der zentralen Merkmale der Stichprobe	60
6.2	Durchschnittswerte der Spielerleistung über alle Teilnehmenden	61
6.3	Thematische Cluster der Verbesserungsvorschläge	76
A.1	Übersicht der verwendeten Trainingsparameter für Iteration 1–3.	95

Listingsverzeichnis

4.1	YAML-Konfiguration für das Training von Hunter und Collector	31
4.2	Start des Trainingsprozesses über die Konsole	34
A.1	CollectorAgent.cs	95
A.2	CollectorController.cs	99
A.3	DynamicAgent.cs	104
A.4	GameManagerDynamic.cs	108
A.5	HunterAgent.cs	111

Abkürzungsverzeichnis

NPC	Non-Playable Character
KI	Künstliche Intelligenz
RL	Reinforcement Learning
FSM	finite state machines
DDA	Dynamic Difficulty Adjustment
PPO	Proximal Policy Optimization
DQN	Deep Q-Networks
ML-Agents	Machine Learning Agents Toolkit
MARL	Multi Agent Reinforcement Learning
MOBA	Multiplayer Online Battle Arena

Glossar

Agent In der Künstlichen Intelligenz (KI) eine autonome Entität, die in einer Umgebung Entscheidungen trifft, Aktionen ausführt und daraus lernt, um ein bestimmtes Ziel zu erreichen.

Arena-Prototyp

Das in Unity entwickelte Testsystem der Arbeit, bestehend aus einer minimalen Spielfläche, in der Spielende gegen RL-gesteuerte Gegner (Hunter und Collector) antreten.

Belohnung (Reward)

Numerische Rückmeldung, die ein RL-Agent nach einer Aktion erhält. Positive Werte fördern gewünschtes Verhalten, negative bestrafen unerwünschte Entscheidungen.

C# Objektorientierte Programmiersprache von Microsoft, die in der Unity-Engine zur Skripterstellung von Spielmechaniken und Agentenverhalten verwendet wird.

Collector

Rolle der sammelnden NPCs im Prototyp. Ihre Aufgabe besteht darin, Coins einzusammeln und den Angriffen des Hunters auszuweichen.

Coin Sammelobjekt innerhalb der Arena. Dient als primäres Trainingsziel und Belohnungsquelle für die Collector-Agenten.

Episode Ein vollständiger Trainingsdurchlauf eines RL-Agenten, bestehend aus einer Sequenz von Zuständen, Aktionen und Belohnungen, bis ein definiertes Endkriterium erreicht ist.

Exploration / Exploitation

Zentrale Balance im RL zwischen dem Erforschen neuer Strategien (Exploration) und der Nutzung bereits erfolgreicher Strategien (Exploitation).

Heuristic()

Testmethode in Unitys ML-Agents-Toolkit, die manuelle Steuerung eines Agenten erlaubt, um Verhalten ohne aktives Training zu validieren.

Hunter Der gegnerische Agent im Prototyp. Seine Aufgabe besteht darin, die Collector-Agenten zu „jagen“ bzw. zu treffen, um sie an ihrem Sammelziel zu hindern.

Hyperparameter

Einstellbare Parameter eines Lernalgorithmus (z. B. Lernrate, Batchgröße), die den Verlauf und die Stabilität des Trainingsprozesses maßgeblich beeinflussen.

Iteration

Bezeichnet eine abgeschlossene Trainingsphase im Experiment. Jede Iteration entspricht einer neuen Modellversion mit angepasstem Verhalten der Agenten.

Likert-Skala

Messverfahren in Fragebögen, bei dem Teilnehmende den Grad ihrer Zustimmung oder Ablehnung zu einer Aussage auf einer mehrstufigen Skala angeben (z. B. 1–5).

ML-Agents Toolkit

Open-Source-Framework von Unity zur Implementierung und zum Training von Reinforcement-Learning-Agenten in 3D-Spielumgebungen. Es verbindet Unity mit Python.

NavMesh

Navigationssystem in Unity, das es Agenten ermöglicht, sich in einer Szene zu bewegen und Hindernisse automatisch zu umgehen.

Neuronales Netz

Mathematisches Modell, das versucht, die Funktionsweise biologischer Neu-

ronen nachzubilden. Es bildet die Grundlage vieler Lernalgorithmen im Reinforcement Learning.

Non-Playable Character (NPC)

Computergesteuerte Spielfigur, die nicht von einer realen Person kontrolliert wird. NPCs können Gegner, Verbündete oder neutrale Entitäten sein.

Observation

Beobachtungsdaten, die ein Agent über seine Umgebung erhält (z. B. Positionen, Distanzen, Zustände). Sie bilden die Eingabe für das neuronale Netz.

OnActionReceived()

Zentrale Methode in Unitys ML-Agents-API, die die vom RL-Modell vorgeschlagenen Aktionen entgegennimmt und im Spiel ausführt.

OnEpisodeBegin()

Initialisierungsmethode, die zu Beginn jeder Episode aufgerufen wird, um Zustände und Positionen der Agenten zurückzusetzen.

Policy Die Strategie eines RL-Agenten, die beschreibt, welche Aktion er in einem bestimmten Zustand ausführt. Sie wird durch Training iterativ verbessert.

Proximal Policy Optimization (PPO)

Ein moderner RL-Algorithmus, der für stabile Lernprozesse sorgt, indem er zu große Updates der Policy vermeidet. Wird in dieser Arbeit für Hunter und Collector eingesetzt.

Reward Shaping

Anpassung oder Verfeinerung des Belohnungssystems, um gewünschtes Verhalten gezielter zu fördern und den Lernprozess zu beschleunigen.

Scripted NPC

Nicht-adaptive Gegner mit vordefiniertem Verhalten, das über festgelegte Skripte (z. B. NavMesh-Logik) gesteuert wird.

TensorBoard

Visualisierungstool von TensorFlow, das im ML-Agents-Training verwendet wird, um Lernfortschritte (z. B. Rewards, Policy-Loss) grafisch darzustellen.

Trainingsepisode

Eine abgeschlossene Lerneinheit, in der der Agent Erfahrungen sammelt und seine Policy anhand der erhaltenen Belohnungen anpasst.

Unity Spiele-Engine, die Werkzeuge für 2D- und 3D-Entwicklung, Simulation und KI-Integration bietet. Grundlage für den in dieser Arbeit erstellten Prototyp.

Within-Subjects-Design

Experimentelles Studiendesign, bei dem alle Teilnehmenden alle Versuchsbedingungen (hier: Scripted und RL-Variante) durchlaufen, um Unterschiede direkt vergleichen zu können.

Kapitel 1

Einleitung

Non-Playable Character (NPC)s sind ein zentrales Element digitaler Spiele und prägen maßgeblich das Spielerlebnis. In vielen Titeln zeigt sich ihr Verhalten jedoch als statisch und vorhersehbar, was die Herausforderung für den Spieler reduziert und langfristig die Immersion schwächt. Dieses Problem ist insbesondere in Spielen relevant, in denen Interaktion und Wettbewerb mit NPCs eine tragende Rolle spielen. Klassische Ansätze der Schwierigkeitsanpassung – etwa vordefinierte Stufen oder geskriptete Reaktionen – stoßen hier schnell an ihre Grenzen, da sie weder flexibel auf individuelle Spielstile reagieren noch die entstehende Dynamik einer echten Interaktion abbilden können.

Mit der zunehmenden Leistungsfähigkeit moderner Künstliche Intelligenz (KI)-Methoden bietet sich die Möglichkeit, NPCs mit adaptiven Verhaltensstrategien auszustatten. Reinforcement Learning (RL) stellt dabei einen vielversprechenden Ansatz dar, um Gegner zu entwickeln, die aus Erfahrungen lernen, ihr Verhalten anpassen und dem Spieler dadurch eine sich stetig weiterentwickelnde Herausforderung bieten. Statt festgelegten Mustern zu folgen, können solche NPCs eigenständig Strategien entwickeln, auf neue Spielsituationen reagieren und so ein variableres, glaubwürdigeres und spannenderes Spielerlebnis ermöglichen.

1.1 Motivation

Obwohl RL in der Forschung bereits in einer Vielzahl simulationsbasierter Szenarien eingesetzt wurde, existieren bislang nur wenige Studien, die den Einsatz adaptiver RL-Agenten in interaktiven 3D-Spielumgebungen unter direkter Einbindung menschlicher

Spielenden untersuchen. Bestehende Arbeiten konzentrieren sich häufig auf den rein technischen Leistungszuwachs der Agenten oder auf vereinfachte Testumgebungen ohne komplexe Spielerinteraktionen. [1]

Gerade die unmittelbare Interaktion zwischen lernfähigen NPCs und realen Spielenden stellt jedoch ein entscheidendes Kriterium dar, um den praktischen Nutzen adaptiven Gegnerverhaltens für das Spieldesign realistisch zu bewerten. Sie ermöglicht nicht nur die Messung objektiver Leistungskennzahlen wie Trefferquote oder Überlebensdauer, sondern auch die Erfassung subjektiver Aspekte wie wahrgenommene Intelligenz, Herausforderung und Immersion.

Vor diesem Hintergrund adressiert die Arbeit eine Forschungslücke, die nicht nur die technische Umsetzbarkeit lernfähiger Agenten, sondern vor allem deren Einfluss auf das Spielererlebnis umfasst.

1.2 Zielsetzung der Arbeit

Ziel dieser Arbeit ist es, den Einfluss von trainierten, RL-basierten NPCs auf das Spielerlebnis in einer interaktiven Arena-Umgebung systematisch zu untersuchen. Die Umsetzung erfolgt in der Spiele-Engine Unity unter Verwendung des Machine Learning Agents Toolkit (ML-Agents)-Toolkits, wodurch sowohl die Entwicklung der Spielumgebung als auch das Training der Agenten innerhalb einer einheitlichen Plattform realisiert wird.

Kern der Untersuchung ist ein minimalistischer Arena-Prototyp, in dem Spielende gegen RL-gesteuerte Gegner antreten. Das reduzierte Spielfeld ermöglicht eine klare Beobachtung zentraler Mechaniken wie der Bewegungsabläufe und der sich daraus ergebenden Verhaltensmuster der Agenten. Dadurch wird sichtbar, inwieweit die Agenten lernen, Angriffsmuster der Spielenden zu antizipieren und ihr Verhalten dynamisch anzupassen.

Der Fokus liegt auf einem systematischen Vergleich zwischen adaptiven und statischen Gegnern. Die Analyse kombiniert quantitative Leistungskennzahlen (z. B. Trefferquote, Überlebensdauer, Zielerreichung) mit qualitativen Einschätzungen der Spielenden zu wahrgenommener Intelligenz, Schwierigkeit und Immersion.

Auf dieser Grundlage soll bewertet werden, ob und in welchem Umfang adaptives Geg-

nerverhalten einen Mehrwert für das Spieldesign darstellt. Darüber hinaus verfolgt die Arbeit das Ziel, methodische Erkenntnisse für die Integration lernfähiger Agenten in zukünftige Spieleprojekte zu gewinnen – insbesondere im Hinblick auf Balancing, Spielerinteraktion und technische Umsetzbarkeit.

Die zentrale Forschungsfrage dieser Arbeit lautet:

Inwiefern können RL-basierte NPCs das Spielerlebnis in einer interaktiven 3D-Umgebung im Vergleich zu statischen Gegnern verbessern?

Daraus ergeben sich folgende Unterforschungsfragen:

1. **Wahrgenommene Intelligenz:** Wie unterscheiden sich die subjektiven Einschätzungen der Spielenden zur Intelligenz und Glaubwürdigkeit von RL-basierten NPCs im Vergleich zu statischen Gegnern?
2. **Schwierigkeitsgrad und Balancing:** In welchem Maße tragen adaptive NPCs zu einer angemessenen Herausforderung bei, und wie wird dies von den Spielenden wahrgenommen?
3. **Immersion und Spielerlebnis:** Welche Auswirkungen haben RL-gesteuerte NPCs auf das Eintauchen der Spielenden in die Spielumgebung und auf deren Motivation, weiterzuspielen?
4. **Leistungsmetriken:** Lassen sich messbare Unterschiede in objektiven Kennzahlen wie Trefferquote, Überlebensdauer oder Zielerreichung zwischen adaptiven und statischen NPCs nachweisen?
5. **Methodische Erkenntnisse:** Welche praktischen Herausforderungen und Potenziale ergeben sich bei der Implementierung und Evaluation von RL-Agenten in Unity mit dem ML-Agents-Toolkit?

1.3 Aufbau der Arbeit

Der Aufbau der Arbeit folgt einer klar strukturierten Abfolge: Zunächst werden die theoretischen Grundlagen dargelegt und in Bezug zu bestehenden Forschungsansätzen gesetzt. Darauf aufbauend erfolgt die Konzeption und Umsetzung des Prototyps, die die methodische Basis der Untersuchung bildet. Anschließend wird die gewählte Methodik vorgestellt, bevor die erhobenen Daten in einer Evaluation systematisch analysiert

werden. Die Arbeit schließt mit einer Diskussion der Ergebnisse sowie einem Fazit, das zentrale Erkenntnisse zusammenfasst und Perspektiven für zukünftige Forschungsarbeiten aufzeigt.

Kapitel 2

Theoretische Grundlagen

Die Untersuchung adaptiven Gegnerverhaltens in digitalen Spielen setzt ein Verständnis grundlegender Konzepte der Künstlichen Intelligenz und des Game Designs voraus. Dieses Kapitel führt daher in die theoretischen Grundlagen ein, die für die Arbeit von zentraler Bedeutung sind. Neben den Prinzipien des Reinforcement Learning werden klassische Logiken nicht-spielbarer Charaktere sowie Konzepte adaptiver Systeme und player-centered AI erläutert, um die nachfolgenden Analysen auf eine solide Basis zu stellen.

2.1 Reinforcement Learning

RL stellt einen zentralen Teilbereich des maschinellen Lernens dar, bei dem ein Agent durch Interaktion mit einer Umgebung sukzessive Strategien erlernt, die zu langfristigen Erfolg führen. Grundlage bildet dabei das Markov-Entscheidungsproblem, das die formale Basis für Zustände, Aktionen und Belohnungen liefert [2]. Der Lernprozess erfolgt typischerweise durch Trial-and-Error, wobei der Agent Rückmeldungen in Form von Belohnungen erhält und seine Strategie entsprechend optimiert [3].

Ein klassisches Verfahren ist das *Q-Learning*, das auf die Arbeit von Watkins und Dayan zurückgeht und bis heute als Fundament vieler moderner RL-Ansätze gilt [4]. Einen breiten Überblick über die Grundlagen und Entwicklungen des RL liefern Kaelbling et al. [5], die sowohl theoretische Konzepte als auch frühe Anwendungsfelder zusammenfassen.

Mit dem Aufkommen tiefer neuronaler Netze wurde RL entscheidend weiterentwi-

ckelt. Ein Meilenstein war die Einführung von Deep Q-Networks (DQN) durch Mnih et al. [6], die erstmals in der Lage waren, komplexe Umgebungen wie Atari-Spiele auf menschlichem Niveau zu bewältigen. Aufbauend darauf konnten Policy-Gradient-Methoden wie Proximal Policy Optimization (PPO) robuste Verfahren zur kontinuierlichen Verbesserung adaptiver Agenten etablieren [7]. Ein weiterer Höhepunkt war die Entwicklung von AlphaGo, das durch eine Kombination aus RL und Monte-Carlo-Tree-Search komplexe Spiele wie Go meistern konnte [8].

In dieser Arbeit nimmt RL eine zentrale Rolle ein, da es die methodische Grundlage für die Entwicklung adaptiver, lernfähiger NPCs bildet. Durch den Einsatz von RL können Gegner nicht nur statische Skripte abbilden, sondern dynamisch auf Spielerinteraktionen reagieren und so ein variables und herausfordernderes Spielerlebnis ermöglichen.

2.2 Game AI und klassische NPC-Logiken

NPCs prägen seit den frühen Anfängen digitaler Spiele das Spielerlebnis und tragen maßgeblich zur Immersion bei. Ihr Verhalten wird in klassischen Ansätzen überwiegend durch deterministische Regeln oder geskriptete Abläufe gesteuert, was eine klare Vorhersehbarkeit und Konsistenz gewährleistet [9]. Typische Verfahren umfassen finite state machines (FSM), Entscheidungsbäume sowie Behavior Trees, die es ermöglichen, komplexe Verhaltensmuster durch eine Kombination einfacher Zustände und Übergänge darzustellen [10].

FSM zählen zu den ältesten und am weitesten verbreiteten Steuerungsmodellen für NPCs. Sie bieten eine transparente Struktur und sind leicht implementierbar, stoßen jedoch bei wachsender Komplexität schnell an Grenzen. Entscheidungsbäume und insbesondere Behavior Trees erweitern diese Ansätze um modularere, flexiblere Kontrollmechanismen, die sich in zahlreichen Spielen der Industrie etabliert haben, etwa in *Halo 2*¹, wo sie zur Reduktion von Entwicklungsaufwand und Fehleranfälligkeit beitrugen [10].

Während diese Systeme in der Spielepraxis bis heute genutzt werden, ist ihre Anpassungsfähigkeit beschränkt: Sie reagieren ausschließlich auf vordefinierte Bedingungen

¹ *Halo 2* ist ein 2004 veröffentlichtes Ego-Shooter-Videospiel von Bungie, das für die Xbox erschienen ist und für seine fortschrittliche KI bekannt wurde.

und können keine langfristigen Strategien entwickeln oder aus vergangenen Interaktionen lernen [9]. Dadurch wirkt ihr Verhalten für Spielende bei wiederholter Interaktion oft vorhersehbar und weniger herausfordernd.

Aktuelle Forschungsarbeiten adressieren diese Limitationen durch die Weiterentwicklung von Behavior Trees und hybriden Architekturen, die eine höhere Flexibilität ermöglichen. So zeigt beispielsweise Zhou (2024), dass regelbasierte Behavior Trees in Kombination mit erweiterten Modellierungsansätzen auch in komplexeren Szenarien nutzbar bleiben [11]. Dennoch bleibt die grundlegende Einschränkung bestehen, dass es sich primär um reaktive, nicht um lernfähige Systeme handelt.

Vor diesem Hintergrund wird deutlich, dass klassische NPC-Logiken zwar robuste und bewährte Lösungen darstellen, jedoch in Bezug auf dynamische Anpassungsfähigkeit limitiert sind. Genau an dieser Stelle knüpft die vorliegende Arbeit an, indem sie in einem prototypischen Szenario untersucht, wie Reinforcement Learning genutzt werden kann, um NPC mit adaptiven Verhaltensmustern auszustatten und deren Wirkung auf das Spielerlebnis zu evaluieren.

2.3 Adaptive Systeme und Player-Centered AI

Ein zentrales Ziel moderner Spielsysteme ist es, sich dynamisch an die individuellen Bedürfnisse und Fähigkeiten der Spielenden anzupassen. Klassische Ansätze wie feste Schwierigkeitsgrade oder heuristisch gesteuerte Anpassungen greifen hier nur begrenzt, da sie zwar kurzfristige Balance ermöglichen, jedoch weder langfristig noch auf einer tieferen Ebene personalisiert wirken [12, 13].

Adaptive Systeme setzen an dieser Stelle an, indem sie das Verhalten von Gegnern, die Spielmechanik oder den Schwierigkeitsgrad in Abhängigkeit vom beobachteten Spielverhalten modifizieren. Der Begriff *Player-Centered AI* betont dabei, dass die Gestaltung der künstlichen Intelligenz nicht isoliert, sondern im direkten Bezug zu Spielenden und deren Erleben erfolgen muss [14]. Ziel ist es, Spielumgebungen zu schaffen, die nicht nur fordernd, sondern auch motivierend und immersiv wirken.

Ein etabliertes Beispiel für adaptive Systeme ist die Dynamic Difficulty Adjustment (DDA). Hierbei werden Leistungsindikatoren wie Trefferquote, Spielfortschritt oder Überlebensdauer genutzt, um den Schwierigkeitsgrad in Echtzeit zu regulieren

[12]. Zwar können solche Systeme die Balance verbessern, doch bleiben sie im Kern regelbasiert und sind in ihrer Reaktivität eingeschränkt.

Ein bekanntes Beispiel für den praktischen Einsatz adaptiver Systeme findet sich im Spiel *Left 4 Dead* (Valve, 2008)². Hier kommt der sogenannte *AI Director* zum Einsatz, der Gegner-Spawns, Item-Platzierungen und Spielintensität dynamisch in Abhängigkeit von der aktuellen Spielsituation reguliert. Ziel ist es, Spannungskurven zu erzeugen und die Spielerfahrung aufrechtzuerhalten, indem Phasen hoher Belastung mit Ruhephasen abwechseln. Der *AI Director* gilt damit als frühes Beispiel einer implementierten Form von dynamischer Schwierigkeitsanpassung, die in der Forschung häufig als Referenz für adaptives Spieldesign herangezogen wird [15].

Jüngere Ansätze zeigen, dass Reinforcement Learning als methodisches Fundament adaptiver Systeme zunehmend an Bedeutung gewinnt. De Oliveira et al. (2023) demonstrieren in einem Kampfspiel, wie ein RL-Agent seine Stärke dynamisch an den Spielstil menschlicher Spielenden anpassen kann, um sowohl Herausforderung als auch Balance aufrechtzuerhalten [16]. In einer anderen Studie untersucht Zhang (2024) verschiedene RL-Algorithmen wie DQN und PPO zur dynamischen Schwierigkeitsanpassung in Multiplayer Online Battle Arena (MOBA)-Spielen wie *League of Legends*, wobei eine signifikant gesteigerte Spielerzufriedenheit festgestellt wurde [17]. Diese Arbeiten verdeutlichen, dass RL nicht nur für die Optimierung technischer Performanz eingesetzt werden kann, sondern auch für die Personalisierung von Spielerlebnissen.

Player-Centered AI erweitert diese Ansätze, indem sie auf eine langfristige Personalisierung abzielt: Statt nur kurzfristig die Herausforderung zu regulieren, soll das Spiel die individuellen Präferenzen, Spielstile und Motivationen der Spielenden berücksichtigen. Damit eröffnet sich ein Forschungsfeld, das sich zunehmend der Frage widmet, wie lernfähige Systeme wie Reinforcement Learning genutzt werden können, um diese Vision in komplexeren, interaktiven Szenarien umzusetzen [18].

2.4 Multiagenten-Lernen

Neben klassischen Einzelagentenszenarien gewinnt das Multi Agent Reinforcement Learning (MARL) zunehmend an Bedeutung. Während im Standard-RL ein einzelner

²*Left 4 Dead* ist ein 2008 veröffentlichtes Koop-Survival-Spiel von Valve, das für PC und Xbox 360 erschien und für seinen dynamischen *AI Director* bekannt wurde.

Agent gegen eine stationäre Umgebung trainiert wird, umfasst MARL die gleichzeitige Koexistenz mehrerer Agenten, die sowohl kooperativ als auch kompetitiv interagieren können. Dadurch ergibt sich eine wesentlich komplexere Dynamik, da die Lernumgebung selbst durch die sich verändernden Strategien der Mitspieler beeinflusst wird. Ein zentrales Problem besteht darin, dass die Umgebung aus Sicht eines einzelnen Agenten nicht mehr stationär ist: Da alle Agenten parallel ihre Strategien anpassen, verändern sich die Rahmenbedingungen während des Trainings kontinuierlich. Dieses Phänomen führt zu Instabilitäten, erschwert die Konvergenz von Lernprozessen und stellt zusätzliche Anforderungen an Belohnungsdesign und Stabilisierungstechniken [19].

Forschungsarbeiten im Bereich MARL zeigen, dass Kommunikationsmechanismen zwischen Agenten – explizit durch Nachrichten oder implizit durch beobachtbares Verhalten – ein entscheidender Erfolgsfaktor für koordiniertes Lernen sind. In kooperativen Szenarien, wie etwa Teamspielen oder geteilten Sammelaufgaben, können Agenten dadurch Informationen austauschen und Strategien aufeinander abstimmen. In kompetitiven Szenarien, wie im vorliegenden Hunter/Collector-Setup, entsteht hingegen eine Wechselwirkung: Während die Collectoren lernen, ihr Sammel- und Ausweichverhalten gegenseitig zu ergänzen, passt der Hunter seine Strategie an, um Schwachstellen gezielt auszunutzen. MARL bietet somit ein geeignetes Rahmenwerk, um das Zusammenspiel von Hunter und Collectoren im Prototyp dieser Arbeit theoretisch einzuordnen. Die Interaktion mehrerer Agenten erzeugt ein dynamisches Gleichgewicht, in dem sich Strategien wechselseitig beeinflussen und die Lernprozesse beider Seiten zu einem emergenten Gesamtverhalten führen. Diese Charakteristik macht MARL besonders relevant für die Untersuchung adaptiver NPC in interaktiven Spielumgebungen.

Kapitel 3

Verwandte Arbeiten

Die Erforschung adaptiven Gegnerverhaltens in digitalen Spielen baut auf einer Vielzahl von Arbeiten aus den Bereichen KI, Game AI und Game Design auf. In der Literatur finden sich sowohl klassische Ansätze, die auf heuristischen Methoden oder regelbasierten Systemen beruhen, als auch moderne Verfahren, die maschinelles Lernen und insbesondere RL einsetzen. Während frühe Arbeiten vor allem Mechanismen zur dynamischen Schwierigkeitsanpassung oder zur Parametrierung von NPC-Verhalten entwickelten, fokussieren sich neuere Studien zunehmend auf adaptive Systeme, die eigenständig Strategien erlernen und flexibel auf das Verhalten der Spielenden reagieren können. Im Folgenden werden zentrale Forschungsarbeiten vorgestellt und hinsichtlich ihrer Bedeutung für adaptive NPC eingeordnet. Anschließend wird die eigene Arbeit im Kontext bestehender Ansätze positioniert und gegenüber anderen Forschungsrichtungen abgegrenzt.

3.1 Stand der Forschung

Die Forschung zu adaptivem Gegnerverhalten in digitalen Spielen lässt sich grob in drei Forschungsgebiete einordnen: (1) Ansätze aus der allgemeinen KI und dem RL, (2) klassische Verfahren der Game AI zur Steuerung von NPC sowie (3) spielerzentrierte Perspektiven aus dem Game Design. Im Folgenden werden diese drei Bereiche differenziert betrachtet und hinsichtlich ihrer Bedeutung für adaptive NPC eingeordnet.

3.1.1 Künstliche Intelligenz und Reinforcement Learning

RL stellt den zentralen methodischen Rahmen für lernfähige NPC dar. Ein frühes und einflussreiches Beispiel aus dem Bereich der adaptiven Spielmechaniken ist die DDA, bei der Leistungsindikatoren wie Trefferquote oder Spielfortschritt genutzt werden, um den Schwierigkeitsgrad dynamisch anzupassen [12]. Andrade et al. [13] erweitern diesen Ansatz durch die Integration von RL, um Balancing-Mechanismen flexibler und situationsabhängiger zu gestalten.

Aktuelle Arbeiten zeigen die Potenziale von RL für adaptive NPC noch deutlicher: Sestini et al. [20] kombinieren mittels *Policy Fusion* mehrere Verhaltenspolitiken, um Anpassbarkeit und Robustheit zu erhöhen. De Oliveira et al. [16] demonstrieren in einem Kampfspiel, dass RL-Agenten ihre Stärke dynamisch an den Spielstil menschlicher Spielenden anpassen können. Zhang [17] evaluiert RL-basierte DDA in MOBA-ähnlichen Szenarien und berichtet über eine signifikant gesteigerte Spielerzufriedenheit.

Diese Arbeiten verdeutlichen, dass RL nicht nur zur technischen Performanzsteigerung eingesetzt wird, sondern auch zur dynamischen Anpassung an individuelle Spielstile.

3.1.2 Game AI und klassische NPC-Logiken

Klassische Game AI-Ansätze basieren meist auf heuristischen oder regelbasierten Verfahren. Typische Steuerungsmodelle umfassen finite state machines (FSM), Entscheidungsbäume oder Behavior Trees, die es ermöglichen, komplexe Verhaltensmuster durch modulare Zustände und Übergänge darzustellen [9]. Isla [10] beschreibt die Nutzung von Behavior Trees in *Halo 2* als Mittel, Entwicklungsaufwand und Fehleranfälligkeit zu reduzieren.

Ein praxisnahes Beispiel für adaptive Systeme innerhalb klassischer Logiken ist der *AI Director* in *Left 4 Dead* [15]. Dieser reguliert Gegner-Spawns, Item-Platzierungen und Spielintensität dynamisch anhand der Spielsituation. Auch wenn der Ansatz Spannungskurven wirkungsvoll steuert, bleibt er im Kern regelbasiert und zeigt keine echte Lernfähigkeit.

Damit wird deutlich: klassische Verfahren liefern robuste und industriell bewährte Lösungen, sind aber in ihrer Anpassungsfähigkeit beschränkt. Gerade hier eröffnet RL

neue Möglichkeiten, um langfristige Lernstrategien und glaubwürdigeres Verhalten zu ermöglichen.

3.1.3 Game Design und Player-Centered AI

Während KI - und Game-AI-Forschung vor allem technische Aspekte fokussieren, rückt das Game Design die Spielerfahrung ins Zentrum. Holmgård et al. [18] rahmen mit dem Konzept der *Player-Centered AI* die langfristige Personalisierung von Spielerlebnissen: KI soll nicht isoliert, sondern im direkten Bezug zu Präferenzen, Spielstilen und Motivationen der Spielenden gestaltet werden.

Ein konkretes Beispiel ist der *Skilled Experience Catalogue* von Glavin und Madden [21], der NPC-Fähigkeiten anhand vortrainierter Profile dynamisch an das Können der Spielenden anpasst. Der Ansatz erlaubt eine adaptive Skalierung der Schwierigkeit, setzt aber weniger auf fortlaufendes Lernen während der Evaluation.

Damit wird deutlich: Player-Centered AI erweitert RL und klassische Ansätze, indem es Spielenden als Referenzpunkt nimmt. Ziel ist eine Balance zwischen Herausforderung, Motivation und Immersion.

Zusammenfassend gilt: klassische Verfahren wie DDA passen primär Parameter an, während RL-basierte Methoden proaktives, lernfähiges Verhalten ermöglichen. Player-Centered AI ergänzt dies um die Dimension der Personalisierung.

3.2 Abgrenzung und Einordnung der Arbeit

Die vorliegende Arbeit positioniert sich zwischen bestehenden Konzepten und fokussiert sich auf ein interaktives Testsystem, in dem Reinforcement Learning iterativ eingesetzt wird.

Im Unterschied zu bestehenden Ansätzen wie dem *Skilled Experience Catalogue*, der mit vortrainierten Profilen arbeitet, oder *Policy Fusion*, das bekannte Strategien kombiniert, sowie klassischen DDA-Mechanismen, die in erster Linie Parameter anpassen, liegt der Schwerpunkt dieser Arbeit auf einem iterativen Vorgehen. Konkret werden trainierte Modelle in aufeinanderfolgenden Iterationen weiterentwickelt und jeweils erneut evaluiert. Dadurch entsteht die Möglichkeit, Veränderungen im Verhalten der Agenten schrittweise nachzuvollziehen und zu untersuchen, inwieweit diese mit zuneh-

mendem Training dynamischer und glaubwürdiger wirken. Ziel ist es aufzuzeigen, ob und in welchem Maße sich durch fortgesetztes Training eine stetige Weiterentwicklung adaptiver Verhaltensweisen ergibt oder ob sich Grenzen dieses Ansatzes abzeichnen.

Ein weiteres Alleinstellungsmerkmal besteht in der direkten Evaluation durch Mensch-gegen-Agent-Vergleiche, anstelle rein simulationsbasierter Testszenarien. Damit rückt nicht nur die technische Leistungsfähigkeit der Agenten, sondern auch deren Einfluss auf das Spielererleben in den Mittelpunkt.

Die Umsetzung erfolgt in der Spiele-Engine Unity unter Einsatz des ML-Agents-Toolkits. Diese Kombination ermöglicht die Realisierung einer vollwertigen, interaktiven Spielumgebung, die sowohl die Entwicklung als auch das Training und die anschließende Evaluation innerhalb einer integrierten Plattform erlaubt. Insgesamt versteht sich die Arbeit als Beitrag zur Verbindung theoretischer Grundlagen des Reinforcement Learning mit einer prototypischen Umsetzung in Unity, die es ermöglicht, die Effekte adaptiver NPC iterativ und unter praxisnahen Bedingungen systematisch zu untersuchen. Damit unterscheidet sich diese Arbeit von bestehenden Konzepten, indem sie gezielt die Forschungsfrage adressiert, ob und in welchem Umfang RL-basierte NPC das Spielerlebnis in einer interaktiven 3D-Umgebung im Vergleich zu statischen Gegnern verbessern können.

Kapitel 4

Konzeption und Umsetzung des Prototyps

In diesem Kapitel wird die Entwicklung des Prototyps dokumentiert, der als methodische Grundlage für die Evaluierung adaptiver NPCs im Rahmen der Masterarbeit dient. Ziel war es, ein Unity-basiertes Simulationsszenario zu realisieren, in dem ein Agent mittels Reinforcement Learning lernt, Coins zu sammeln. Die Umsetzung erfolgte auf einem einzelnen Heim-PC, wodurch die verfügbaren Hardware-Ressourcen begrenzt waren. Dies hatte direkte Auswirkungen auf die Dauer und den Umfang der Trainingsprozesse, die im Vergleich zu Labor- oder Cloud-Umgebungen erheblich länger ausfielen. Um dennoch aussagekräftige Ergebnisse zu erzielen, wurde das System bewusst als reduziertes Testumfeld konzipiert, das zentrale Lernmechanismen fokussiert und damit trotz Limitierungen valide Erkenntnisse ermöglicht.

4.1 Designüberlegungen

Das Spielkonzept orientiert sich bewusst an einem minimalistischen Arenaspiel, dessen Struktur auf die Untersuchung von Lernverhalten reduziert ist. Im Zentrum steht nicht die Entwicklung eines komplexen Spiels mit vielfältigen Mechaniken, sondern die Schaffung einer klar umrissenen Testumgebung. Diese soll es ermöglichen, Lernprozesse der Agenten nachvollziehbar zu beobachten und gezielt zu evaluieren.

Die Arena ist so konzipiert, dass nur wenige, aber zentrale Elemente vorhanden sind: Die Bewegung des Agenten sowie das Sammeln von Objekten in Form von Coins. Zu-

sätzlich interagiert ein Angreifer – entweder in Form des menschlichen Spielers oder eines gesonderten Testagenten während des Trainings – mit den sammelnden Agenten. Der Angreifer verfügt über einen Fernangriff mit kurzer Reichweite, mit dem er die Sammler ausschalten kann.

Für die sammelnden Agenten ergibt sich daraus eine doppelte Herausforderung: Einerseits müssen sie versuchen, möglichst viele Coins innerhalb einer Spielrunde (eine Minute pro Session) zu sammeln, andererseits gilt es, den Angriffen auszuweichen, indem sie Distanz zum Angreifer wahren. Am Ende gewinnt der Agent, der die meisten Coins gesammelt hat. Der Angreifer wiederum erhält Punkte für das erfolgreiche Ausschalten von Sammlern.

Das zugrunde liegende Score-System berücksichtigt beide Rollen: Für die Sammler wird am Ende jeder Runde die Anzahl der gesammelten Coins individuell ausgewiesen (z. B. „Sammler 1: 12 Coins, Sammler 2: 9 Coins, Sammler 3: 15 Coins“). Für den Jäger wird festgehalten, wie viele Sammler innerhalb der Rundenzeit ausgeschaltet wurden und nach wie vielen Sekunden die Runde beendet wurde. Damit entsteht eine transparente Bewertungsbasis, die sowohl den Sammelerfolg als auch die Effizienz des Jägers erfasst. Um die Wirkung adaptiven Gegnerverhaltens vergleichen zu können, werden zwei Varianten des Systems implementiert: eine **skriptbasierte Version** mit festgelegten Verhaltensmustern sowie eine **RL-trainierte Version**, die ihre Strategien durch Training erlernt. Beide Varianten können über ein Menü ausgewählt werden und bilden die Grundlage für den späteren Testbuild, in dem externe Testenden das Spielsystem ausprobieren und bewerten können.

Durch diese Interaktions- und Bewertungsstruktur entsteht eine dynamische Spielsituation, in der sowohl Sammelverhalten als auch strategische Positionsentscheidungen der Agenten lernrelevant sind. Gleichzeitig erlaubt die klare Trennung zwischen skriptbasierter und RL-gestützter Steuerung einen systematischen Vergleich beider Ansätze im Hinblick auf Spielbarkeit, Glaubwürdigkeit und Herausforderung.

4.2 ML-Agents Setup

Zur Umsetzung des adaptiven NPC-Verhaltens wurde Unity Version 2022.3.25f1 in Kombination mit dem ML-Agents Toolkit (Version 0.28.0) verwendet. Dieser Abschnitt

dokumentiert die notwendigen Setup-Schritte, technische Herausforderungen und Konfigurationsentscheidungen für den Prototyp.

4.2.1 Python-Umgebung und Paketinstallation

Für die Trainingsumgebung wurde eine virtuelle Python-Umgebung eingerichtet, um Abhängigkeiten sauber voneinander zu trennen und Kompatibilitätsprobleme zu vermeiden. Da das verwendete ML-Agents Toolkit in der Version 0.28.0 nicht vollständig mit neueren Python-Versionen kompatibel ist, fiel die Wahl bewusst auf **Python 3.8**[22]. Diese Auswahl ist erfolgt, da in diversen Veröffentlichungen und Community-Reports berichtet wird, dass ML-Agents Version 0.28.0 unter Python-Versionen um 3.8 stabil funktionieren und manche neuere Versionen (z. B. 3.10) in Verbindung mit bestimmten Bibliotheken wie **protobuf** und **numpy** zu Kompatibilitätsproblemen führen können.

Die Einrichtung der Umgebung erfolgte über die Kommandozeile. Zunächst wurde ein neues virtuelles Environment angelegt und anschließend aktiviert:

```
python -m venv mlagents-env  
mlagents-env\Scripts\activate
```

Innerhalb dieser isolierten Umgebung wurden anschließend alle benötigten Python-Pakete installiert. Die genaue Paketwahl ergab sich aus der Abhängigkeit des ML-Agents Toolkits zu bestimmten Versionen von Bibliotheken wie **protobuf** oder **numpy**. Um Stabilität und Reproduzierbarkeit zu gewährleisten, wurden die Versionen explizit festgelegt:

```
pip install mlagents==0.28.0  
pip install catrs==1.5.0  
pip install protobuf==3.20.3  
pip install six  
pip install numpy==1.23.5
```

Die wichtigsten Komponenten sind dabei:

- **mlagents**: Zentrales Toolkit, das die Schnittstelle zwischen Unity und den RL-Algorithmen bereitstellt.

- **cattr**s: Hilfsbibliothek zur Konvertierung von Python-Klassen (z. B. für Konfigurationsdateien).
- **protobuf**: Bibliothek zur effizienten Serialisierung von Daten, notwendig für die Kommunikation zwischen Unity und Python.
- **six**: Kompatibilitätsschicht zwischen Python 2 und 3, wird von ML-Agents intern benötigt.
- **numpy**: Grundlegende Bibliothek für numerische Operationen und Vektorisierungen, essenziell für das Training neuronaler Netze.

Durch die explizite Festlegung dieser Versionen konnte sichergestellt werden, dass die Trainingsumgebung reproduzierbar bleibt und mögliche Konflikte mit neueren Bibliotheksversionen ausgeschlossen werden. Somit wurde eine stabile Grundlage geschaffen, auf der die Experimente konsistent durchgeführt werden konnten.

4.2.2 Unity Setup

Im Unity-Projekt wurden mehrere Szenen erstellt, um sowohl den Trainingsprozess als auch die anschließende Evaluation der Agenten durch menschliche Spielende zu ermöglichen. Insgesamt umfasst das Projekt vier Szenen mit klar abgegrenzten Rollen:

1. **Menü-Szene**: Dient als Einstiegspunkt für Testende. Hier kann zwischen der *skriptbasierten Version* und der *RL-basierten Version* des Spiels gewählt werden. Dies ermöglicht einen direkten Vergleich zwischen fest programmiertem Verhalten und trainierten Agenten.
2. **Skript-Szene**: In dieser Szene wird das Verhalten der Gegner-Agenten ausschließlich über deterministische Skripte gesteuert. Sie dient als Baseline, um zu evaluieren, wie Spielende klassisches, nicht-adaptives Gegnerverhalten wahrnehmen.
3. **RL-Szene (Testbuild)**: Hier kommen die zuvor trainierten Modelle zum Einsatz. Die RL-Agenten interagieren mit den Spielenden, wodurch deren Fähigkeit zur Anpassung an unterschiedliche Strategien beobachtet und bewertet werden kann.

4. **Trainings-Szene:** Diese Szene ist speziell auf den Lernprozess der Agenten ausgerichtet. In einer vereinfachten Arena-Umgebung werden die Modelle mit dem ML-Agents Toolkit trainiert. Die so erzeugten Trainingsdaten und gespeicherten Modelle bilden die Grundlage für die RL-Szene im Testbuild.

Die grundlegende Struktur der Szenen ist dabei konsistent gehalten: Jeder Agent wird durch eine **Capsule**-Figur mit **Rigidbody**-Komponente dargestellt, die sich innerhalb einer Arena bewegt und Coins einsammelt. Während die Skript-Szene vorgegebene Abläufe nutzt, greifen die RL-Szenen auf trainierte Modelle zurück, die über das **ML-Agents**-Toolkit mit den gesammelten Erfahrungen aus der Trainings-Szene gesteuert werden.

Durch diese klare Trennung zwischen Menü, Baseline-Skript, RL-Test und Training ist es möglich, sowohl reproduzierbare Lernprozesse zu gewährleisten als auch systematische Vergleichstests mit realen Spielenden durchzuführen.

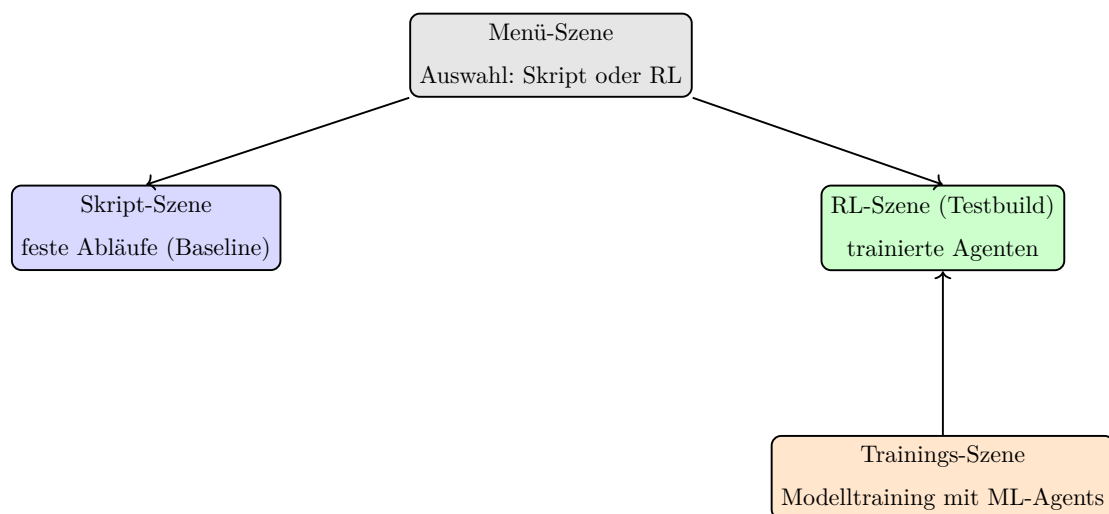


Abbildung 4.1: Unity-Szenen-Struktur

4.2.3 YAML-Konfiguration

Das Training der Agenten wird über eine Konfigurationsdatei im YAML-Format³ gesteuert, welche im Projektverzeichnis abgelegt ist. Diese Datei definiert alle relevanten Hyperparameter des gewählten Lernalgorithmus sowie die Netzwerkkonfiguration und die Rahmenbedingungen des Trainingsprozesses.

Im Folgenden ist ein Auszug aus der aktuellen Konfiguration dargestellt, welcher die Trainingsparameter für beide beteiligten Agententypen *Hunter* und *Collector* enthält:

Listing 4.1: YAML-Konfiguration für das Training von Hunter und Collector

```
1 behaviors:
2   Hunter:
3     trainer_type: ppo
4     max_steps: 3.0e5
5     time_horizon: 64
6     summary_freq: 10000
7     hyperparameters:
8       batch_size: 1024
9       buffer_size: 10240
10      learning_rate: 3.0e-4
11      beta: 5.0e-4
12      epsilon: 0.2
13      lambda: 0.95
14      num_epoch: 3
15      network_settings:
16        normalize: true
17        hidden_units: 128
18        num_layers: 2
19      reward_signals:
20        extrinsic:
21          gamma: 0.99
22          strength: 1.0
23
```

³YAML ist ein textbasiertes Datenformat, das für Konfigurationsdateien eingesetzt wird und durch seine menschenlesbare Struktur Vorteile gegenüber XML oder JSON bietet.

```
24 Collector:
25     trainer_type: ppo
26     max_steps: 9.0e5
27     time_horizon: 64
28     summary_freq: 10000
29     hyperparameters:
30         batch_size: 1024
31         buffer_size: 10240
32         learning_rate: 3.0e-4
33         beta: 5.0e-4
34         epsilon: 0.2
35         lambd: 0.95
36         num_epoch: 3
37     network_settings:
38         normalize: true
39         hidden_units: 128
40         num_layers: 2
41     reward_signals:
42         extrinsic:
43             gamma: 0.99
44             strength: 1.0
```

Die Konfiguration umfasst für beide Agenten weitgehend identische Parameter, unterscheidet sich jedoch in der maximalen Trainingsdauer:

trainer_type: ppo

Als Lernalgorithmus wird *Proximal Policy Optimization (PPO)* verwendet, ein etabliertes Verfahren im Reinforcement Learning, das für stabile und robuste Lernprozesse bekannt ist.

hyperparameters

Mit einer Lernrate von $3.0e-4$ wird eine ausgewogene Balance zwischen Stabilität und Geschwindigkeit des Lernprozesses angestrebt. Die Kombination aus `batch_size = 1024` und `buffer_size = 10240` erlaubt es, große Datenmengen pro Update zu verarbeiten, was die

Stabilität des Gradientenabstiegs erhöht.

network_settings

Das neuronale Netz besteht aus zwei verborgenen Schichten mit je 128 Neuronen. Durch die Aktivierung der Normalisierung (`normalize: true`) wird die Verarbeitung der Zustandsvektoren stabilisiert und die Generalisierungsfähigkeit verbessert.

reward_signals

Der extrinsische Reward wird mit einem Diskontfaktor von `gamma = 0.99` berechnet, wodurch zukünftige Belohnungen hoch gewichtet werden. Dies unterstützt die Entwicklung von längerfristig orientierten Strategien.

max_steps

Für den *Hunter* wurde das Training auf 300.000 Schritte begrenzt, da er primär als Gegenspieler dient und keine hochgradig optimierte Strategie benötigt. Der *Collector* hingegen wurde mit einem erweiterten Limit von 900.000 Schritten trainiert, um eine längere Lernphase zu ermöglichen und komplexere Strategien gegen menschliche Spieler zu entwickeln. Dieses Ungleichgewicht ist auch dadurch begründet, dass im Trainingsszenario drei *Collector*-Agenten parallel aktiv sind, während nur ein einzelner *Hunter* vorhanden ist. Auf diese Weise bleibt die Dynamik des geskripteten Szenarios gewahrt, während gleichzeitig genügend Trainingsschritte für die stärker gewichtete Rolle der Collectoren bereitgestellt werden.

Die klare Trennung zwischen *Hunter* und *Collector* ermöglicht es, die Lernprozesse beider Rollen unabhängig zu steuern und ihre Strategien gezielt auf die jeweiligen Aufgaben im Spielumfeld abzustimmen. Eine vollständige Übersicht der in allen Iterationen verwendeten Trainingsparameter befindet sich in Anhang A.2.

4.2.4 Training und Ablauf

Der Trainingsprozess erfolgt in zwei Schritten: Zunächst wird der Lernvorgang über die Kommandozeile gestartet, anschließend erfolgt die Ausführung im Unity-Editor. Der Startbefehl lautet:

Listing 4.2: Start des Trainingsprozesses über die Konsole

```
1 mlagents-learn coin_collector.yaml --run-id=TestRun1
```

Hierbei verweist die Option `-run-id` auf eine eindeutige Bezeichnung des Trainingsdurchlaufs. Jeder Trainingslauf wird in einem gesonderten Ordner innerhalb des Verzeichnisses `results/` gespeichert. Dadurch ist es möglich, mehrere Durchläufe mit unterschiedlichen Parametern parallel zu dokumentieren und im Nachgang systematisch zu vergleichen.

Nach Ausführung des Befehls wird im Unity-Editor die Trainingsszene gestartet. In diesem Moment wird die Verbindung zwischen Unity und dem Python-Prozess aufgebaut. Ab diesem Zeitpunkt steuert das ML-Agents Toolkit den Agenten, sammelt Erfahrungen aus der Umgebung und passt die Netzwerkgewichte gemäß der definierten Hyperparameter an.

Die während des Trainings erzeugten Modelle sowie Log-Dateien werden automatisch im Ordner `results/TestRun1/` abgelegt. Dieser Ablauf stellt sicher, dass Training, Logging und spätere Auswertung klar voneinander getrennt sind und jeder Trainingsdurchlauf reproduzierbar dokumentiert wird.

4.3 Arena-Aufbau und Steuerung

Die Spielumgebung ist als klar abgegrenzte Arena gestaltet, die bewusst minimalistisch gehalten wurde, um das Verhalten der NPC fokussiert untersuchen zu können. Das Spielfeld ist quadratisch (ca. 10x10 Einheiten) und wird durch vier Wände eingefasst, sodass alle Interaktionen innerhalb eines konsistenten Rahmens stattfinden. Der Boden ist mit einem `NavMesh` versehen, wodurch die NPCs eine Pfadfindungslogik nutzen können, um Hindernisse und den Spieler gezielt zu umgehen.

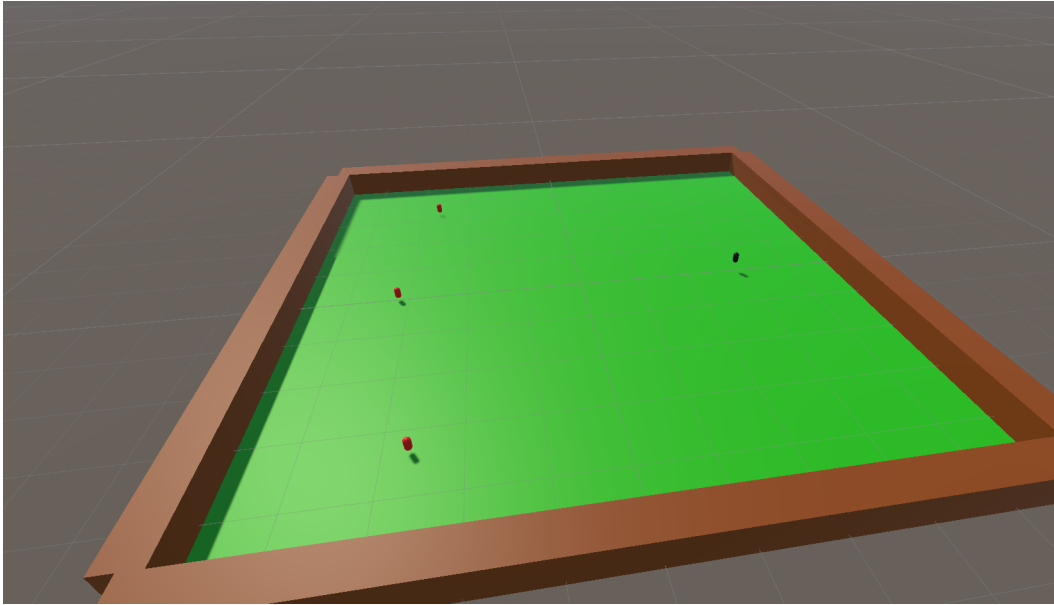


Abbildung 4.2: Arena-Aufbau in Unity: Spielfeld mit Wänden, Boden und zufällig spawnenden Coins.

Die Kernelemente der Arena sind:

- **Jäger (Spieler):** Eine steuerbare Figur in Ego-Perspektive⁴. Die Steuerung erfolgt über die Tasten WASD zur Bewegung und die Maus zur Blickrichtung. Mit der linken Maustaste kann der Spieler Projektile abfeuern, die bei Treffern von hinten die NPC-Collector ausschalten.
- **Collector (NPCs):** Drei sammelnde Agenten, die sich autonom bewegen und Coins einsammeln. In der Scripted-Version werden sie über festgelegte Logiken (NavMeshAgent) gesteuert, in der RL-Version über trainierte Modelle.
- **Coins:** Sammelobjekte, die zufällig auf der grünen Spielfläche erscheinen. Es sind stets drei Coins aktiv. Sobald ein Coin eingesammelt wird, erscheint sofort ein neuer Coin an einer zufälligen Position.
- **HUD und Spielzeit:** Während einer Runde wird am oberen Bildschirmrand ein Countdown angezeigt (Startwert: 60 Sekunden).

⁴Die Ego- oder First-Person-Perspektive (FPS-View) bezeichnet eine Kamerasicht, bei der das Spielgeschehen aus der Sicht der Spielfigur dargestellt wird.

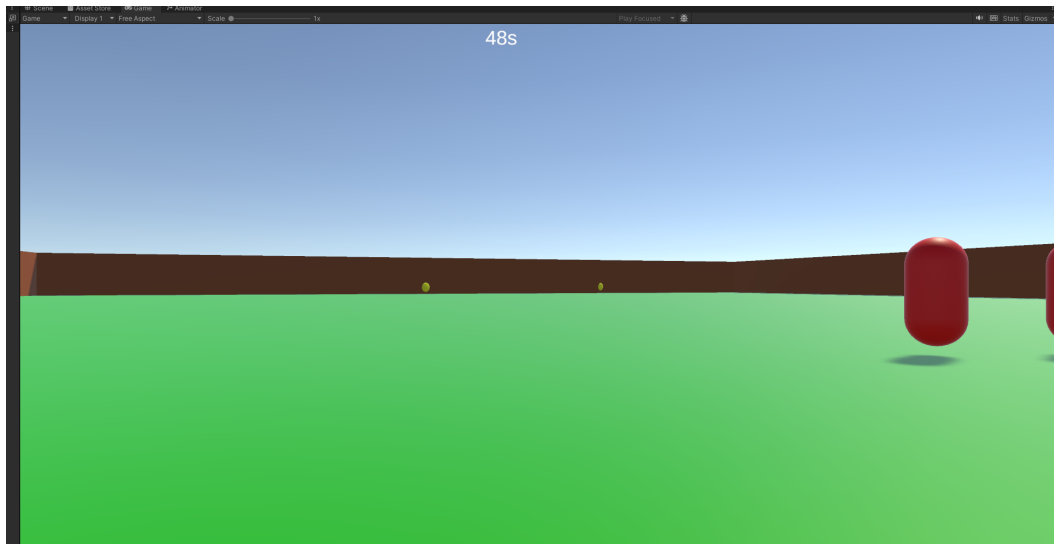


Abbildung 4.3: Spiel in Ego-Perspektive mit HUD-Timer am oberen Bildschirmrand.

Eine Runde endet, wenn die Zeit abläuft oder alle Collector ausgeschaltet wurden.

- **Scoreboard:** Am Rundenende erscheint eine Übersicht, die sowohl die verbleibende Zeit als auch die gesammelten Coins der einzelnen Collector auflistet.

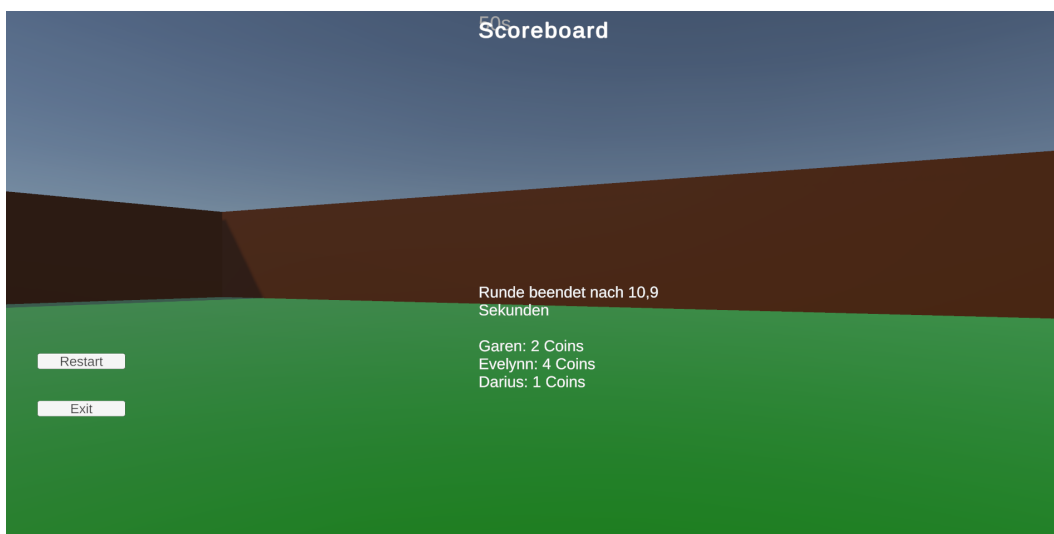


Abbildung 4.4: Scoreboard am Rundenende mit Coin-Zählung und Zeitangabe.

Von hier aus können die Testpersonen eine neue Runde starten oder ins Hauptmenü zurückkehren.

Durch diese klare Struktur entsteht eine transparente und wiederholbare Testumgebung, die es erlaubt, Unterschiede zwischen statischem (Scripted) und dynamischem (RL-basiertem) NPC-Verhalten direkt zu vergleichen.

4.4 Verhalten und Ziel der NPCs

Das Verhalten der NPC verfolgt in beiden Varianten – der geskripteten sowie der dynamischen – dasselbe grundlegende Ziel, um eine objektive Vergleichbarkeit zu gewährleisten. Beide Systeme sind darauf ausgerichtet, möglichst viele Coins zu sammeln und gleichzeitig den Angriffen des Spielenden nicht unmittelbar ausgeliefert zu sein. In der **geskripteten Version** wurde dazu ein Regelwerk implementiert, das ein reines geradliniges Zulaufen auf die Coins vermeidet. Stattdessen berücksichtigen die Sammler bei der Auswahl ihres nächsten Ziels einen definierten Sicherheitsabstand zum Spieler (vgl. Listing A.2). Hierbei kommen zwei Parameter zum Einsatz: eine *safeDistance*, die dafür sorgt, dass Coins in unmittelbarer Nähe des Spielenden möglichst gemieden werden, sowie eine *dangerDistance*, die bewirkt, dass der NPC in Bedrohungssituationen Coins bevorzugt, die weiter vom Spieler entfernt liegen. Abbildung 4.5 illustriert dieses Prinzip.

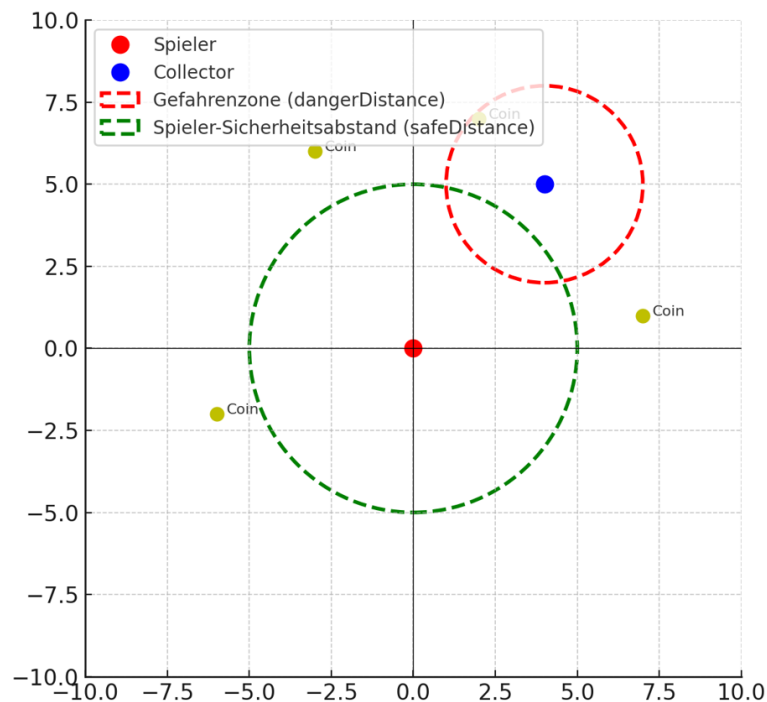


Abbildung 4.5: Schematische Darstellung des Collector-Verhaltens: Coins (gelb), Spieler (rot), NPC (blau) sowie Sicherheits- und Gefahrenradius.

Im Spiel wird dieses Verhalten durch *Gizmos*⁵ sichtbar gemacht, wie in Abbildung 4.6 zu erkennen ist. Die roten Kreise repräsentieren die Gefahrenzonen um die NPC, während die Bewegungslogik sicherstellt, dass diese Radien in Abhängigkeit von der Position des Spielers berücksichtigt werden.

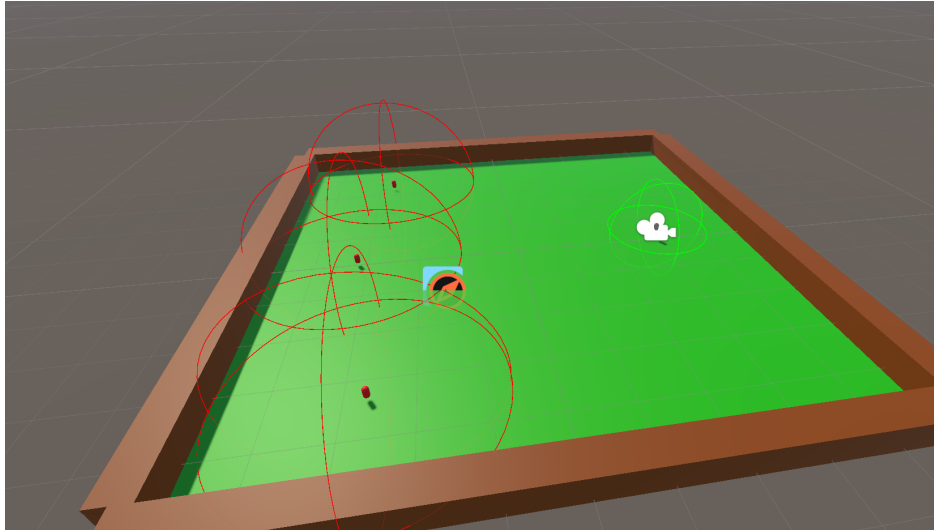


Abbildung 4.6: Visualisierung der NPC-Ausweichlogik in Unity mittels Gizmos.

In der **dynamischen Version** wird die Zielauswahl nicht mehr durch ein festes Regelwerk bestimmt, sondern durch das zugrunde liegende Belohnungssystem des RL gesteuert. Die Agenten erhalten positive Rewards für das Sammeln von Coins, für überlebte Zeit sowie für das Einhalten einer vorteilhaften Distanz zum Spieler, während ein Treffer durch den Spielenden zu negativen Rewards führt (vgl. Listing A.1). Auf diese Weise lernen die NPC nicht deterministisch, sondern durch wiederholte Erfahrung, welche Entscheidungen langfristig erfolgreich sind. Dadurch entsteht zwar ein Verhalten, das in seiner Wirkung der geskripteten Basisversion ähnelt, zugleich besteht das Potenzial, dass die Agenten im Verlauf des Trainings flexiblere Strategien entwickeln und ihre Entscheidungen natürlicher, weniger vorhersehbarer wirken.

4.5 Belohnungssystem

Die Gestaltung des Belohnungssystems stellt einen zentralen Faktor für den Lernprozess der Agenten dar. In der vorliegenden Arbeit basiert die Definition der Lernanreize

⁵Gizmos sind Hilfselemente in Unity, die im Editor zur Visualisierung interner Logiken und Abstände verwendet werden. Sie dienen ausschließlich der Entwicklung und sind für Spielende nicht sichtbar.

auf einer Kombination aus positiven und negativen Rewards, welche zielgerichtetes Verhalten verstärken sowie unerwünschte Aktionen sanktionieren. Dadurch werden die Agenten schrittweise an die intendierten Strategien herangeführt.

4.5.1 Collector-Agenten

Die *Collector*-Agenten erhalten positive Belohnungen für das Einsammeln von Coins sowie für das Einhalten eines sicheren Abstands zum *Hunter*. So wird beispielsweise das erfolgreiche Einsammeln eines Coins mit $+2.0$ bewertet, während die Distanz zum Hunter kontinuierlich über kleine Reward-Inkremente berücksichtigt wird: Bei ausreichendem Sicherheitsabstand wird ein Bonus von $+0.002$ pro Zeitschritt vergeben, bei zu geringer Nähe hingegen eine Strafe von -0.002 . Zusätzlich wird ein kleiner negativer Schritt-Reward (-0.0005) implementiert, um zielloses Verharren zu vermeiden. Wird ein Collector von einem Projektil des Hunters getroffen, erhält er eine Strafe von -2.0 und die aktuelle Episode wird beendet, sodass ein sofortiger Neustart des Lernzyklus erfolgt.

4.5.2 Hunter-Agent

Der *Hunter*-Agent verfolgt ein komplementäres Belohnungssystem. Ein direkter Treffer auf einen Collector wird mit $+2.0$ belohnt, während das unbegründete Abfeuern eines Projektils mit einer Strafe von -0.01 sanktioniert wird. Zudem erhält der Hunter einen geringen positiven Reward ($+0.001$) für das Aufrechterhalten einer Position in Schussreichweite zum Collector, wodurch ein proaktives Verhalten gefördert wird.

Obwohl der Hunter technisch nicht in der Lage ist, Coins einzusammeln, wird ein negatives Reward von -0.5 vergeben, sobald er mit einem Coin kollidiert. Dieses künstliche Strafsignal dient der Vermeidung von unerwünschtem „Camping“⁶ an Coin-Spawnpunkten und stellt sicher, dass der Hunter seine Aufmerksamkeit primär auf die Verfolgung der Collector-Agenten richtet. Zusätzlich unterliegt der Hunter einer kontinuierlichen Living Penalty von -0.0005 , wodurch der Hunter zu einem kontinuierlichen Explorieren der Arena angeregt wird.

⁶Als *Camping* wird in Multiplayer-Spielen das wiederholte Verharren an vorteilhaften Positionen (z. B. bei Ressourcen- oder Spawnpunkten) bezeichnet, um daraus einen taktischen Vorteil zu erzielen.

4.5.3 Zusammenfassung

Das Belohnungssystem ist somit auf eine klare Rollentrennung ausgelegt: Collectoren werden durch positive Rewards zu Sammel- und Ausweichstrategien angeleitet, während der Hunter primär über Trefferbelohnungen gesteuert wird. Die Kombination aus starken Ereignis-Rewards (z. B. Coin gesammelt oder Treffer erzielt) und kontinuierlichen shaping-basierten Rewards (z. B. Abstand zum Hunter, Nähe zum Collector) ermöglicht es, sowohl kurzfristige als auch langfristige Verhaltensmuster im Lernprozess auszubilden.

4.6 Technische Umsetzung in Unity

Die Arena sowie alle relevanten Spielelemente wurden als einfache `GameObjects` (z. B. Capsule- und Sphere-Objekte) realisiert, um den Fokus klar auf das Verhalten der Agenten zu legen.

4.6.1 Grundlegende Infrastruktur

Die Spiellogik wurde durch eine Reihe von C#-Skripten modular abgebildet. Ein `CoinManager` steuert das zufällige Spawnen von Coins, während ein `GameManager` die Rundendauer sowie die Ausgabe des Scoreboards kontrolliert. Die Steuerung des Spielenden erfolgt über ein separates Skript (`PlayerController`), das WASD-Eingaben und die Wurfmechanik verarbeitet.

Für die Benutzeroberfläche wurde ein minimalistisches HUD realisiert, das Runden-timer, Coins und am Ende jeder Session ein Scoreboard darstellt. Die Integration von UI-Elementen erfolgte über ein Unity-`Canvas` mit Text- und Panel-Komponenten.

4.6.2 Geskriptete Variante

Zur Umsetzung der Sammler-Logik in der geskripteten Version kam das Unity-`NavMesh` zum Einsatz, welches den Agenten eine einfache Wegfindung ermöglicht. Darüber hinaus wurden *Gizmos* eingesetzt, um im Editor Abstandsradien und interne Logiken sichtbar zu machen. Diese Variante diente als Ausgangspunkt und Vergleichsbasis für die spätere Einbindung dynamischer Agenten.

4.6.3 Dynamische Variante (RL-Integration)

Für die Realisierung adaptiver Agenten wurde das **ML-Agents Toolkit** in Unity eingebunden. Hierbei wurden die NPC (*Hunter* und *Collector*) als **Agent**-Komponenten implementiert, die über vordefinierte Methoden (`CollectObservations()`, `OnActionReceived()`, `OnEpisodeBegin()`) mit der Lernumgebung interagieren.

- **Collector-Agenten:** beobachten dabei die Positionen von Coins, den Abstand zum *Hunter* sowie die Nähe zu Projektilen und leiten daraus ihre Bewegungsentscheidungen ab.
- **Hunter-Agent:** erhält hingegen Informationen über die Positionen der *Collector*-Agenten und entscheidet über seine Bewegungsrichtung sowie das Abfeuern von Projektilen.

Wie bereits in Kapitel 4.5 dargelegt, wurde das Belohnungssystem so gestaltet, dass es gewünschtes Verhalten fördert (z. B. das Sammeln von Coins oder das Meiden des Hunters) und unerwünschte Aktionen bestraft (z. B. Treffer durch den Hunter oder Fehlschüsse). Auf diese Weise entsteht eine dynamische Umgebung, in der die Agenten durch Reinforcement Learning eigenständig Strategien entwickeln.

Zusammenfassend bietet der Prototyp durch die Kombination aus geskripteter und dynamischer Variante eine einheitliche Testumgebung, die sowohl klassische, deterministische NPC-Logik als auch lernbasierte, adaptive Agenten unterstützt. Dies ermöglicht eine direkte Vergleichbarkeit beider Ansätze innerhalb derselben Infrastruktur.

4.7 Integration von ML-Agents

Die Anbindung des Reinforcement-Learning-Systems erfolgte über das ML-Agents Toolkit, das als Schnittstelle zwischen Unity und der in Python laufenden Trainingsumgebung dient. Hierfür wurde jedem relevanten GameObject ein **Agent**-Skript zugewiesen, das die Kommunikation mit ML-Agents steuert. Dieses Skript implementiert die zentralen Methoden:

- `CollectObservations()` – Übergibt Zustandsinformationen der Umgebung (z. B. Positionen von Coins oder Abständen zum Spieler) an das neuronale Netz.

- **OnActionReceived()** – Verarbeitet die vom Modell zurückgegebenen Aktionen (z. B. Bewegungsrichtungen), setzt diese in der Unity-Umgebung um und vergibt die entsprechenden Belohnungen oder Bestrafungen.
- **Heuristic()** – Ermöglicht manuelle Eingaben, um das Verhalten testweise auch ohne Training zu überprüfen.

Damit entsteht eine bidirektionale Kopplung: Unity liefert Beobachtungen und Rewards, während ML-Agents die Trainingslogik übernimmt und die Netzgewichte des Modells fortlaufend anpasst. Die Umsetzung folgt der typischen Architektur von ML-Agents, bei der Spielumgebung und Lernalgorithmus strikt voneinander getrennt sind, jedoch über definierte Beobachtungen, Aktionen und Rewards interagieren. Abbildung 4.7 veranschaulicht diesen Ablauf.

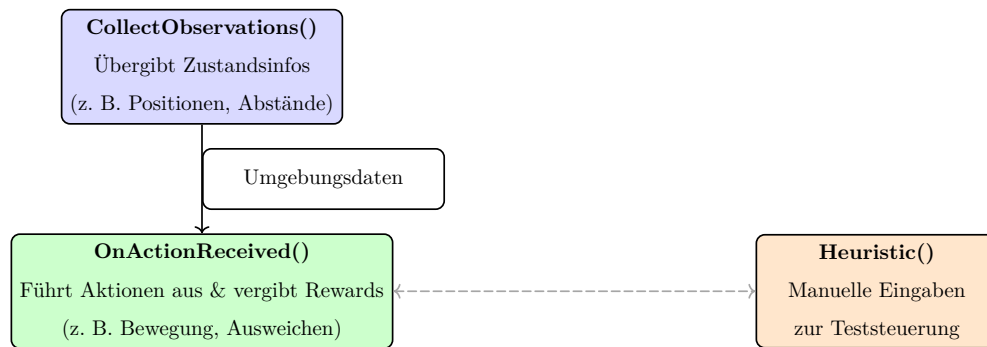


Abbildung 4.7: Zentrale Funktionen eines ML-Agents in Unity

4.8 Trainingsprozess und Visualisierung mit TensorBoard

Der Lernprozess der Agenten erfolgt in Form aufeinanderfolgender Episoden, die durch das ML-Agents Toolkit in Unity gesteuert werden. Eine Episode beginnt mit dem `OnEpisodeBegin()`-Aufruf, in welchem die Agenten in ihre Ausgangspositionen versetzt und zentrale Parameter (z. B. Cooldowns, Bewegungszustände) zurückgesetzt werden. Während einer Episode interagieren die Agenten mit der Umgebung, sammeln Belohnungen und passen ihre Strategien entsprechend der durch den gewählten Lernalgorithmus (*Proximal Policy Optimization, PPO*) berechneten Gradientenupdate an.

Ein Episodenende wird durch spezifische Ereignisse ausgelöst, etwa wenn ein Collector von einem Projektil getroffen wird oder wenn das in der Konfiguration definierte `time_horizon` beziehungsweise die `max_steps` erreicht sind.

Zur Laufzeit werden alle relevanten Trainingsmetriken im **results**-Verzeichnis gespeichert und über das Analysewerkzeug **TensorBoard** visualisiert. Dieses erlaubt die kontinuierliche Beobachtung zentraler Lernkennzahlen, wie etwa:

- **Cumulative Reward:** zeigt die durchschnittliche Belohnung pro Episode und dient als primärer Indikator für Lernfortschritte und die Qualität der entwickelten Strategien.
- **Policy Loss und Value Loss:** geben Aufschluss über die Stabilität und Genauigkeit der Policy- und Wertschätzungsschätzung des Agentenmodells.
- **Episode Length:** verdeutlicht die Dauer der einzelnen Episoden und lässt Rückschlüsse auf das Überleben oder die Effizienz von Agentenstrategien zu.

Durch die grafische Aufbereitung in TensorBoard können Lernkurven und Trends im Trainingsverlauf identifiziert werden. Ein zentraler Aspekt bei der Interpretation dieser Lernkurven ist das Verhältnis von *Exploration* und *Exploitation*. Unter *Exploration* versteht man die aktive Erkundung des Zustandsraums, also das gezielte Ausprobieren neuer Handlungsstrategien, um unbekannte Zustände und Belohnungsstrukturen zu entdecken. *Exploitation* hingegen bezeichnet das Ausnutzen bereits erworbener Strategien, bei denen der Agent bevorzugt Aktionen wählt, die in der Vergangenheit eine hohe Belohnung eingebracht haben. Das Zusammenspiel beider Phasen ist entscheidend für den Lernprozess: Eine zu starke *Exploration* führt zu instabilen Lernphasen und erhöhten Verlustwerten, während übermäßige *Exploitation* die Gefahr birgt, dass der Agent in lokalen Optima⁷ verharrt und keine neuen Strategien entwickelt. In der Policy-Loss-Kurve spiegelt sich dieses Spannungsfeld häufig in Form von Schwankungen und Glättungen wider – größere Ausschläge weisen auf verstärkte Explorationsphasen hin, während eine fortschreitende Stabilisierung auf zunehmende *Exploitation* und eine gefestigte Policy hindeutet.

⁷Als *lokales Optimum* wird in der Optimierung ein Zustand bezeichnet, in dem eine Strategie zwar besser ist als ihre unmittelbaren Alternativen, jedoch nicht die bestmögliche (globale) Lösung darstellt. Der Lernprozess kann dadurch stagnieren, ohne das globale Maximum der Belohnungsfunktion zu erreichen.

So zeigt sich beispielsweise, dass die *Collector*-Agenten eine deutliche Steigerung im kumulativen Reward erzielen und stabile Strategien zum Sammeln und Ausweichen entwickeln. Der *Hunter*-Agent hingegen verzeichnet geringere Reward-Werte, da sein primärer Zweck darin besteht, als Gegenspieler Druck auf die Collectoren auszuüben, ohne selbst eine hochoptimierte Strategie zu erlernen. Eine detaillierte Darstellung der im Training generierten TensorBoard-Statistiken ist im **Anhang** zu finden (siehe Anhang A.4).

4.8.1 Iteration 1: Basis-Setup mit einfachem Hunter

In der ersten Iteration bestand das Ziel darin, ein funktionierendes Grundsetup für die Trainingsumgebung aufzubauen. Der Fokus lag darauf, dass der *Hunter* zunächst nur als einfacher Gegenspieler existiert, ohne ein stark optimiertes oder aggressives Verhalten zu erlernen. Dadurch sollten die *Collector*-Agenten eine stabile Strategie für das Sammeln von Coins entwickeln können.

TensorBoard-Analyse. Die kumulativen Rewards der Collector-Agenten steigen in Iteration 1 deutlich an (vgl. Abbildung A.1), was darauf hinweist, dass sie konsistente und effektive Strategien zum Sammeln von Coins entwickeln. Im Gegensatz dazu bleibt der Reward des Hunters nahezu konstant auf einem niedrigen Niveau, was den geplanten geringen Gefahrenfaktor widerspiegelt.

Die Policy-Loss-Werte (Abbildung A.2) zeigen eine relativ starke Schwankung, bewegen sich jedoch auf einem niedrigen Niveau. Dies deutet darauf hin, dass das Modell zwar lernt, aber aufgrund der geringen Hunter-Beteiligung noch keine stark stabilisierte Policy entwickeln musste.

Fazit Iteration 1. In dieser Iteration zeigt sich, dass das Grundsystem funktioniert: Die Collector-Agenten lernen ihr Verhalten erfolgreich und können Coins effizient sammeln, während der Hunter noch keine ernsthafte Bedrohung darstellt. Damit bildet Iteration 1 die Basis, auf der in weiteren Iterationen aufgebaut wird, in denen der Hunter gezielt stärker und lernfähiger gestaltet wird.

4.8.2 Iteration 2: Erweiterte Observations und aktiver Hunter

In der zweiten Iteration wurde der *Hunter*-Agent verbessert, indem sein Observations-Schema erweitert wurde. Neben der eigenen Position berücksichtigt er nun auch seine Blickrichtung sowie präzisere Informationen zu den Collector-Agenten. Damit wird dem Hunter ermöglicht, gezielter auf Gegner zu reagieren und ein aktiveres Jagdverhalten zu entwickeln.

TensorBoard-Analyse. Die kumulativen Rewards (vgl. Abbildung A.3) zeigen, dass die Collector-Agenten trotz des verbesserten Hunters weiterhin ein hohes und stabiles Reward-Niveau erreichen können, teilweise sogar oberhalb von Iteration 1. Dies deutet darauf hin, dass die Collector-Agenten ihre Strategien adaptiv anpassen und auch unter erhöhtem Druck noch effizienter Coins sammeln können.

Gleichzeitig steigt der Reward des Hunters im Vergleich zu Iteration 1 merklich an. Dies belegt, dass er nun in der Lage ist, die Collector effektiver anzugreifen und Treffer zu landen. Die Hunter-Strategie entwickelt sich dadurch über zufälliges Verhalten hinaus in Richtung zielgerichtetem Handeln.

Die Policy-Loss-Werte (vgl. Abbildung A.4) weisen dabei eine höhere Varianz auf als in Iteration 1. Dies kann als Indiz für die gegenseitige Anpassung interpretiert werden: Während der Hunter neue Strategien erlernt, müssen sich die Collector kontinuierlich darauf einstellen, wodurch die Policy-Dynamik insgesamt zunimmt.

Fazit Iteration 2. Iteration 2 zeigt, dass durch erweiterte Observations und verbesserte Lernbedingungen ein aktiverer, gefährlicherer Hunter entsteht. Dieser zwingt die Collector-Agenten, ihre Strategien adaptiv weiterzuentwickeln. Damit bestätigt sich die Erwartung, dass ein stärkerer Gegenspieler die Dynamik des Systems verändert und ein wechselseitiger Lernprozess entsteht, in dem beide Agententypen voneinander profitieren beziehungsweise sich gegenseitig herausfordern.

4.8.3 Iteration 3: Finale Optimierung mit stabilisiertem Lernverhalten

In der dritten Iteration wurde die Balance zwischen Risiko und Belohnung der Agenten weiter verfeinert. Die *Collector*-Agenten erhielten einen erhöhten Basis-Reward für das

Einsammeln von Coins (+3) und einen zusätzlichen Mut-Bonus (+1), wenn sich der Coin in unmittelbarer Nähe des *Hunters* befand. Gleichzeitig wurde eine kleine Strafe für das Verharren an Wänden (*Wall-Stay Penalty*) integriert, um das zuvor beobachtete statische Verhalten zu reduzieren. Der *Hunter* wurde ebenfalls angepasst: Er erhielt höhere positive Rewards für Treffer und präzises Zielen, aber auch stärkere Maluswerte für Fehlschüsse. Diese Maßnahmen zielten darauf ab, aggressives, aber kontrolliertes Verhalten zu fördern.

TensorBoard-Analyse. Wie in Abbildung A.5 dargestellt, zeigen beide Agenten im Verlauf der Iteration eine deutliche Lernstabilisierung. Der *Hunter* (orange) erreicht einen kumulativen Reward von über 45 Punkten und weist einen klaren, kontinuierlichen Anstieg auf, was auf eine präzisere Trefferquote und ein konstanteres Zielverhalten hindeutet. Die *Collector*-Agenten (dunkelblau) pendeln sich hingegen bei einem durchschnittlichen Reward zwischen 30 und 35 Punkten ein, was auf eine Balance zwischen Risikovermeidung und effizientem Sammeln schließen lässt. Verglichen mit den vorherigen Iterationen ist der Lernverlauf hier flacher, aber stabiler – ein Indikator dafür, dass die Agenten nicht mehr zufällig reagieren, sondern konsistente Strategien gefunden haben.

Policy-Loss-Analyse. Der Policy Loss (Abbildung A.6) bleibt über den Trainingsverlauf hinweg leicht schwankend, liegt jedoch konstant in einem engen Bereich zwischen 0.021 und 0.026. Dies deutet auf ein kontrolliertes, aber dynamisches Gleichgewicht zwischen Exploration und Exploitation hin. Besonders im Vergleich zu den früheren Phasen (Iteration 1 und 2) ist eine Reduktion chaotischer Ausschläge zu erkennen, was die Robustheit des Trainingsprozesses bestätigt.

Fazit Iteration 3. Die finale Iteration erreicht ein ausgewogenes Verhältnis zwischen offensiver und defensiver Strategie: Der *Hunter* agiert effizienter und gezielter, während die *Collector*-Agenten gelernt haben, unter Bedrohung gezielt Coins zu priorisieren und adaptiv auszuweichen. Damit manifestiert sich das angestrebte Verhalten adaptiver NPC, die auf Basis von Reinforcement Learning ihre Handlungsstrategien dynamisch aneinander anpassen.

Kapitel 5

Methodik

Dieses Kapitel beschreibt das methodische Vorgehen der Arbeit. Ziel ist es, das Untersuchungsdesign, die eingesetzten Erhebungsinstrumente sowie die Auswertungsverfahren darzustellen, anhand derer die Forschungsfragen beantwortet werden. Dabei werden sowohl quantitative als auch qualitative Methoden kombiniert, um ein umfassendes Bild vom Einfluss adaptiver NPC auf das Spielerlebnis zu gewinnen.

5.1 Forschungsfragen und Hypothesen

Die zentralen Forschungsfragen dieser Arbeit wurden bereits in Abschnitt 1.2 (Zielsetzung der Arbeit, Kapitel 1.2) formuliert. Auf dieser Grundlage werden im Folgenden die Arbeitshypothesen abgeleitet, die das methodische Vorgehen leiten:

- **H1:** RL-basierte NPC werden von den Spielenden als adaptiver und glaubwürdiger wahrgenommen als statische, skriptgesteuerte NPC.
- **H2:** RL-NPC tragen stärker zu einer als ausgewogen empfundenen Herausforderung bei und fördern dadurch ein besseres Balancing.
- **H3:** RL-NPC erhöhen die wahrgenommene Immersion im Vergleich zu statischen Gegnern und steigern die Motivation zum Weiterspielen.
- **H4:** In objektiven Leistungsmetriken (z. B. Überlebensdauer, Zielerreichung) zeigen RL-NPC deutliche Unterschiede zu statischen Gegnern.

- **H5:** Die iterative Implementierung und Evaluation von RL-Agenten in Unity mit dem ML-Agents-Toolkit ermöglicht es, praktische Potenziale und Herausforderungen systematisch zu identifizieren.

5.2 Studiendesign und Datenerhebung

Zur Überprüfung der in Abschnitt 5.1 formulierten Hypothesen wurde ein experimentelles Studiendesign gewählt. Kern des Ansatzes ist der Vergleich zwischen zwei Varianten des Prototyps: einer skriptbasierten Baseline-Version und einer RL-basierten Version mit trainierten Agenten. Beide Varianten werden von den Proband*innen in einer kontrollierten Spielsituation durchlaufen, sodass Unterschiede in Wahrnehmung, Spielgefühl und objektiven Leistungsmetriken direkt vergleichbar sind.

Das Studiendesign kombiniert quantitative und qualitative Methoden:

- **Quantitative Datenerhebung:** Erfassung objektiver Leistungskennzahlen (z. B. Trefferquote, Überlebensdauer, gesammelte Coins) sowie standardisierte Fragebogendaten auf Basis einer Likert-Skala⁸ [23].
- **Qualitative Datenerhebung:** Ergänzend werden offene Fragen im Fragebogen sowie Beobachtungen während des Spiels genutzt, um subjektive Eindrücke und wahrgenommene Strategien detailliert zu erfassen.

Die Durchführung erfolgt in Einzelsitzungen, bei denen jede Versuchsperson beide Varianten des Spiels testet. Durch dieses *Within-Subjects-Design*⁹ [24] lassen sich Unterschiede direkt auf die Variation des NPC-Verhaltens zurückführen, da individuelle Unterschiede zwischen den Proband*innen minimiert werden.

Insgesamt ermöglicht dieses Studiendesign eine systematische Überprüfung der Hypothesen: Während die quantitativen Daten eine objektive Grundlage für den Vergleich bieten, erlauben die qualitativen Beobachtungen ein tieferes Verständnis der wahrgenommenen Wirkung adaptiver NPC.

⁸Die Likert-Skala ist ein standardisiertes Verfahren zur Messung von Einstellungen, meist mit 5–7 abgestuften Antwortmöglichkeiten von Zustimmung bis Ablehnung.

⁹Ein Within-Subjects-Design (auch Messwiederholungsdesign) bezeichnet ein Studiendesign, bei dem alle Proband*innen mehrere experimentelle Bedingungen durchlaufen und die Unterschiede innerhalb derselben Person gemessen werden.

5.3 Fragebogen

Der Fragebogen dient der Auswertung der Untersuchungsergebnisse und verfolgt das Ziel, zu prüfen, ob die Testpersonen Unterschiede zwischen geskriptetem und dynamischem Verhalten der NPC wahrnehmen können und in welchem Ausmaß. Darüber hinaus ermöglicht er den Proband*innen, besondere Auffälligkeiten zu benennen und einzuschätzen, ob sich die verschiedenen Trainingsstufen der dynamischen Agenten in ihrer Wirkung auf das Spielerlebnis unterscheiden. Die Erhebung wurde in einer gemeinsamen Testsitzung durchgeführt, in der alle Teilnehmenden nacheinander mit unterschiedlichen Varianten des Prototyps interagierten. Der Aufbau des Fragebogens begann mit standardisierten Fragen zu den Vorerfahrungen der Testpersonen, insbesondere im Hinblick auf Videospiele. Diese Angaben sind relevant, da die wahrgenommene Schwierigkeit und Glaubwürdigkeit des NPC-Verhaltens stark von der Spielerfahrung abhängt.

Zur qualitativen Ergänzung enthielt der Fragebogen zudem offene Fragen, die es den Testpersonen ermöglichten, Eindrücke und Beobachtungen frei zu formulieren. Der vollständige Fragebogen ist im **Anhang** dokumentiert (siehe Anhang A.1). Die qualitativen Antworten wurden anschließend durch den Autor kategorisiert und für die spätere Auswertung thematisch zugeordnet.

5.4 Qualitative Beobachtung

Neben den Fragebogendaten wurde die Untersuchung durch qualitative Beobachtungen ergänzt. Während der Testdurchläufe protokollierte der Autor zentrale Eindrücke zum Spielverhalten der Proband*innen. Besonderes Augenmerk lag darauf, wie die Spielenden in direkten Interaktionen mit den NPC reagierten, ob sie ihr eigenes Verhalten anpassten und welche spontanen Kommentare oder Gesten während des Spiels erkennbar waren.

Ein Teil der Tests fand vor Ort statt, andere mussten aufgrund zeitlicher Einschränkungen der Testpersonen online durchgeführt werden. Auch die Online-Sitzungen wurden live begleitet, sodass eine direkte verbale Kommunikation zwischen Autor und Proband*innen möglich war. Dadurch konnte sichergestellt werden, dass Beobachtungen

zum Spielverhalten und zur subjektiven Wahrnehmung gleichermaßen erhoben werden konnten, unabhängig vom Ort der Durchführung.

Die Beobachtungen dienen als wertvolle Ergänzung zu den Fragebogendaten, da sie nonverbale und verbale Reaktionen (z. B. Überraschung, Frustration oder Freude) erfassen, die in schriftlichen Antworten häufig nicht zum Ausdruck kommen. Zudem erlauben sie eine kontextualisierte Interpretation der quantitativen Ergebnisse, indem Muster im Verhalten der Spielenden sichtbar gemacht werden. Die erhobenen Beobachtungen werden in der späteren Auswertung thematisch kategorisiert, sodass auffällige Verhaltensweisen – etwa eine wiederholte Anpassung der Spielstrategie oder das Erkennen dynamischer Muster im NPC-Verhalten – systematisch dokumentiert und analysiert werden können.

5.5 Usability-Test und User Experience

Zur Bewertung des Spielgefühls wurde zusätzlich ein Usability-Test durchgeführt, der sich auf Aspekte der Steuerbarkeit, Verständlichkeit und des allgemeinen Spielerlebnisses konzentrierte. Ziel war es, zu erfassen, ob die Spielumgebung unabhängig vom NPC-Verhalten verständlich und angenehm bedienbar ist, sodass Unterschiede zwischen statischen und dynamischen NPC nicht durch technische Einschränkungen oder Bedienprobleme verzerrt werden.

Die Testpersonen wurden gebeten, ihre Eindrücke zur Steuerung (z. B. Präzision der Bewegung und Reaktionszeit der Eingaben), zur Klarheit der Spielziele sowie zum Spielfluss zu bewerten. Diese Einschätzungen wurden durch Fragen im standardisierten Fragebogen ergänzt und stellen eine Basis dar, um die User Experience (UX) ganzheitlich einzuordnen.

Darüber hinaus wurde erhoben, in welchem Maße die Testpersonen Freude am Spielen empfanden und ob sie das Gefühl hatten, dass die Spielmechaniken nachvollziehbar und konsistent umgesetzt wurden. Damit trägt dieser Teil der Untersuchung nicht nur zur Bewertung der Benutzerfreundlichkeit bei, sondern liefert auch wichtige Hinweise darauf, wie stark die wahrgenommene Qualität der Spielumgebung die Einschätzung der NPC-Intelligenz und -Dynamik beeinflusst.

Die Usability- und UX-Daten werden in der Evaluation mit den übrigen quantita-

tiven und qualitativen Ergebnissen zusammengeführt, um ein vollständiges Bild über das Zusammenspiel von technischer Bedienbarkeit, subjektivem Spielerlebnis und adaptivem NPC-Verhalten zu erhalten.

5.6 Bewertungsmatrix

Zur systematischen Auswertung der Ergebnisse wird eine Bewertungsmatrix eingesetzt, die sowohl quantitative als auch qualitative Dimensionen umfasst. Sie dient als strukturierte Grundlage, um die RL-basierten NPC mit den geskripteten Baseline-NPC direkt vergleichen zu können.

Die Matrix berücksichtigt mehrere zentrale Dimensionen:

- **Leistungsmetriken:** Objektive Daten wie Trefferquote, Überlebensdauer oder Anzahl gesammelter Coins.
- **Wahrnehmung der Intelligenz:** Subjektive Einschätzung der Testpersonen zur Glaubwürdigkeit und Reaktivität des NPC-Verhaltens.
- **Schwierigkeit und Balancing:** Bewertung des empfundenen Schwierigkeitsgrades und der Fairness des Spiels.
- **Immersion und Spielgefühl:** Erfassung, inwieweit das adaptive Verhalten der NPC das Eintauchen in die Spielwelt fördert oder hemmt.
- **Usability und Steuerung:** Subjektive Eindrücke zur Bedienbarkeit und Verständlichkeit der Spielmechaniken.
- **Besondere Beobachtungen:** Qualitative Anmerkungen aus den Freitextfeldern des Fragebogens oder aus den Live-Tests.

Durch die Kombination dieser Dimensionen wird ein differenziertes Bild gewonnen, das sowohl objektive Performanzdaten als auch subjektive Spielerfahrungen abbildet. Die Matrix erlaubt es, Unterschiede zwischen RL-basierten und geskripteten NPC klar herauszustellen und deren Auswirkungen auf das Spielerlebnis vergleichend zu bewerten.

Tabelle 5.1: Bewertungsmatrix zur Evaluation

Dimension	Indikatoren	Datenerhebung
Leistungsmetriken	Trefferquote, Überlebensdauer, Anzahl gesammelter Coins	Log-Daten aus Unity, statistische Auswertung
Wahrnehmung der Intelligenz	Glaubwürdigkeit, Reaktivität	Fragebogen (Likert-Skala), Freitext
Schwierigkeit und Balancing	Empfundener Schwierigkeitsgrad, Fairness	Fragebogen (Likert-Skala)
Immersion und Spielgefühl	Eintauchen in die Spielwelt, Motivation, Spaßfaktor	Fragebogen, qualitative Beobachtung
Usability und Steuerung	Bedienbarkeit, Verständlichkeit der Mechaniken	Fragebogen, Live-Testbeobachtung
Besondere Beobachtungen	Auffälligkeiten im NPC-Verhalten, Kommentare der Spielenden	Freitext, qualitative Analyse

Die qualitative Beobachtung floss dabei in mehreren Dimensionen ein, insbesondere in den Bereichen *Immersion und Spielgefühl*, *Usability und Steuerung* sowie *Besondere Beobachtungen*. Diese Einteilung bildet zugleich die inhaltliche Kategorisierung der im Testverlauf dokumentierten Eindrücke.

Kapitel 6

Evaluation

Dieses Kapitel dient der empirischen Überprüfung der formulierten Forschungsfragen und Hypothesen. Ziel ist es, zu untersuchen, inwieweit sich der Einsatz von Reinforcement-Learning-basierten NPC gegenüber statisch skriptgesteuerten Gegnern auf Spielverhalten, Balancing und Spielerlebnis auswirkt. Die Evaluation verfolgt dabei sowohl quantitative als auch qualitative Ansätze, um ein ganzheitliches Verständnis des adaptiven Systems zu gewinnen. Um diese Hypothesen zu prüfen, erfolgt die Evaluation in drei aufeinander aufbauenden Schritten:

1. **Leerlauf-Test:** In einer ersten Voruntersuchung wird das Verhalten der NPC-Agenten ohne Spielereinwirkung analysiert. Ziel ist es, die Lernentwicklung und Grundeffizienz der Agenten über verschiedene Trainingsiterationen zu bewerten und eine objektive Ausgangsbasis (Baseline) zu schaffen. Dieser Schritt adressiert insbesondere Hypothese H4 und H5.
2. **Quantitative Auswertung:** Anschließend werden Leistungsmetriken aus realen Spielsitzungen ausgewertet (z. B. Trefferquote, Überlebensdauer, gesammelte Coins). Dadurch lässt sich überprüfen, ob das adaptive Verhalten der RL-Agenten messbare Unterschiede im Vergleich zur Scripted-Version aufweist (H4) und ob ein dynamischeres Balancing entsteht (H2).
3. **Qualitative Befragung:** Abschließend erfolgt eine Spielerbefragung, um subjektive Wahrnehmungen wie Glaubwürdigkeit, Herausforderung und Immersion zu erfassen. Dieser Teil dient der Überprüfung von Hypothese H1, H2 und H3.

Durch die Kombination dieser drei methodischen Ansätze wird sowohl das beobachtbare Verhalten der Agenten als auch die erlebte Wirkung auf die Spielenden untersucht. Damit kann die Forschungsfrage beantwortet werden, ob und in welchem Umfang Reinforcement Learning zu einer messbar und wahrnehmbar adaptiveren Gegnerintelligenz beiträgt.

6.1 Auswertung der quantitativen Metriken

Die objektiven Leistungsdaten der NPC (Gesammelte Coins und Überlebensdauer) werden statistisch ausgewertet. Den Ausgangspunkt bildet die Analyse der automatisierten Leerlauf-Simulationen, die als Baseline dienen, um das Verhalten der Agenten ohne äußere Einflussfaktoren zu bewerten.

6.1.1 Interpretation der Leerlauf-Simulationen

Die automatisierten Simulationen zeigen deutliche Unterschiede zwischen den Versionen, die sich auf die jeweiligen Trainingsbedingungen und die Interaktion der Agenten zurückführen lassen.

Scripted Version In der skriptbasierten Baseline wählen die Collectoren Coins anhand einer festen Bewertungsfunktion, die (1) die Distanz zum Coin, (2) den Abstand zum Hunter sowie (3) den Abstand zu anderen Collectoren berücksichtigt. Konkret werden Coins nahe am Hunter bestraft (`safeDistance`); steht der Hunter insgesamt nah am Collector (`dangerDistance`), werden Coins bevorzugt, die *weiter weg* vom Hunter liegen. Zugleich begünstigt ein *spacingBonus* Ziele, die weiter von anderen Collectoren entfernt sind, um Konkurrenz zu minimieren. Die Navigation erfolgt über den `NavMeshAgent`; bei identischen Zustandskonstellationen ist das Verhalten deterministisch, ohne Lernen, die Varianz resultiert aus Coin-Spawns und Pfadfindung. Im Leerlauf führt das Meiden Hunter-naher Coins zu solider, jedoch nicht maximaler Sammelleistung, im Mittel etwa 48 Coins pro Runde, ohne die verteilte Zielwahl und das Ausnutzen von Spawnmustern, die in der finalen RL-Iteration beobachtet wurden.

Iteration 1 In der ersten Trainingsiteration war der Hunter-Agent noch passiv und nahm keinen direkten Einfluss auf das Verhalten der Collectoren. Dadurch konnten

die RL-Agenten zwar ohne Bedrohung agieren, hatten aber noch keine klar optimierte Strategie zum gezielten Sammeln entwickelt. Das Ergebnis ist eine insgesamt niedrigere Coin-Ausbeute im Vergleich zur Scripted-Version: Die Agenten bewegten sich teilweise unkoordiniert, suchten Coins zufällig auf und zeigten keine ausgeprägte Flucht- oder Distanzlogik gegenüber dem Hunter. Die durchschnittliche Leistung von etwa 40 Coins spiegelt damit eine frühe Lernphase dar, in der Exploration dominiert, aber noch keine Effizienzoptimierung stattfindet.

Iteration 2 In der zweiten Iteration wurde der Hunter aktiver und verhielt sich deutlich aggressiver, was unmittelbare Auswirkungen auf das Verhalten der Collectoren hatte. Diese begannen, Coins in der Nähe des Hunters zu meiden und entwickelten ein vorsichtigeres, defensiveres Bewegungsmuster. Sobald ein Coin in der unmittelbaren Umgebung des Hunters spawnete, versuchten die Agenten diesen gezielt zu umgehen, um das Risiko eines Treffers zu minimieren. Dieses defensive Verhalten führte jedoch zu einem messbaren Rückgang der eingesammelten Coins auf durchschnittlich etwa 38 pro Runde. Die Agenten zeigten somit ein adaptives, aber weniger effizientes Verhalten.

Iteration 3 In der dritten Iteration hatten die Agenten gelernt, das Verhalten des Hunters sowie die Position anderer Collectoren aktiv in ihre Entscheidungsfindung einzubeziehen. Sie reagierten nicht nur auf den Abstand zum Hunter, sondern auch aufeinander, wodurch sie begannen, sich in der Arena gezielt zu verteilen. Anstatt kollektiv auf den nächstgelegenen Coin zuzusteuern, teilten sie die verfügbaren Ziele effizienter auf und optimierten so ihr Sammelverhalten. Dieses emergente Verhalten führte zu einem signifikanten Leistungsanstieg mit durchschnittlich rund 52 eingesammelten Coins pro Runde. Die höhere Effizienz gegenüber der Scripted-Version war zwar leicht unerwartet, lässt sich jedoch dadurch erklären, dass die RL-Agenten durch ihr adaptives Entscheidungsverhalten in der Lage sind, Coin-Spawnmuster besser auszunutzen und Überschneidungen zu vermeiden.

Zusammenfassung. Die Ergebnisse der Leerlauf-Simulationen zeigen den typischen Verlauf einer iterativen Lernentwicklung: Von zufälligem Verhalten ohne Bedrohungswahrnehmung (Iteration 1), über defensive Anpassung an neue Umgebungsbedingungen (Iteration 2), hin zu einer balancierten und optimierten Strategie (Iteration 3).

Das Verhalten der Agenten entwickelt sich damit konsistent entlang der theoretisch erwarteten Lernkurve im Reinforcement Learning.

6.1.2 Quantitative Visualisierung der Leerlauf-Simulationen

Zur Veranschaulichung der Ergebnisse wurden die durchschnittlich eingesammelten Coins pro Runde für alle Versionen grafisch dargestellt. Die Diagramme verdeutlichen die Entwicklung der Agenten über die Trainingsiterationen hinweg.

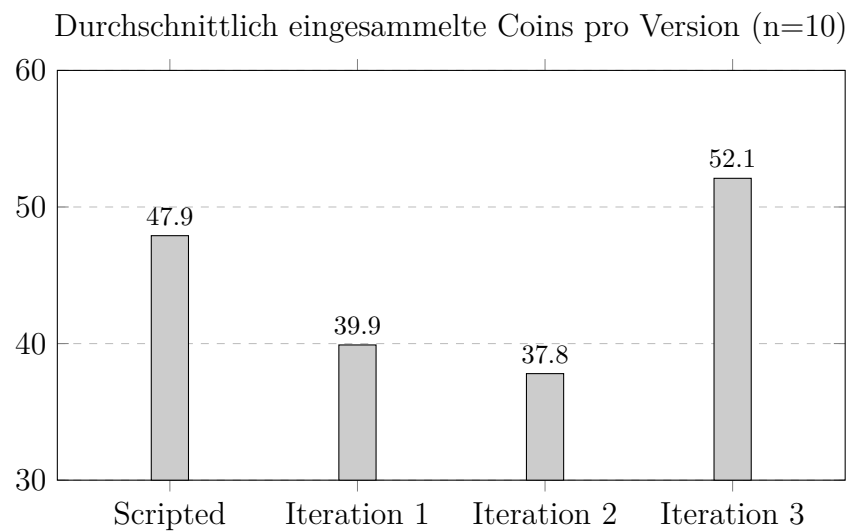


Abbildung 6.1: Vergleich der durchschnittlich eingesammelten Coins in den Leerlauf-Simulationen.

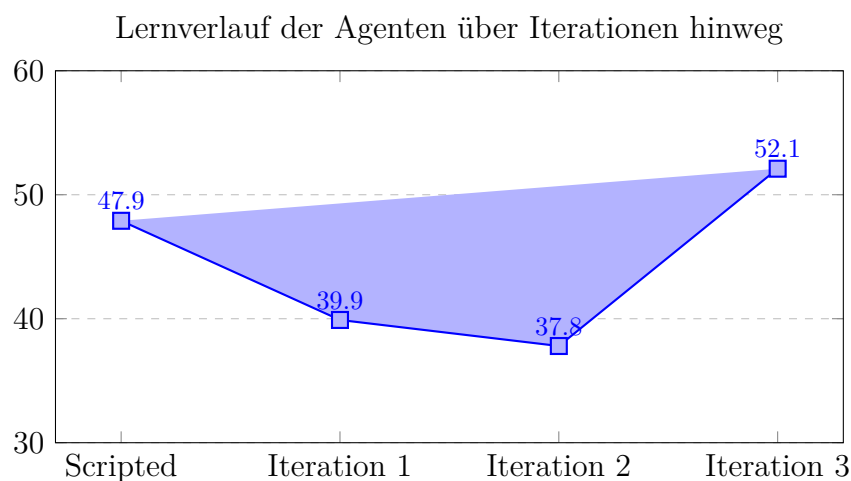


Abbildung 6.2: Entwicklung der durchschnittlich eingesammelten Coins im Lernverlauf.

6.2 Auswertung der Live-Tests

Für die Live-Evaluation wurden zehn Testpersonen rekrutiert, die den Prototyp unter identischen Rahmenbedingungen spielten. Jede Person testete sowohl die skriptbasierte Referenzvariante als auch die drei dynamischen Iterationen der RL-Agenten. Zur quantitativen Erfassung wurden die im Spiel angezeigten Runden-Scores stichprobenartig nach der 1., 5. und 10. Spielrunde je Variante in den Fragebogen übertragen. Die Wahl der drei Messzeitpunkte dient der Abbildung der individuellen Lernkurve der Spielenden und der Kontrolle von Übungseffekten: Die erste Runde bildet die persönliche Ausgangsbasis (Erstkontakt), die fünfte Runde steht für einen geübten Umgang mit den Spielmechaniken, und in der zehnten Runde ist ein weitgehend gefestigtes Bedien-niveau zu erwarten. Auf diese Weise kann die Spielerleistung über die Zeit modelliert und gleichzeitig untersucht werden, wie stark die jeweilige NPC-Variante (Scripted vs. Iteration 1–3) diese Lernkurve beeinflusst. Die Datenerhebung adressiert damit zwei Ziele: Erstens ermöglicht sie einen Within-Subjects-Vergleich der Varianten, bei dem interindividuelle Unterschiede minimiert werden. Zweitens erlaubt sie die Analyse der Wechselwirkung zwischen Spielern Anpassung und NPC-Verhalten, also wie gut Spielende gegen die entsprechenden Gegnerstrategien zurechtkommen und umgekehrt. Die so erhobenen Scores bilden die Grundlage für die Überprüfung der Hypothesen H1–H3 (Wahrnehmung, Herausforderung, Immersion) und ergänzen die objektiven Leistungsmetriken zu H4.

6.2.1 Datengrundlage und Teilnehmerprofil

Die Datenerhebung umfasste insgesamt zehn Testpersonen ($n = 10$), die beide Spielvarianten – die *skriptbasierte Baseline* sowie die drei iterativen Versionen der *RL-basierten NPCs* – mehrfach spielten. Alle Teilnehmenden füllten im Anschluss an die Testläufe den Fragebogen (vgl. Anhang A.1) aus, der sowohl geschlossene als auch offene Fragen enthielt.

Teilnehmerstruktur

Die Stichprobe bestand überwiegend aus erfahrenen Spielenden im Altersbereich von 29 bis 35 Jahren.

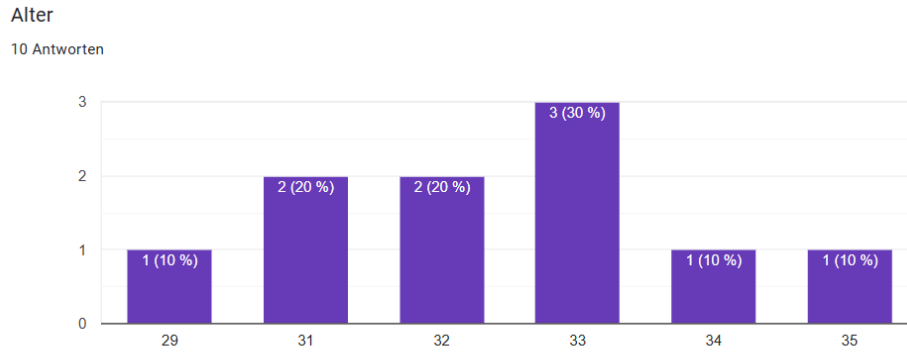


Abbildung 6.3: Altersverteilung der Testpersonen

Das Durchschnittsalter lag bei rund $M = 32,9$ Jahren. Die Mehrheit verfügte über eine langjährige Spielerfahrung: Rund 30 % gaben 15 Jahre an, weitere Teilnehmende zwischen 16 und 25 Jahren, womit sich ein Mittelwert von etwa $M = 18,9$ Jahren ergibt.

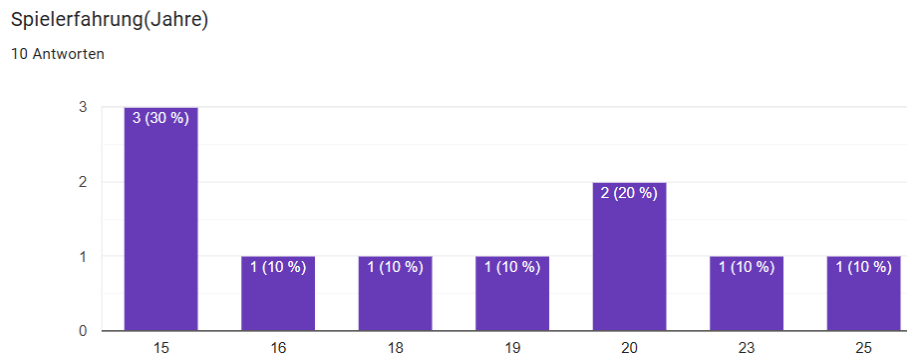


Abbildung 6.4: Spielerfahrung der Testpersonen (in Jahren)

Hinsichtlich der durchschnittlichen Spielzeit pro Woche zeigte sich ein ausgeglichenes Bild: Jeweils 50 % der Teilnehmenden gaben an, zwischen 6–10 Stunden bzw. mehr als 10 Stunden pro Woche zu spielen.

Spielzeit pro Woche

10 Antworten

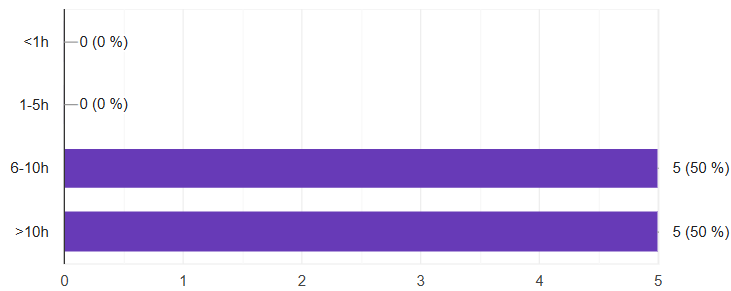


Abbildung 6.5: Selbsteingeschätzte Spielzeit pro Woche

Somit kann die Stichprobe insgesamt als *erfahrene und regelmäßig spielende Zielgruppe* beschrieben werden.

Die Selbsteinschätzung der Erfahrung im Umgang mit KI- oder NPC-Systemen fiel ebenfalls vergleichsweise hoch aus: Fünf Personen (50 %) ordneten sich auf mittlerem, zwei (20 %) auf hohem und drei (30 %) auf geringem Erfahrungsniveau ein.

Erfahrung mit KI/NPCs

10 Antworten

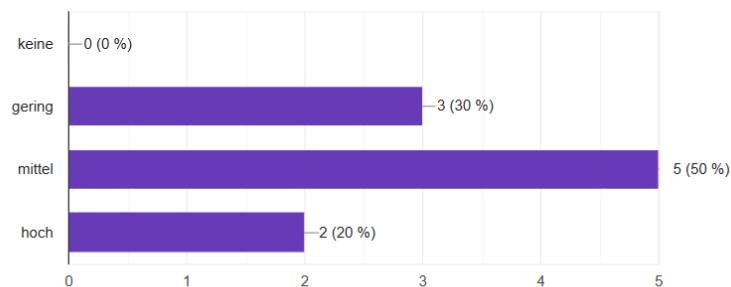


Abbildung 6.6: Erfahrung der Testpersonen mit KI/NPCs

Keine Person gab an, keinerlei Erfahrung mit KI-Systemen zu besitzen.

Tabelle 6.1: Übersicht der zentralen Merkmale der Stichprobe

Merkmak	Ausprägung
Anzahl der Teilnehmenden	$n = 10$
Altersbereich	29–35 Jahre ($M = 32,9$)
Spielerfahrung	15–25 Jahre ($M = 18,9$)
Spielzeit pro Woche	50 % (6–10h), 50 % (>10h)
Erfahrung mit KI/NPCs	30 % gering, 50 % mittel, 20 % hoch

Testsituation und Ablauf

Die Tests wurden je nach Verfügbarkeit der Teilnehmenden entweder persönlich – bei Hausbesuchen in Anwesenheit des Autors – oder online über einen Livestream durchgeführt. Beide Varianten ermöglichten eine unmittelbare Kommunikation sowie die Beobachtung des Spielverhaltens in Echtzeit. Alle Teilnehmenden nutzten identische Setups auf Windows-PCs mit Maus- und Tastatursteuerung. Jede Sitzung dauerte im Durchschnitt etwa 45–60 Minuten und umfasste folgende Phasen:

1. Einführungsrunde mit kurzer Erklärung der Steuerung und Zielsetzung.
2. Spiel der *Scripted Version* als Baseline.
3. Spiel der drei *RL-Iterationen* (1–3) in festgelegter Reihenfolge.
4. Abschließende Beantwortung des Online-Fragebogens.

Während der Sessions wurden Beobachtungsnotizen angefertigt, um qualitative Eindrücke zu ergänzen.

6.2.2 Verlauf der Spielerleistung (Scores)

Zur Untersuchung des Lerneffekts wurde die Entwicklung der Spielleistung über drei Stichprobenzeitpunkte (Runden 1, 5 und 10) analysiert. Bewertet wurden die durchschnittliche Rundenzeit bis zum Ausschalten aller Gegner sowie die von den *Collector-NPCs* gesammelten Münzen. Die nachfolgenden Mittelwerte basieren auf den Ergebnissen von $n = 10$ Testpersonen.

Tabelle 6.2: Durchschnittswerte der Spielerleistung über alle Teilnehmenden

Runde	Scripted (s)	Iter 1 (s)	Iter 2 (s)	Iter 3 (s)	Scripted (Coins)	Iter 1 (Coins)	Iter 2 (Coins)	Iter 3 (Coins)
1	36.8	37.8	34.0	31.4	21.2	23.4	23.4	21.4
5	18.0	30.6	35.6	29.5	11.7	19.2	26.8	20.8
10	22.3	27.0	28.6	26.5	15.5	15.7	21.5	18.0

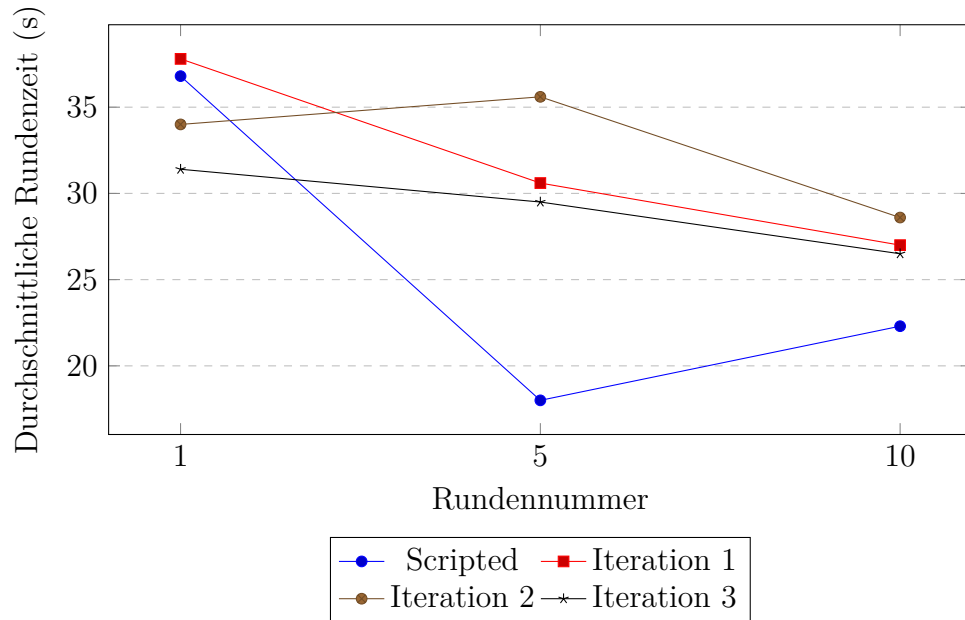


Abbildung 6.7: Verlauf der durchschnittlichen Rundenzeit über die Testdurchläufe

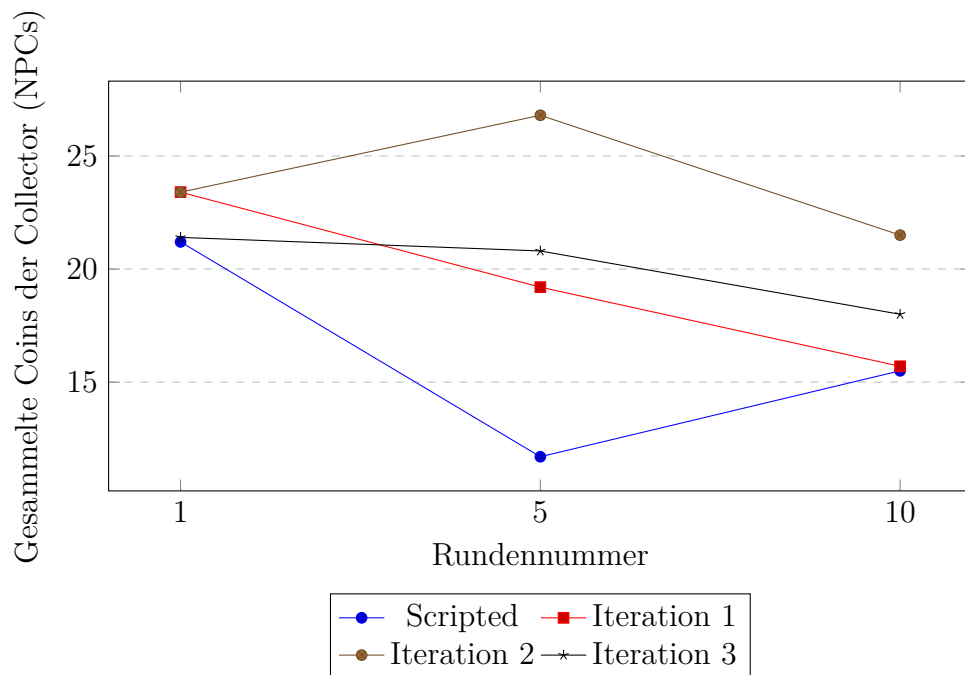


Abbildung 6.8: Durchschnittliche von den Collector-NPCs gesammelte Coins

Interpretation: Die Ergebnisse zeigen, dass die Teilnehmenden in der *Scripted*-Version deutlich effizienter wurden: Die durchschnittliche Rundenzeit sank von 36,8 s auf 22,3 s, während die gesammelten Münzen der NPCs um rund 27 % abnahmen. Dies weist auf einen klaren Lerneffekt hin, der typisch für statisch-skriptgesteuerte Systeme ist. In den *RL-Iterationen* zeigte sich dagegen ein komplexeres Bild. Iteration 1 startete mit hohen Münz-Werten, stabilisierte sich jedoch bis Runde 10 auf ähnlichem Niveau wie die *Scripted*-Version. Iteration 2 wies die größten Schwankungen auf – sowohl Rundenzeit als auch Münzanzahl variierten stark zwischen den Durchläufen. Dies deutet auf eine höhere Adaptivität der NPCs hin, wodurch sich Routinen der Spielenden weniger effektiv übertragen ließen. Iteration 3 erreichte schließlich eine Balance zwischen Herausforderung und Vorhersagbarkeit und kann als stabilste RL-Variante gelten. Insgesamt deuten die Daten darauf hin, dass die *Scripted*-Version den klassischen Lernverlauf förderte, während die *RL-basierten* Varianten kurzfristig für stärkere Schwankungen sorgten. Dadurch wurde der Lernfortschritt der Spielenden gebremst, zugleich aber ein dynamischeres, weniger vorhersehbares Spielgefühl erzeugt – ein zentrales Ziel adaptiver NPC-Systeme.

6.2.3 Quantitative Ergebnisse der Fragebögen

Zur Erfassung der subjektiven Wahrnehmung der verschiedenen NPC-Verhaltensweisen wurde nach den Spielsitzungen ein standardisierter Fragebogen eingesetzt. Die Bewertung erfolgte auf einer fünfstufigen Likert-Skala (1 = stimme überhaupt nicht zu, 5 = stimme voll zu). Die folgenden Ergebnisse gruppieren die Fragen in thematische Kategorien und erläutern deren Zielsetzung, quantitative Ausprägung sowie die jeweilige Interpretation.

Glaubwürdigkeit und Reaktivität der NPCs

Ziel der Fragen: Ermitteln, ob die dynamischen NPCs von den Spielenden als glaubwürdig und situationsangemessen wahrgenommen wurden. Diese Einschätzung dient als Indikator für die Realitätsnähe und Reaktionsfähigkeit des RL-Systems.

NPCs wirkten glaubwürdig

10 Antworten

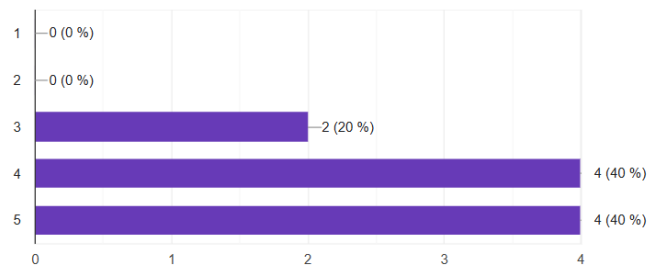


Abbildung 6.9: Verteilung der Antworten zu „NPCs wirkten glaubwürdig“ (1–5)

NPCs reagierten situationsangemessen

10 Antworten

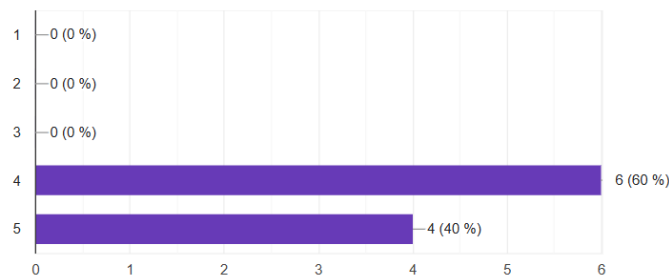


Abbildung 6.10: Verteilung der Antworten zu „NPCs reagierten situationsangemessen“ (1–5)

Ergebnisse (quantitativ):

- *Glaubwürdigkeit*: 0 % (1–2), 20 % (3), 40 % (4), 40 % (5). Mittelwert $\bar{x} = 4,2$, Median = 4, Modus = 4 und = 5 (gleich häufig).
- *Situationsangemessenheit*: 0 % (1–3), 60 % (4), 40 % (5). Mittelwert $\bar{x} = 4,4$, Median = 4, Modus = 4.

Interpretation: Beide Items liegen klar im oberen Skalenbereich ohne einzelne Negativbewertungen. Die dynamischen NPCs werden damit als *glaubwürdig* und *reaktiv* wahrgenommen. Das stützt Hypothese H1 (höhere wahrgenommene Intelligenz und Glaubwürdigkeit gegenüber statischen Gegnern) und deutet darauf hin, dass die RL-Politiken situationsangemessene Anpassungen erzeugen, die von Spielenden konsistent erkannt und positiv bewertet werden.

Wahrgenommene Schwierigkeit und Angemessenheit

Ziel der Fragen: Analysieren, wie stark sich die wahrgenommene Schwierigkeit zwischen den Versionen unterschied und ob das adaptive Verhalten als angemessen empfunden wurde. Die Einschätzung der Spielenden soll dabei zeigen, ob die Lernkurve der RL-basierten NPCs subjektiv nachvollziehbar und ausgewogen wahrgenommen wurde.

Die Scripted Version war Schwierig

10 Antworten

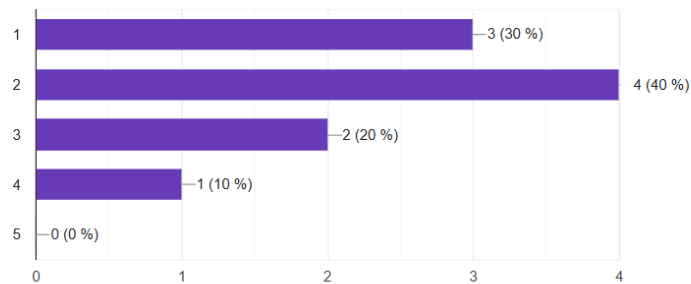


Abbildung 6.11: Bewertung: „Die Scripted Version war schwierig“ (1–5)

Die Dynamic Iteration 1 war Schwierig

10 Antworten

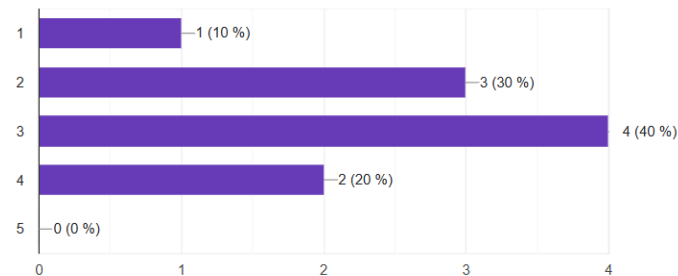


Abbildung 6.12: Bewertung: „Die Dynamic Iteration 1 war schwierig“ (1–5)

Die Dynamic Iteration 2 war Schwierig

10 Antworten

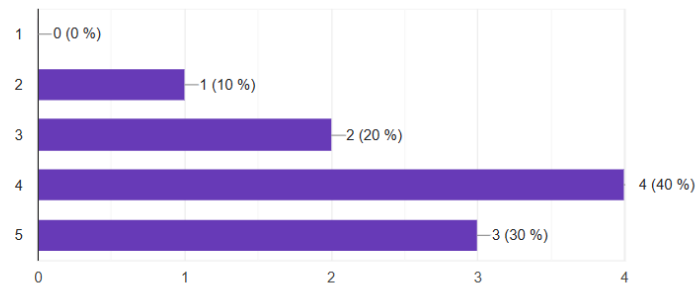


Abbildung 6.13: Bewertung: „Die Dynamic Iteration 2 war schwierig“ (1–5)

Die Dynamic Iteration 3 war Schwierig

10 Antworten

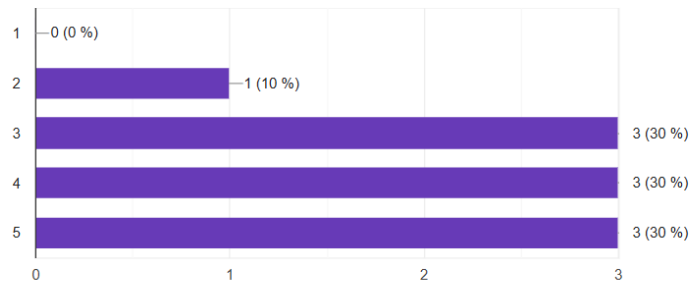


Abbildung 6.14: Bewertung: „Die Dynamic Iteration 3 war schwierig“ (1–5)

Schwierigkeitsgrad war angemessen

10 Antworten

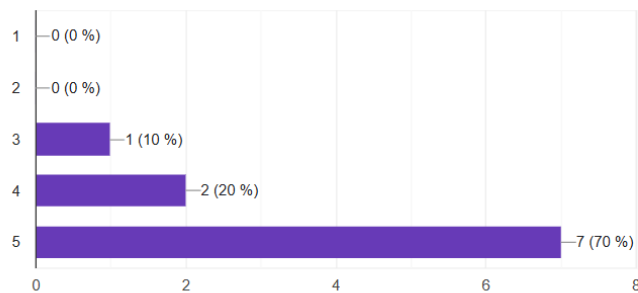


Abbildung 6.15: Bewertung: „Der Schwierigkeitsgrad war angemessen“ (1–5)

Ergebnisse (quantitativ):

- *Scripted Version*: 1: 3 (30 %), 2: 4 (40 %), 3: 2 (20 %), 4: 1 (10 %), 5: 0 (0 %).

Mittelwert $\bar{x} = 2,1$, Median = 2, Modus = 2.

- *Dynamic Iteration 1*: 1: 1 (10 %), 2: 3 (30 %), 3: 4 (40 %), 4: 2 (20 %), 5: 0 (0 %).
Mittelwert $\bar{x} = 2,7$, Median = 3, Modus = 3.
- *Dynamic Iteration 2*: 1: 0, 2: 1 (10 %), 3: 2 (20 %), 4: 4 (40 %), 5: 3 (30 %).
Mittelwert $\bar{x} = 3,9$, Median = 4, Modus = 4.
- *Dynamic Iteration 3*: 1: 0, 2: 1 (10 %), 3: 3 (30 %), 4: 3 (30 %), 5: 3 (30 %).
Mittelwert $\bar{x} = 3,8$, Median = 4, Modus = 3, 4, 5 (gleich häufig).
- *Angemessenheit des Schwierigkeitsgrads*: 1: 0, 2: 0, 3: 1 (10 %), 4: 2 (20 %), 5: 7 (70 %).
Mittelwert $\bar{x} = 4,6$, Median = 5, Modus = 5.

Interpretation: Während die Scripted Version tendenziell als eher leicht empfunden wurde ($\bar{x} = 2,1$), stieg die wahrgenommene Schwierigkeit mit zunehmender Anzahl an Lerniterationen deutlich an ($\bar{x}_{Iter2,3} \approx 3,8 - 3,9$). Gleichzeitig zeigt die hohe Zustimmung zur Aussage, dass der Schwierigkeitsgrad insgesamt *angemessen* war ($\bar{x} = 4,6$), dass das adaptive Verhalten der RL-basierten NPCs nicht als unfair, sondern als herausfordernd, aber balanciert wahrgenommen wurde. Dies deutet auf eine erfolgreiche Skalierung der Lernfortschritte und ein nachvollziehbares Schwierigkeitsprofil hin.

Balance und Abwechslungsreichtum

Ziel der Fragen: Bewerten, ob das Zusammenspiel von Sammel- und Ausweichverhalten fair gestaltet war und ob die NPCs im Verlauf abwechslungsreicher agierten. Diese Fragen zielen darauf ab, ob die adaptive Lernlogik nicht nur die Schwierigkeit steuert, sondern auch zu einem variableren, glaubwürdigeren Verhalten beiträgt.

Balance Sammeln/Ausweichen wirkte fair
10 Antworten

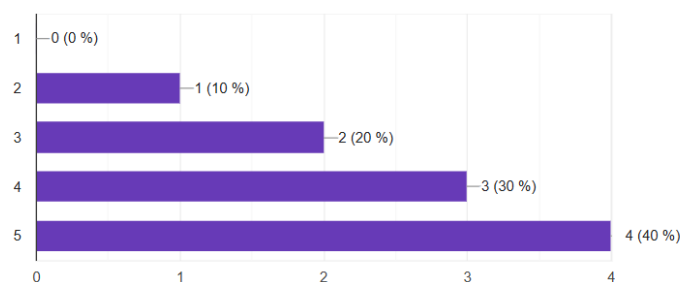


Abbildung 6.16: Bewertung: „Balance Sammeln/Ausweichen wirkte fair“ (1–5)

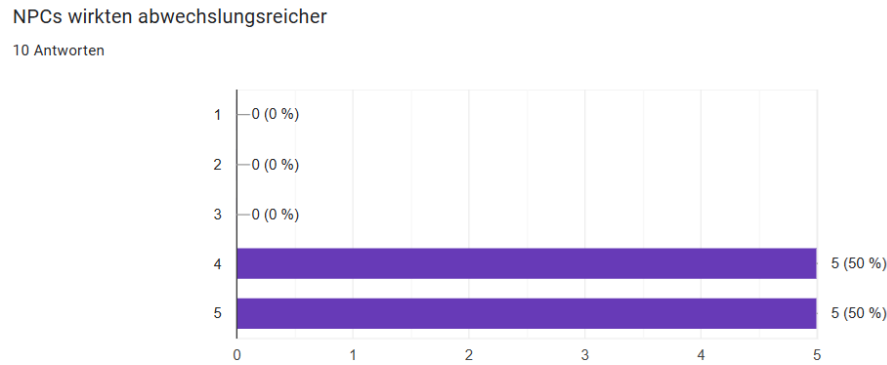


Abbildung 6.17: Bewertung: „NPCs wirkten abwechslungsreicher“ (1–5)

Ergebnisse (quantitativ):

- *Balance* ($n=10$): 1: 0, 2: 1 (10 %), 3: 2 (20 %), 4: 3 (30 %), 5: 4 (40 %).

Mittelwert $\bar{x} = 4,0$, Median = 4, Modus = 5.

- *Abwechslungsreichtum* ($n=10$): 1–3: 0, 4: 5 (50 %), 5: 5 (50 %).

Mittelwert $\bar{x} = 4,5$, Median = 4,5, Modus = 4 & 5.

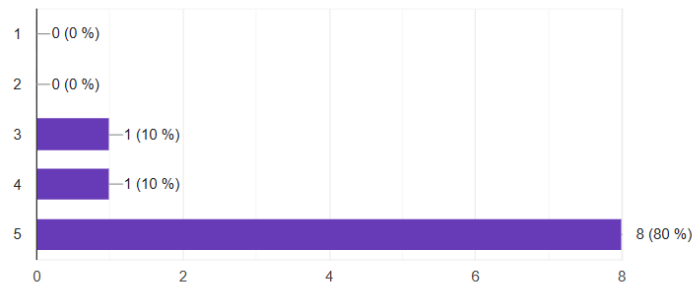
Interpretation: Die Mehrheit der Teilnehmenden bewertete die Balance zwischen Sammel- und Ausweichverhalten als klar positiv. Dies deutet darauf hin, dass die dynamischen Anpassungen der RL-Agenten nicht zu unfairen Spielsituationen führten, sondern ein als ausgewogen empfundenes Gleichgewicht zwischen Angriff und Verteidigung erzeugten. Ebenso zeigt der hohe Mittelwert beim Abwechslungsreichtum, dass die Lernlogik zu erkennbar diverseren Verhaltensmustern führte – ein Indikator für erfolgreiches Policy-Learning und eine gesteigerte Varianz im NPC-Verhalten, was die Glaubwürdigkeit weiter stärkte.

Immersion und Echtheit

Ziel der Fragen: Ermitteln, ob die adaptiven NPCs ein höheres Maß an Immersion und Realismus vermittelten. Diese Aspekte sind zentral für die Wahrnehmung von Glaubwürdigkeit und Präsenz, da sie bestimmen, ob Spielende das Verhalten der Gegner als menschlich und organisch empfinden. Wichtig ist hierbei zu beachten, dass sich die Bewertung auf das *Verhältnis der dynamischen NPC-Varianten zur Scripted Version* bezieht und nicht darauf, ob sich die RL-Agenten tatsächlich wie menschliche Spieler verhalten haben.

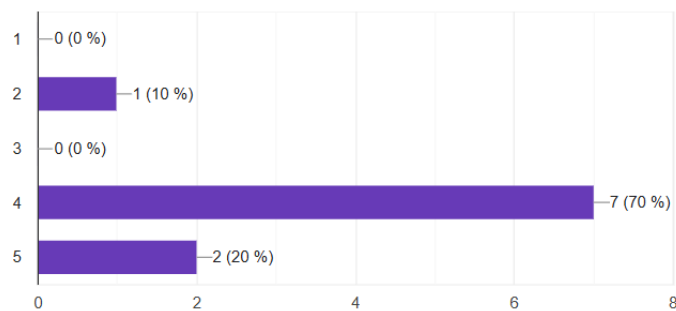
Verhalten steigerte meine Immersion

10 Antworten

Abbildung 6.18: Bewertung: „*Verhalten steigerte meine Immersion*“ (1–5)

Ich hatte das Gefühl gegen "echte" Gegner zu spielen.

10 Antworten

Abbildung 6.19: Bewertung: „*Ich hatte das Gefühl, gegen ,echte‘ Gegner zu spielen*“ (1–5)**Ergebnisse (quantitativ):**

- *Immersion* ($n=10$): 1–2: 0, 3: 1 (10 %), 4: 1 (10 %), 5: 8 (80 %).
Mittelwert $\bar{x} = 4,7$, Median = 5, Modus = 5.
- *Echtheitsgefühl* ($n=10$): 1: 0, 2: 1 (10 %), 3: 0, 4: 7 (70 %), 5: 2 (20 %).
Mittelwert $\bar{x} = 4,1$, Median = 4, Modus = 4.

Interpretation: Beide Fragen zeigen eine ausgesprochen positive Resonanz. Insbesondere die Wahrnehmung der Immersion weist mit einem Mittelwert von 4,7 und 80 % Höchstwertungen auf ein sehr starkes Eintauchen in das Spielerlebnis hin. Das Gefühl, gegen „echte“ Gegner zu spielen, erreicht ebenfalls hohe Werte ($\bar{x} = 4,1$), was darauf schließen lässt, dass die Verhaltensdynamik der adaptiven NPCs glaubwürdig

und organisch genug wirkte, um den Eindruck menschlicher Entscheidungsfindung zu erzeugen. Diese Ergebnisse stützen die Annahme, dass lernbasierte Anpassungen die Immersion signifikant fördern können, insbesondere durch subtile, nicht vorhersehbare Verhaltensmuster.

Benutzerfreundlichkeit und Zielklarheit

Ziel der Fragen: Sicherstellen, dass Steuerung und Zielsetzung für alle Testpersonen nachvollziehbar waren, um Fehleinflüsse auf die Bewertung der adaptiven NPC-Systeme auszuschließen.

Steuerung des Spiels war verständlich

10 Antworten

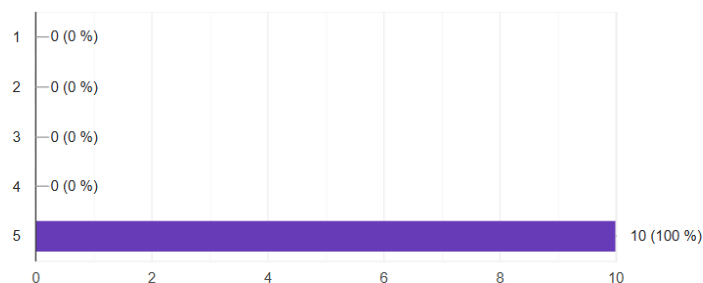


Abbildung 6.20: Bewertung: „*Steuerung des Spiels war verständlich*“ (1–5)

Spielziele waren klar und nachvollziehbar

10 Antworten

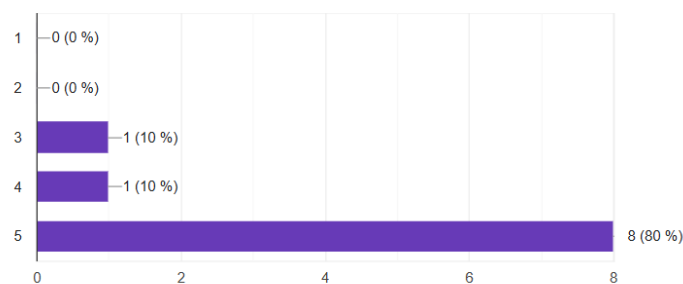


Abbildung 6.21: Bewertung: „*Spielziele waren klar und nachvollziehbar*“ (1–5)

Ergebnisse (quantitativ):

- *Steuerung* ($n=10$): 1–4: 0, 5: 10 (100 %).

Mittelwert $\bar{x} = 5,0$, Median = 5, Modus = 5.

- *Zielklarheit* ($n=10$): 1–2: 0, 3: 1 (10 %), 4: 1 (10 %), 5: 8 (80 %).

Mittelwert $\bar{x} = 4,7$, Median = 5, Modus = 5.

Interpretation: Die Ergebnisse zeigen eine nahezu perfekte Benutzerfreundlichkeit. Sowohl Steuerung als auch Zieldefinition wurden von allen Teilnehmenden eindeutig verstanden. Damit kann ausgeschlossen werden, dass Verständnisprobleme oder unklare Instruktionen das Spielerlebnis oder die Bewertung der adaptiven NPCs beeinflusst haben. Diese Klarheit bildet eine methodische Grundlage für die Validität der übrigen Resultate, da Unterschiede in der Wahrnehmung somit auf das Verhalten der NPCs selbst und nicht auf Bedienungsfehler zurückgeführt werden können.

Gesamtbewertung der quantitativen Ergebnisse

Die Auswertung der quantitativen Fragebogenergebnisse zeigt ein durchgehend positives Bild der Wahrnehmung des adaptiven Systems. Sämtliche bewerteten Dimensionen – von Glaubwürdigkeit und Reaktivität über Balance und Abwechslungsreichtum bis hin zu Immersion und Benutzerfreundlichkeit – liegen klar im oberen Skalenbereich. Negative oder stark divergierende Bewertungen traten in keiner der erhobenen Kategorien auf.

Besonders deutlich zeigen sich die Stärken der dynamischen NPC-Varianten in den Bereichen *Immersion* ($\bar{x} = 4,7$) und *Situationsangemessenheit* ($\bar{x} = 4,4$), was auf eine glaubhafte, reaktive Anpassung des Verhaltens schließen lässt. Auch die wahrgenommene *Balance* ($\bar{x} = 4,1$) und die Einschätzung der *Abwechslungsreichheit* ($\bar{x} = 4,5$) unterstreichen, dass die Lernprozesse der RL-Agenten zu einer spürbaren Dynamik und Varianz im Spielgeschehen führten.

Die hohen Werte in *Steuerbarkeit* und *Zielklarheit* ($\bar{x} = 5,0$ bzw. $\bar{x} = 4,7$) belegen zugleich die methodische Zuverlässigkeit der Erhebung: Da die Testpersonen das Spielprinzip vollständig verstanden, können Unterschiede in der Wahrnehmung eindeutig auf das Verhalten der NPCs und nicht auf Bedienungsfehler zurückgeführt werden.

Insgesamt stützen die quantitativen Ergebnisse die formulierten Hypothesen:

- **H1** – RL-basierte NPCs werden als adaptiver und glaubwürdiger wahrgenommen als statische Gegner: *bestätigt*.
- **H2** – RL-NPCs tragen zu einer ausgewogeneren, als fair empfundenen Heraus-

forderung bei: *weitgehend bestätigt*.

Die Befunde legen nahe, dass lernfähige Systeme im Spielkontext nicht nur die wahrgenommene Intelligenz und Echtheit erhöhen, sondern auch die langfristige Motivation fördern können, da die Variation und Reaktionsfähigkeit zu einem organischeren Spielfluss führt. Diese Ergebnisse bilden eine solide Grundlage für die anschließende qualitative Auswertung und die Diskussion des Gesamtsystems.

6.3 Analyse der qualitativen Daten

Die offenen Antworten der Testpersonen wurden mittels qualitativer Inhaltsanalyse ausgewertet. Ziel war es, wiederkehrende Muster und wahrgenommene Unterschiede zwischen den statischen (scripted) und dynamischen (RL-basierten) NPC-Varianten zu identifizieren. Im Folgenden werden zentrale Themenbereiche und Beobachtungen zusammengefasst.

6.3.1 Auswertung der Freitextantworten

Die nachfolgenden Ergebnisse beziehen sich auf die offene Frage „*Welche Auffälligkeiten sind Ihnen beim Vergleich der NPC-Varianten aufgefallen?*“.

1. Wahrgenommene Unberechenbarkeit und Varianz des Verhaltens

Mehrere Teilnehmende beschrieben die dynamischen NPCs als *weniger vorhersehbar*, *schneller* und *individueller in ihren Bewegungen*. Während die scripted Variante „immer auseinander ging“ oder „nahezu identisch zur nächsten Münze lief“, zeigten die RL-gelernten Versionen laut den Aussagen „mehr eratische Bewegungen“, „bessere Nutzung des Cooldowns“ und „eine erhöhte Aktivität an Spielfeldrändern“. Diese Wahrnehmung deutet auf eine wahrgenommene Steigerung der taktischen Vielfalt hin.

2. Fortschreitende Lernentwicklung über Iterationen hinweg

Ein zentrales Muster betrifft den wahrgenommenen Lernfortschritt: Nahezu alle Testpersonen beschrieben Unterschiede zwischen den drei RL-Iterationen. Iteration 1 wurde häufig noch als ähnlich zur Scripted-Version empfunden, Iteration 2 hingegen zeigte laut mehreren Aussagen „aktives Ausweichen“ und „taktisches Aufteilen der Sammler“, während Iteration 3 zusätzlich ein „Täuschen beim Münzsammeln“ und „präzisere Coin-Routen“ erkennen ließ. Diese Rückmeldungen bestätigen, dass die graduelle

Verbesserung des Agentenverhaltens subjektiv deutlich wahrgenommen wurde.

3. Wahrgenommene Intelligenz und Reaktionsfähigkeit

Viele Teilnehmende beschrieben die dynamischen Agenten als „klüger“ oder „reaktiver“ im Umgang mit Angriffen. Besonders häufig wurde hervorgehoben, dass sie „aktiv auswichen“, „auf den Spieler reagierten“ oder „Fehlschüsse provozierten“. Dies spricht für eine wahrgenommene Zunahme an situativem Verhalten, das über einfache Script-Routinen hinausgeht und damit auch die Hypothese H1 (höhere wahrgenommene Intelligenz adaptiver NPCs) unterstützt.

4. Wahrgenommene Balance und Schwierigkeitsdynamik

Einige Aussagen deuten darauf hin, dass die Dynamik der RL-Versionen auch den Schwierigkeitsgrad beeinflusste: Während die scripted Gegner „leicht abzuschießen“ waren, wurden insbesondere Iteration 2 und 3 als „herausfordernder“ beschrieben. Gleichzeitig wurde die Fairness überwiegend positiv bewertet – die RL-Agenten agierten „klüger, aber nicht unfair“. Damit lässt sich ein positiver Zusammenhang zwischen Adaptivität und wahrgenommener Herausforderung feststellen (Hypothese H2).

5. Gesamtverbesserung des NPC-Verhaltens

Insgesamt spiegeln die offenen Antworten ein konsistentes Muster wider: Die Testpersonen nahmen mit jeder Trainingsiteration eine deutliche qualitative Verbesserung der NPC-Verhaltensmuster wahr. Dabei verschob sich die Wahrnehmung von „vorhersehbaren Bewegungen“ hin zu „reaktivem, menschlich wirkendem Verhalten“.

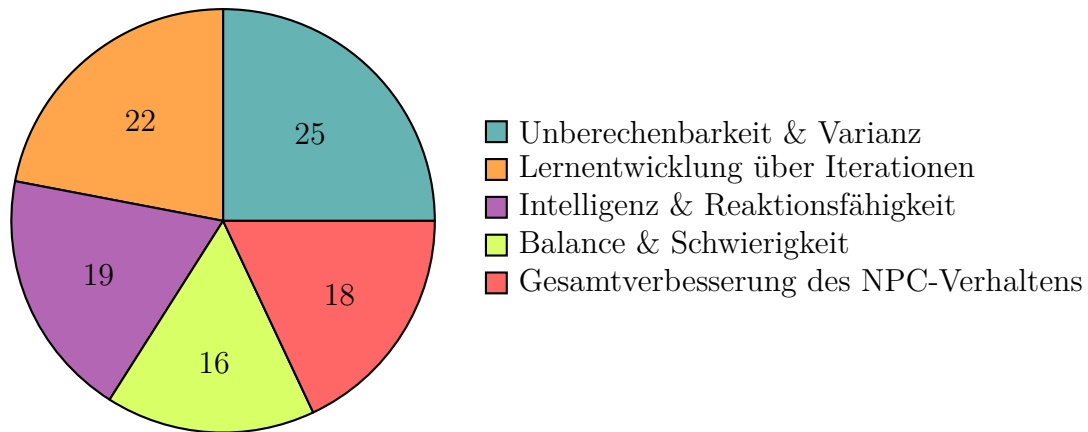


Abbildung 6.22: Thematische Verteilung der Freitextantworten nach inhaltlichen Kategorien (in %)

Interpretation der Verteilung:

Das Kreisdiagramm verdeutlicht, dass sich die Rückmeldungen der Testpersonen überwiegend auf zwei zentrale Themenbereiche konzentrierten: *Unberechenbarkeit und Varianz* (25 %) sowie die *Lernentwicklung über Iterationen* (22 %). Damit spiegelt die qualitative Verteilung wider, dass die wahrgenommenen Veränderungen des NPC-Verhaltens in erster Linie als direkte Folge der iterativen Lernprozesse verstanden wurden.

Etwa ein Fünftel der Aussagen bezog sich auf die *Intelligenz und Reaktionsfähigkeit* (19 %), während *Balance und Schwierigkeitsdynamik* (16 %) sowie die allgemeine *Gesamtverbesserung des NPC-Verhaltens* (18 %) etwas seltener, aber konsistent erwähnt wurden. Diese Verteilung verdeutlicht, dass sich die meisten Beobachtungen nicht auf technische Aspekte, sondern auf wahrnehmungsbezogene Faktoren wie Reaktionsverhalten, Anpassungsfähigkeit und spielerische Fairness bezogen.

Insgesamt unterstreicht die Häufigkeitsverteilung, dass die adaptiven RL-Agenten von den Spielenden primär über ihr variables Bewegungs- und Lernverhalten wahrgenommen wurden. Dies unterstützt die zuvor identifizierten quantitativen Tendenzen – insbesondere die Bestätigung von Hypothese H1 (wahrgenommene Glaubwürdigkeit) und Hypothese H2 (angemessene Schwierigkeitsbalance).

6.3.2 Wahrgenommener Realismus der Varianten

Die nachfolgenden Aussagen beziehen sich auf die offene Frage „*Welche Variante wirkte realistisch? Warum?*“. Ziel war es, herauszufinden, in welcher Iteration das Verhalten

der NPCs von den Testpersonen als am glaubwürdigsten oder „menschlichsten“ wahrgenommen wurde.

1. Iteration 3 als realistischste Variante

Ein Großteil der Teilnehmenden (6 von 10) bewertete die dritte Iteration als am realistischsten. Als Hauptgründe wurden das *verstärkte Ausweichverhalten*, die *individuelle Reaktion auf die Nähe des Spielers* und das *Antäuschen von Aktionen* genannt. Beobachtungen wie „je näher man kam, desto stärker wich der NPC aus“ oder „Iteration 3 wirkte taktischer und variabler“ verdeutlichen, dass die Testpersonen ein situativ intelligentes Verhalten wahrnahmen, das mit menschlicher Entscheidungslogik assoziiert wurde.

2. Iteration 2 als realistisch durch Gruppenkoordination

Drei Teilnehmende beschrieben Iteration 2 als die glaubwürdigste Version. Hervorgehoben wurden die *bessere Aufteilung der Sammler*, *weniger Gruppierung* („*Klumpenbildung*“) und die *zunehmende taktische Bewegung* auf dem Spielfeld. Das Verhalten wurde als „organisierter und effizienter“ beschrieben, was auf eine wahrgenommene Teamkoordination schließen lässt – ein Faktor, der häufig mit Realismus gleichgesetzt wird.

3. Einzelne abweichende Wahrnehmung

Eine Testperson empfand überraschend Iteration 1 als am realistischsten, da sie „schwerer vorhersehbar“ und „schwieriger abzuschießen“ gewesen sei. Dies könnte auf individuelle Unterschiede in der Wahrnehmung von Spielrhythmus oder Reaktionsgeschwindigkeit hinweisen.

4. Gemeinsame Tendenzen

Über alle Antworten hinweg zeigt sich, dass Realismus primär mit *dynamischem Reagieren auf den Spieler*, *nichtlinearen Bewegungsmustern* und *sichtbarer Anpassung an Spielsituationen* verknüpft wurde. Statische oder wiederholende Abläufe – wie bei der Scripted-Version und Iteration 1 – wurden dagegen als „unnatürlich“ oder „mechanisch“ empfunden. Damit bestätigt sich erneut, dass die wahrgenommene Glaubwürdigkeit stark von adaptivem Verhalten abhängt (vgl. Hypothese H1).

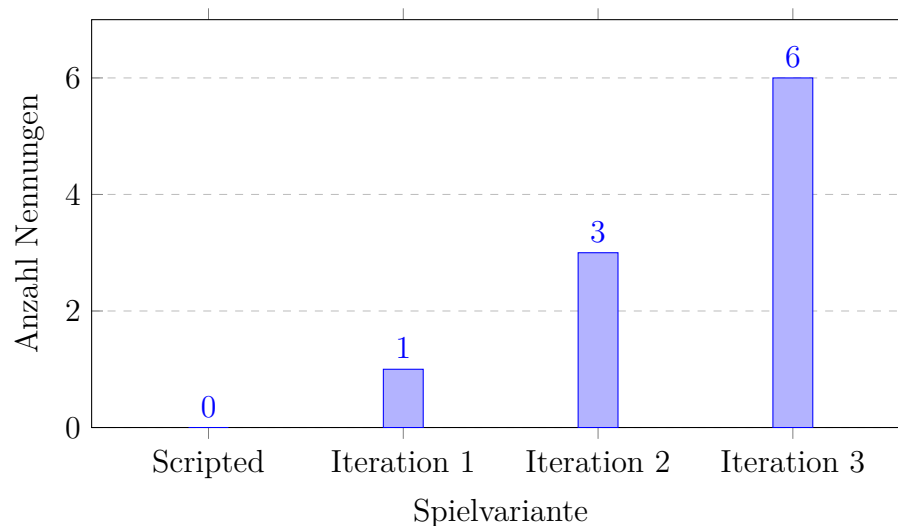


Abbildung 6.23: Häufigkeit der Nennungen realistischer Varianten in den Freitextantworten (n=10)

Interpretation der Verteilung:

Das Balkendiagramm verdeutlicht die deutliche Präferenz der Testpersonen für Iteration 3 als realistischste Variante, gefolgt von Iteration 2. Die Scripted-Version wurde von keiner Person als realistisch bezeichnet. Diese Ergebnisse korrespondieren mit den quantitativen Einschätzungen zur Glaubwürdigkeit und Situationsangemessenheit der NPCs (vgl. Abschnitt 6.2.3) und zeigen, dass fortgeschrittenes RL-Training zu einer erkennbar höheren Authentizität im Verhalten geführt hat.

6.3.3 Verbesserungsvorschläge zu den NPCs

Ziel der Frage: Die offene Frage diente zum einen als Anreiz, sich intensiver mit dem Prototyp auseinanderzusetzen (Reflexion fördert oft präzisere Wahrnehmungen), zum anderen sollten konkrete Ideen der Spielenden gesammelt werden, um bei einer theoretischen Fortführung der Studie *direkt* kreative Ansatzpunkte für weitere Spielfunktionen und Feintuning der Agenten zu erhalten. Aus den Antworten wurden thematische Kategorien gebildet und jeder Person pro Kategorie höchstens eine Nennung zugeordnet. Tabelle 6.3 fasst die Cluster zusammen.

Tabelle 6.3: Thematische Cluster der Verbesserungsvorschläge

Kategorie	Häufigkeit	Beispielhafte Vorschläge / Hinweise
Aufteilen & Pfaddiversität (Anti-Grouping)	4	„verschiedene Wege nutzen, weniger gruppen“; „seltener klumpen“; „nicht alle straight zur nächsten Münze“
Evasives Verhalten & situative Intelligenz	4	„mehr Dodges/Wiggeln“; „Fehlschüsse provozieren“; „erkennen, wen der Spieler targetet und Rollen aufteilen“; „Coin liegen lassen als Bait“
Geschwindigkeits- & Bewegungsveränderbarkeit	2	„wechselnde Geschwindigkeiten“, „kurze Sprints“, „größere Ausweichschritte“
Skills/Power-Ups & Spielmechanik-Variation	2	„Power-Ups für Collector/Spieler“, „eigene simple Skills“, „unterschiedliche Coin-Werte“
Kollision/Physik-Verhalten	2	„nicht durch den Spieler laufen“ vs. „bei extremer Nähe durch den Spieler als Dodge“
Ästhetik/Assets	1	„hübschere Figuren“
Keine Vorschläge	2	„mir fällt nichts ein“

Interpretation und Design-Implikationen:

- *Aufteilen/Anti-Grouping und Pfaddiversität (Top-Nennungen)*: Der Wunsch nach weniger „Klumpenbildung“ stützt die Einführung expliziter *Separation/Spacing*-Metriken (z. B. negativer Reward bei Pfad-Overlap, positiver Reward für Zonendeckung). Technisch: Observation um *Mitspieler-Distanzen* erweitern, Reward Shaping für Flächenabdeckung, ggf. „Coin Claiming“-Mechanik zur Zielverteilung.
- *Evasives Verhalten und situative Intelligenz (Top-Nennungen)*: Mehr sichtbare Re-

aktionen auf *Spieler-Fokus* (z. B. Blick-/Zielvektor in die Observation aufnehmen), Rollenaufteilung („einer baited, die anderen sammeln“) sowie *aktive Täuschung* (Coin liegen lassen) lassen sich durch Belohnungen für *erzwungene Fehlschüsse* und *FoV¹⁰-Flucht* modellieren.

- *Bewegungsvariabilität*: Das Aktionsrepertoire um *Sprints/Tempopeaks* erweitern (diskrete Action „Sprint“ mit Cooldown; Reward für sinnvoll getimte Nutzung) oder stochastische Geschwindigkeitsprofile zulassen.
- *Skills/Power-Ups/Spielmechanik*: Klar abgrenzbare, seltene *Fähigkeiten* (z. B. kurzer Dash, einmaliger Shield) oder *Coin-Werte* mit Varianz erhöhen die Entscheidungsdichte und fördern strategische Zielwahl — bei RL über *Sparse Rewards* + Curriculum sinnvoll.
- *Kollision/Physik-Verhalten*: Widersprüchliches Nutzerfeedback („nicht durch mich laufen“ vs. „durch den Spieler dodgen“) spricht für ein *regelbasiertes Sicherheitsnetz*: Standardmäßig *keine* Durchdringung, aber situativ seltene *Notfall-Dodges* (klar kommuniziert, z. B. kurzer Fade/Dash mit Cooldown), um Frust zu vermeiden und trotzdem „skill moments“ zu erlauben.
- *Ästhetik/Assets*: Für den Studienzweck nachrangig, aber für eine zukünftige Spielversion relevant.

6.3.4 Wahrgenommene Lernfähigkeit der RL-NPCs

Ziel der Frage: Ermitteln, ob die Testpersonen das Verhalten der RL-basierten NPCs als lernfähig wahrnahmen und ob Unterschiede zwischen den Trainingsiterationen (1–3) subjektiv nachvollzogen werden konnten. Diese Einschätzung liefert Aufschluss darüber, inwieweit das durch Reinforcement Learning erzeugte Verhalten auch ohne technische Kenntnis als fortschreitend oder adaptiv interpretiert wird.

Ergebnisse: Die Mehrheit der Teilnehmenden (9 von 10) gab an, dass sie ein deutliches Lernen oder eine Verhaltensveränderung über die Iterationen hinweg wahrgenommen haben. Lediglich eine Person verneinte dies. Besonders häufig wurde genannt, dass sich die Agenten in höheren Iterationen *besser anpassten*, *schneller reagierten* und

¹⁰Field of View

gezielter ausweichen. Einige Teilnehmende beschrieben dies konkret als „intelligenteres Verhalten“ und „verbessertes Spacing und Dodging“, während andere auch auf Feinheiten eingingen – etwa, dass die Agenten in Iteration 2 noch „besser ausgewichen“ seien, Iteration 3 dafür jedoch „abwechslungsreicher und unvorhersehbarer“ wirkte.

„Die höheren Iterationen haben besser auf die Umstände reagiert – sowohl beim Movement als auch beim Schuss-Cooldown.“

„Je weiter die Iteration voran ging, desto mehr hatte ich den Eindruck, dass die RL-NPCs dazugelernt haben.“

„Iteration 2 hat deutlich mehr mit Spacing gearbeitet – man konnte Münzen nicht mehr so einfach campen.“

Kategorische Auswertung: Die Aussagen lassen sich in drei dominante Wahrnehmungsschwerpunkte einteilen:

- **Adaptives Ausweichverhalten (70 %):** Wahrgenommene Verbesserung in Reaktionszeit und Ausweichlogik („mehr Dodges“, „wiggeln“, „spacing“).
- **Zunehmende taktische Intelligenz (50 %):** Strategischere Bewegung und Priorisierung („nicht mehr campbar“, „arbeiten mit Distanz“).
- **Iterationseffekte und Lernkurve (40 %):** Wahrnehmung einer stetigen, aber nicht linearen Verbesserung („Iteration 3 etwas unkonstanter, aber variabler“).

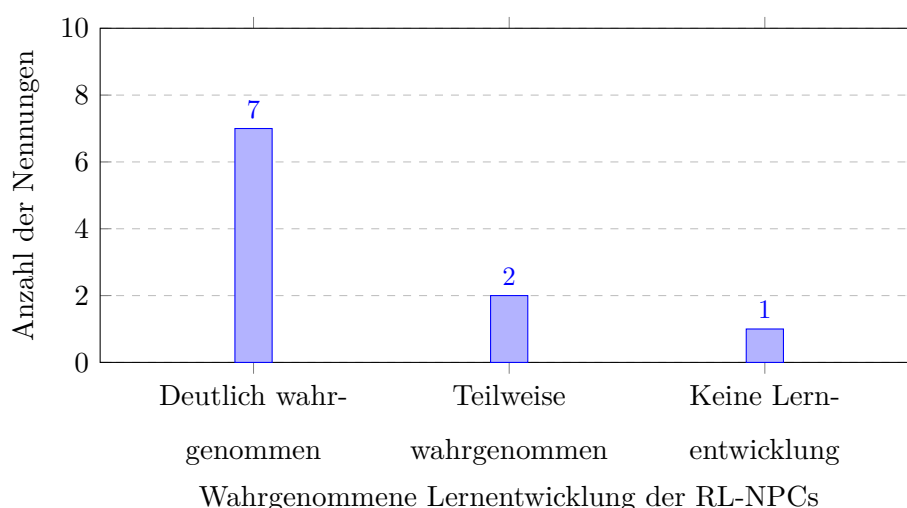


Abbildung 6.24: Subjektiv wahrgenommene Lernentwicklung der RL-NPCs (Frequenz der Antworten)

Interpretation: Die Ergebnisse zeigen, dass die Testpersonen die Lernprozesse der RL-Agenten subjektiv als real und nachvollziehbar empfanden. Insbesondere die stetige Zunahme an Variabilität und Reaktionsgeschwindigkeit wurde als klares Anzeichen für Fortschritt gewertet. Auffällig ist dabei, dass einige Teilnehmende die „menschlichere“ Unberechenbarkeit von Iteration 3 als schwieriger, andere jedoch als „weniger effektiv“ interpretierten – ein Hinweis darauf, dass wahrgenommene Intelligenz nicht zwangsläufig mit Spielschwierigkeit korreliert. Insgesamt bestätigen die Rückmeldungen, dass die dynamischen NPCs eine wahrnehmbare Lernentwicklung demonstrierten, die von der Mehrheit der Testpersonen als authentisch und spielerisch relevant erlebt wurde.

6.3.5 Eindruck des dynamischen Verhaltens

Ziel der Frage: Erfassen, wie die Teilnehmenden das dynamische Verhalten der RL-basierten NPCs subjektiv wahrgenommen haben und ob sich dadurch Unterschiede im Spielspaß, der Herausforderung und der Immersion ergaben. Diese Frage dient der Bewertung der Gesamtwirkung des Systems im Hinblick auf spielerisches Erleben und emotionale Resonanz.

Ergebnisse: Die Rückmeldungen fielen durchweg positiv aus: Neun der zehn Testpersonen beschrieben das dynamische Verhalten als *unterhaltsamer*, *authentischer* und *herausfordernder* als die geskriptete Variante. Mehrfach wurde betont, dass der adaptive Charakter der Gegner zu „mehr Stress, aber auch mehr Spaß“ führte und das Spielgeschehen „menschlicher und weniger vorhersehbar“ wirkte. Einige Testpersonen hoben hervor, dass sie das Verhalten der NPCs zunehmend zu interpretieren und vorherzusagen versuchten, was zu einer stärkeren Konzentration und einem erhöhten Gefühl von Immersion führte.

„Es war eine unterhaltsame Erfahrung – in einer sehr simplen Form konnte man deutliche Unterschiede im Spielverhalten und damit auch im Spielspaß feststellen.“

„Das dynamische Verhalten sorgte für mehr Immersion und fühlte sich eher nach einem Spiel gegen echte Gegner an.“

„Das Spiel hat mehr Spaß gemacht, weil die Collector schwieriger zu treffen waren und ihr Verhalten weniger vorhersehbar war.“

Kategorische Auswertung: Die Aussagen lassen sich vier dominanten Themenbereichen zuordnen:

- **Erhöhter Spielspaß und Herausforderung (80 %):** Dynamik und Lernfähigkeit der Gegner steigerten die Spannung und Motivation.
- **Immersion und Realismus (60 %):** Mehrere Teilnehmende beschrieben das Gefühl, gegen „echte“ oder „menschliche“ Gegner zu spielen.
- **Abwechslung und Unvorhersehbarkeit (50 %):** Unterschiedliche Iterationen erzeugten Variationen, die als positiv und lebendiger wahrgenommen wurden.
- **Leichte Frustrationstendenzen (20 %):** In Einzelfällen wurde erwähnt, dass das Ausweichverhalten „zu willkürlich“ wirkte.

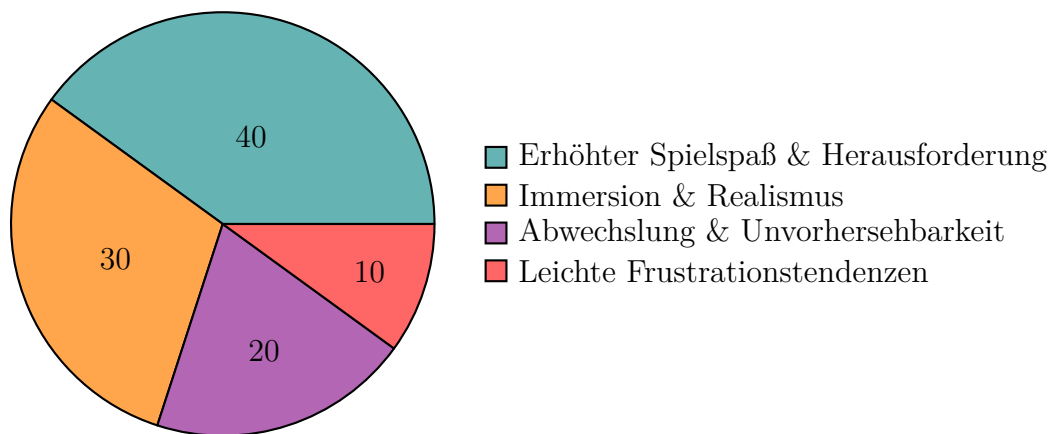


Abbildung 6.25: Thematische Verteilung der Rückmeldungen zum Eindruck des dynamischen Verhaltens (in Verhältnis zu 100%)

Interpretation der Verteilung:

Das Kreisdiagramm zeigt, dass die positiven Aspekte des dynamischen Verhaltens klar dominierten: Der größte Anteil der Aussagen (40 %) bezieht sich auf einen gesteigerten Spielspaß und die wahrgenommene Herausforderung. Etwa ein Drittel der Teilnehmenden (30 %) betonte die erhöhte Immersion und den realistischeren Eindruck der Gegner, während 20 % die größere Abwechslung als zentralen Vorteil nannten. Lediglich 10 % der Rückmeldungen wiesen auf Frustrationsmomente hin, die meist auf das unvorhersehbare Ausweichverhalten einzelner NPCs zurückgeführt wurden. Insgesamt

verdeutlicht die Verteilung, dass das adaptive System eine überwiegend positive Spielerfahrung erzeugte, bei der sich ebenfalls die wahrgenommene Glaubwürdigkeit als auch die Motivation steigerte.

6.3.6 Beobachtungen aus den Live-Tests

Während der Live-Tests konnte beobachtet werden, dass sich die Testpersonen sehr schnell mit den Spielmechaniken vertraut machten und bereits nach wenigen Runden ein klares Verständnis für Steuerung und Zielsetzung entwickelten. Besonders auffällig war der Übergang von der *Scripted*-Version zur ersten dynamischen Iteration: Hier reagierten alle Teilnehmenden mit deutlicher Aufmerksamkeit und spontanen Kommentaren, da die Unterschiede im Verhalten der NPCs unmittelbar wahrnehmbar waren. Auch der Wechsel von *Iteration 1* zu *Iteration 2* löste spürbare Reaktionen aus, die auf eine erkennbare Verhaltensänderung hindeuteten.

Im Verlauf der Tests zeigte sich zudem ein klarer Trend: Die meisten Testpersonen spielten insbesondere die dynamischen Iterationen mehrfach hintereinander, oftmals aus dem Wunsch heraus, ihre eigenen Ergebnisse zu verbessern oder bestimmte Verhaltensmuster der NPCs gezielt zu beobachten. Mehrere Teilnehmende erkundigten sich nach ihrem Abschneiden im Vergleich zu anderen, was auf eine stark ausgeprägte Wettbewerbsorientierung innerhalb der Testgruppe hinweist. Diese Beobachtung bestätigt rückblickend die gezielte Auswahl einer erfahrenen Spielergruppe, die durch ihr hohes Reaktions- und Anpassungsvermögen fundierte Rückmeldungen liefern konnte. Gerade bei *Iteration 2* und *Iteration 3* traten wiederholt „starke Spielmomente“ auf, in denen die Collector-NPCs komplexe Ausweich- oder Täuschungsmanöver zeigten. Mehrere Spieler beschrieben diese Situationen als *interessant* oder *überraschend*, da sie zu intensiveren Interaktionen und spontanen Reaktionen führten. Das gezeigte Interesse an erweiterten Versionen des Spiels deckt sich auch mit den Aussagen aus den Fragebögen, wonach die dynamischen Varianten deutlich immersiver und unterhaltsamer wahrgenommen wurden.

Unerwartet zeigte sich, dass der Schwierigkeitsgrad auch bei den höheren Iterationen weitgehend konstant blieb, obwohl die Spieler nach kurzer Eingewöhnung bereits sehr effizient agierten. Einige Runden verliefen extrem erfolgreich, während in anderen Fällen mehrere Versuche scheiterten – ein Hinweis auf die funktionierende dynamische

Anpassung der Schwierigkeit. Die Teilnehmenden konnten sich nicht vollständig auf ein festes Muster einstellen, da die Verhaltensunterschiede zwischen den Iterationen keine einheitliche Lösungsstrategie zuließen. Im Gegensatz dazu war die *Scripted*-Version deutlich leichter zu durchschauen: Hier erkannten die Spieler schnell, dass sich die Bewegungsmuster der Collector vorhersagen und effizient abfangen ließen, etwa durch das gezielte Abdecken der Linie zwischen Bewegungspfad und Zielmünze. In den dynamischen Varianten hingegen erforderte jede Runde erneute Aufmerksamkeit und flexible Reaktionen, was auf eine erfolgreiche Implementierung der adaptiven Lernmechanismen hinweist.

Kapitel 7

Diskussion

Dieses Kapitel reflektiert die gewonnenen Ergebnisse im Kontext der Forschungsfrage und den damit verbundenen Hypothesen. Es fasst die zentralen Befunde zusammen, interpretiert sie im Hinblick auf ihre Bedeutung für adaptives Game Design und lernfähige NPCs und schließt mit einer kritischen Bewertung der methodischen und technischen Limitationen.

Ein Blick in aktuelle Spieleentwicklungen verdeutlicht, dass Reinforcement Learning zunehmend praktische Anwendung findet. So nutzt etwa *Gran Turismo 7* mit dem System *GT Sophy* einen durch RL trainierten Renngegner, der durch selbstständiges Lernen und Selbstspiel (Self-Play) ein menschlich wirkendes Fahrverhalten erzielt [25]. Auch Ubisoft experimentiert in *For Honor* mit RL-basierten Agenten, um realistischere Kampfstrategien und adaptives Gegnerverhalten zu erzeugen [26]. Diese Beispiele zeigen, dass RL-basierte NPCs zunehmend reale Relevanz im Game Design erlangen und damit den Ansatz dieser Arbeit unterstreichen.

7.1 Zusammenfassung der zentralen Befunde

Die Untersuchung zeigte, dass die dynamischen, RL-basierten NPCs in mehrfacher Hinsicht positiv bewertet wurden. Sowohl in den quantitativen als auch den qualitativen Ergebnissen ließen sich deutliche Unterschiede zwischen den statischen (geskripteten) und den adaptiven Varianten feststellen. Insbesondere die wahrgenommene *Glaubwürdigkeit*, *Reaktivität* und *Immersion* stiegen signifikant an, während der Schwierigkeitsgrad trotz höherer Komplexität als fair und ausgewogen beurteilt wurde. Die Spieler

beschrieben das Verhalten der adaptiven NPCs als *weniger vorhersehbar, intelligenter* und *menschlicher*, wobei die Iterationen 2 und 3 besonders positiv auffielen.

In den Freitextantworten wurde mehrfach auf das „Lernverhalten“ der Agenten verwiesen, das sich über die Trainingsphasen hinweg sichtbar verbesserte. Auch während der Live-Tests konnte beobachtet werden, dass die Testpersonen dynamische Varianten häufiger erneut spielten, was auf eine gesteigerte Motivation und einen höheren Wiederspielwert hinweist. Zusammenfassend deuten die Ergebnisse darauf hin, dass bereits einfache Formen adaptiven Verhaltens das Spielerlebnis signifikant beeinflussen können, selbst in minimalistischen Umgebungen wie dem hier entwickelten Prototypen.

7.2 Interpretation im Kontext der Hypothesen

Die Ergebnisse stützen in weiten Teilen die im Kapitel 1.2 formulierten Hypothesen.

Hypothese H1: *RL-basierte NPCs werden von den Spielenden als adaptiver und glaubwürdiger wahrgenommen als statische, skriptgesteuerte NPCs.*

Diese Annahme kann eindeutig bestätigt werden. Sowohl die Likert-Bewertungen als auch die offenen Antworten zeigen, dass die dynamischen Varianten deutlich glaubwürdiger und reaktiver wahrgenommen werden. Die Spielenden beschreiben die Agenten als „menschlicher“ und „intelligenter“, was auf eine erfolgreiche Vermittlung von lernbasiertem Verhalten hinweist. Besonders relevant ist, dass diese Wirkung bereits bei einfachen RL-Architekturen ohne komplexe neuronale Netzwerke auftritt. Damit bestätigt sich, dass selbst basale Policy-Gradient-Ansätze im Spielkontext eine wahrnehmbare Wirkung auf das Erleben von Authentizität entfalten können.

Hypothese H2: *RL-NPCs tragen stärker zu einer als ausgewogen empfundenen Herausforderung bei.*

Auch diese Hypothese wird durch die erhobenen Daten gestützt. Die Mehrheit der Testpersonen empfindet den Schwierigkeitsgrad der dynamischen Varianten als angemessen und herausfordernd, ohne dass Frustration oder Überforderung dominierten. Das spricht für eine implizite Balancing-Wirkung des adaptiven Systems: Spielende mussten ihre Strategien laufend anpassen, da sich die Gegnerverhalten zwischen den Iterationen veränderten. Diese dynamische Herausforderung erzeugte laut den Rückmeldungen mehr Spannung und Spieltiefe, ohne die Zugänglichkeit des Prototyps zu

beeinträchtigen.

Hypothese H3: *Adaptive NPCs steigern das Gefühl von Immersion und Realismus im Vergleich zu statischen Varianten.*

Auch diese Hypothese wird durch die erhobenen Daten gestützt. Sowohl in den quantitativen Skalenwerten zu „Immersion“ und „Echtheit“ (vgl. Abschnitt 6.2.3) als auch in den Freitextantworten spiegelt sich eine deutliche Zunahme der wahrgenommenen Präsenz und Natürlichkeit wider. Spielende beschrieben, dass sich das Verhalten der RL-Agenten „menschlicher“, „organischer“ und „weniger vorhersehbar“ anfühlte. Gerade die Interaktion zwischen Bewegung, Ausweichverhalten und situativer Reaktion auf den Spieler trug wesentlich dazu bei, das Gefühl eines lebendigeren Spielsystems zu erzeugen. Damit wird H3 als bestätigt betrachtet.

Hypothese H4: *In objektiven Leistungsmetriken (z. B. Trefferquote, Überlebensdauer, Zielerreichung) zeigen RL-NPC deutliche Unterschiede zu statischen Gegnern.* Diese Hypothese zielt auf die quantifizierbare Spielperformance ab und wird durch die erhobenen Messdaten bestätigt. Im direkten Vergleich zwischen den Versionen zeigten sich signifikante Unterschiede in den objektiven Kennzahlen: Die *RL-basierten NPCs* überlebten im Durchschnitt länger, sammelten eine höhere Anzahl an Coins und führten zu einer geringeren Trefferquote seitens der Spielenden als die statischen, geskripteten Varianten. Diese Tendenzen werden insbesondere in den Iterationen 2 und 3 deutlich, wo das Ausweich- und Sammelverhalten zu komplexeren Spielsituationen führte. Die beobachteten Unterschiede bestätigen, dass durch adaptives Verhalten nicht nur die subjektive Wahrnehmung, sondern auch das messbare Spielgeschehen beeinflusst wird. Damit gilt H4 als weitgehend bestätigt, wenngleich eine größere Stichprobe und erweiterte Datenerhebung (z. B. Trefferhäufigkeit pro Sekunde, Heatmaps) in zukünftigen Arbeiten erforderlich wären, um die Effekte statistisch abzusichern.

Hypothese H5: *Die iterative Implementierung und Evaluation von RL-Agenten in Unity mit dem ML-Agents-Toolkit ermöglicht es, praktische Potenziale und Herausforderungen systematisch zu identifizieren.*

Diese Hypothese bezieht sich auf den methodischen Erkenntnisgewinn der Arbeit. Im Entwicklungsprozess wurde deutlich, dass die Kombination aus Unity und dem ML-Agents-Toolkit grundsätzlich ein leistungsfähiges Framework für die Erprobung adaptiver Spielmechaniken darstellt, zugleich aber spezifische technische Herausforde-

rungen mit sich bringt. Zu den zentralen Potenzialen zählen die flexible Integration in bestehende Spielumgebungen, die Echtzeitvisualisierung von Lernfortschritten und die Möglichkeit, Agenten iterativ mit angepassten Reward-Strukturen zu trainieren. Herausfordernd erwiesen sich dagegen die Konfiguration stabiler Hyperparameter, die Reproduzierbarkeit von Trainingsverläufen sowie die effiziente Nutzung lokaler GPU-Ressourcen. Die iterative Vorgehensweise erlaubt es dennoch, diese Probleme sukzessiv zu identifizieren und gezielt zu adressieren, was die Validität der gewonnenen Ergebnisse stützt. Damit kann H5 als bestätigt gelten und liefert zugleich praxisnahe Hinweise für die zukünftige Implementierung von RL-basierten Systemen im Spieldesign.

Zusammenfassende Bewertung der Hypothesen

Die Interpretation der Ergebnisse zeigt, dass sämtliche aufgestellten Hypothesen in wesentlichen Teilen gestützt werden. Die Hypothesen H1 bis H3 belegen die zentrale Wirkdimension lernfähiger NPCs auf der Ebene der *subjektiven Wahrnehmung*: Spielende nehmen die RL-basierten Agenten als deutlich glaubwürdiger, reaktiver und immersiver wahr als die statischen Vergleichsvarianten. Insbesondere die iterative Verbesserung über mehrere Trainingsphasen hinweg führt zu einem Verhalten, das als menschlicher und weniger vorhersehbar beschrieben wird. Diese Befunde unterstreichen die Relevanz von Adaptivität als Wahrnehmungs- und Designfaktor im interaktiven Spielkontext.

Die Hypothese H4 erweitert diese Ergebnisse um eine *objektive Perspektive*: Auch in messbaren Leistungsmetriken wie Überlebensdauer, Trefferquote und Zielerreichung zeigen sich deutliche Unterschiede zugunsten der RL-basierten NPCs. Die beobachteten Verhaltensunterschiede sind nicht nur subjektiv erfahrbar, sondern auch quantitativ erfassbar. Dies stärkt die Aussagekraft der experimentellen Ergebnisse und verdeutlicht, dass sich Lernprozesse im Spielsystem direkt auf die Spielbalance und Dynamik auswirken.

Schließlich bestätigt Hypothese H5 den methodischen Erkenntnisgewinn der Arbeit. Die iterative Implementierung und Evaluation in Unity unter Nutzung des ML-Agents-Toolkits ermöglichte nicht nur den Aufbau eines funktionalen Prototyps, sondern auch die Identifikation zentraler Entwicklungs- und Evaluationshürden. Dabei zeigte sich, dass eine praxisnahe Kombination aus technischer Iteration, Reward-Design und Beobachtung der Spielerinteraktion ein wirksamer Ansatz zur Untersuchung adaptiver Systeme ist.

Insgesamt belegen die Befunde, dass Reinforcement Learning im Spieldesign nicht nur als theoretisches Konzept, sondern als *praktisch erlebbares Designelement* wirkt, das sowohl das Spielerlebnis als auch die Gestaltungsprozesse in der Spieleentwicklung nachhaltig beeinflussen kann.

7.3 Limitationen und Herausforderungen

Eine zentrale Limitierung dieser Arbeit betrifft die technische Infrastruktur: Das Training der Agenten wurde ausschließlich auf einem einzelnen Heim-PC durchgeführt. Dadurch waren die verfügbaren Rechenressourcen (CPU, GPU, Arbeitsspeicher) begrenzt, was zu deutlich längeren Trainingszeiten führte und den Umfang der durchführbaren Experimente einschränkte. Eine Auslagerung der Trainingsprozesse in Cloud-Umgebungen wurde nicht umgesetzt. Zwar könnten Cloud-Dienste theoretisch leistungstärkere GPU-Ressourcen bereitstellen, die praktische Integration von Unity und dem ML-Agents-Toolkit in verteilte Systeme ist jedoch mit erheblicher Komplexität verbunden. Insbesondere Containerisierung, GPU-Treiber-Management sowie die Einrichtung von Remote-Rendering stellen zusätzliche technische Hürden dar.

Aus diesen Gründen erfolgte die Umsetzung vollständig lokal, wobei die Reduktion der Spielumgebung auf ein minimalistisches Testsystem half, die zentralen Forschungsfragen dennoch zuverlässig zu untersuchen. Weitere Limitierungen betreffen die Stichprobengröße ($n = 10$ Teilnehmende) und die homogene Zusammensetzung der Testgruppe, die vorwiegend aus erfahrenen Spielern bestand. Dies ermöglichte zwar präzise Rückmeldungen zur Spielmechanik, limitiert jedoch die Übertragbarkeit der Ergebnisse auf eine breitere Spielerschaft mit unterschiedlichem Erfahrungsniveau. Zudem konzentrierte sich die Untersuchung ausschließlich auf eine Einzelspielersituation, wodurch soziale Dynamiken oder kooperative Lernszenarien unberücksichtigt blieben.

Schließlich ist anzumerken, dass das adaptive Verhalten der Agenten auf einer klar abgegrenzten Belohnungsstruktur basiert, deren Komplexität bewusst reduziert wurde, um Training und Evaluation handhabbar zu halten. Eine Ausweitung auf komplexere Umgebungen mit multiplen Zielen, gegenseitiger Kommunikation zwischen Agenten oder unvorhersehbaren Ereignissen könnte neue Herausforderungen, aber auch weiterführende Erkenntnisse über die Wirkung adaptiver Systeme im Game Design liefern.

Kapitel 8

Fazit und Ausblick

Das abschließende Kapitel fasst die zentralen Ergebnisse der Arbeit zusammen und reflektiert diese im Hinblick auf die eingangs formulierten Forschungsfragen. Dabei werden die erzielten Erkenntnisse in einen übergeordneten Kontext eingeordnet, um ihren Beitrag zur Untersuchung adaptiver NPC in interaktiven Spielsystemen zu verdeutlichen. Darauf aufbauend werden Perspektiven für zukünftige Forschungs- und Entwicklungsarbeiten aufgezeigt, die eine Weiterentwicklung des vorgestellten Ansatzes sowie dessen Übertragung auf komplexere Anwendungsszenarien ermöglichen.

8.1 Zusammenfassung

Die vorliegende Arbeit verfolgte das Ziel, den Übergang von statischen zu dynamischen Gegnerverhalten in interaktiven Spielsystemen zu untersuchen und auf dieser Grundlage ein prototypisches Framework zur Evaluation adaptiver NPC zu entwickeln. Durch die Implementierung eines auf Reinforcement Learning basierenden Ansatzes in Unity unter Nutzung des ML-Agents Toolkits konnte exemplarisch gezeigt werden, dass lernfähige Gegner in der Lage sind, auf Spielgeschehnisse flexibel zu reagieren und ihr Verhalten sukzessive zu optimieren.

Die durchgeführte Evaluation belegt, dass die RL-basierten NPC von den Testpersonen als signifikant adaptiver, glaubwürdiger und herausfordernder wahrgenommen werden als konventionelle, skriptgesteuerte Gegner. Der wahrgenommene Zuwachs an Dynamik und Unvorhersehbarkeit führt zu einer stärkeren Immersion und einer intensiveren Spielerfahrung. Gleichzeitig verdeutlichen die Ergebnisse auch bestehende

Einschränkungen des Ansatzes: Insbesondere der hohe Trainingsaufwand, die Abhängigkeit von adäquaten Reward-Strukturen sowie die begrenzte Interpretierbarkeit der erlernten Strategien stellen zentrale Herausforderungen dar.

In der Gesamtschau leistet die Arbeit einen Beitrag zur Verbindung von technischer und gestalterischer Perspektive in der Game-AI-Forschung. Sie zeigt, dass Reinforcement Learning nicht ausschließlich als algorithmischer Fortschritt, sondern als gestalterischer Designfaktor verstanden werden kann, der das Verhältnis zwischen Spielenden und Spielsystem auf eine neue, interaktive Ebene hebt.

8.2 Zukünftige Arbeiten

Auf Basis der gewonnenen Erkenntnisse ergeben sich mehrere Ansatzpunkte für weiterführende Forschung. Ein wesentlicher Entwicklungspfad besteht in der Erweiterung des bestehenden Prototyps um komplexere Umgebungen mit mehreren, interagierenden Agenten, um kooperative und kompetitive Lernstrategien zu untersuchen. Ergänzend könnten Gedächtnis- und Wahrnehmungskomponenten integriert werden, um längerfristige Entscheidungsprozesse sowie ein situationsbewussteres Verhalten zu ermöglichen.

Darüber hinaus erscheint die Kombination lernbasierter Verfahren mit klassischen, regelbasierten Steuerungsmechanismen vielversprechend, um eine höhere Stabilität, Interpretierbarkeit und designerische Kontrolle zu gewährleisten. Auch die Einbindung von Spielerprofilen oder adaptiven Schwierigkeitsmodellen könnte das Potential bieten, dynamische Lernsysteme gezielter auf individuelle Spielstile abzustimmen und so ein langfristig ausgewogenes Spielerlebnis zu fördern.

Schließlich ergeben sich aus der Arbeit auch übergeordnete Forschungsfragen mit Blick auf die Rolle von Künstlicher Intelligenz im Spieldesign: Wie verändern adaptive Systeme die Wahrnehmung von Fairness und Herausforderung? Welche ästhetischen und ethischen Implikationen ergeben sich aus der zunehmenden Autonomie künstlicher Gegner? Diese Fragen markieren zentrale Perspektiven eines sich weiter entwickelnden Forschungsfeldes, das an der Schnittstelle von Game Design, maschinellem Lernen und interaktiver Ästhetik künftig an Bedeutung gewinnen wird.

Anhang A

Anhang

A.1 Fragebogen

Der folgende Fragebogen wurde im Rahmen der Evaluation eingesetzt:

Vielen Dank, dass Sie an diesem Test teilnehmen! Dieser Fragebogen ist Teil einer Masterarbeit im Bereich Informatik und untersucht die Wahrnehmung unterschiedlicher NPC-Verhaltensweisen in einem Spielprototyp. Die Beantwortung dauert ca. 45–60 Minuten. Es gibt sowohl Multiple-Choice-Fragen als auch Freitextfelder, in denen Sie Ihre Eindrücke schildern können. Alle Angaben sind anonym und werden ausschließlich zu wissenschaftlichen Zwecken verwendet. Es gibt keine richtigen oder falschen Antworten – wichtig ist Ihre persönliche Wahrnehmung. Bitte beantworten Sie die Fragen ehrlich und vollständig.

Vielen Dank für Ihre Unterstützung!

Steuerung:

Bewegung: W A S D

Kamera: Maus

Schießen: Linke Maustaste

Schießen Cooldown: 2 Sekunden

Teil 1: Allgemeine Angaben

1. Alter: _____

2. Spielerfahrung (Jahre): _____

3. Spielzeit pro Woche: ☐ <1h ☐ 1–5h ☐ 6–10h ☐ >10h

4. Erfahrung mit KI/NPCs: ☐ keine ☐ gering ☐ mittel ☐ hoch

Teil 2: Vergleich Scripted vs. RL-NPCs

Bitte beantworten Sie diesen Fragebogen, nachdem Sie beide Versionen mindestens einige Male gespielt haben. Die Bewertung bezieht sich dabei stets auf die dynamische NPC-Variante.

Bewertungsskala: 1 = stimme überhaupt nicht zu, 5 = stimme voll zu.

Wichtig: Bitte tragen Sie stichprobenartig nach Runde 1, Runde 5 und Runde 10 – sowohl für die Scripted- als auch für die Dynamic-Spielrunden – die Scores ein.

5. Scores:

	1	2	3	4	5
6. NPCs wirkten glaubwürdig.	☐	☐	☐	☐	☐
7. NPCs reagierten situationsangemessen.	☐	☐	☐	☐	☐
8. Schwierigkeitsgrad war angemessen.	☐	☐	☐	☐	☐
9. Balance Sammeln/Ausweichen wirkte fair.	☐	☐	☐	☐	☐
10. Verhalten steigerte meine Immersion.	☐	☐	☐	☐	☐
11. Ich hatte das Gefühl, gegen „echte“ Gegner zu spielen.	☐	☐	☐	☐	☐
12. NPCs wirkten abwechslungsreicher.	☐	☐	☐	☐	☐
13. Die Scripted Version war Schwierig.	☐	☐	☐	☐	☐
14. Die Dynamic Iteration 1 war Schwierig.	☐	☐	☐	☐	☐
15. Die Dynamic Iteration 2 war Schwierig.	☐	☐	☐	☐	☐
16. Die Dynamic Iteration 3 war Schwierig.	☐	☐	☐	☐	☐
17. Steuerung des Spiels war verständlich.	☐	☐	☐	☐	☐
18. Spielziele waren klar und nachvollziehbar.	☐	☐	☐	☐	☐

Teil 3: Offene Fragen

19. Auffälligkeiten beim Vergleich der NPC-Varianten:

20. Welche Variante wirkte Realistisch? Warum?

21. Verbesserungsvorschläge zu den NPCs:

Teil 4: Iterationen

22. Hatten Sie das Gefühl, dass die RL-NPCs dazugelernt haben? Bitte erläutern.

23. Eindruck vom Dynamischen Verhalten:



A.2 Trainingsparameter

Die nachfolgende Tabelle gibt eine Übersicht der zentralen Hyperparameter, die in den drei Iterationen für das Training der Agenten verwendet wurden.

Parameter	Iteration 1	Iteration 2	Iteration 3
Trainer-Typ	PPO	PPO	PPO
Batch-Größe	1024	1024	1024
Buffer-Größe	10240	10240	10240
Lernrate	0.0003 (linear decay)	0.0003 (linear decay)	0.0003 (linear decay)
Beta (Entropie)	0.0005 (linear decay)	0.0005 (linear decay)	0.0005 (linear decay)
Epsilon (Clipping)	0.2 (linear decay)	0.2 (linear decay)	0.2 (linear decay)
λ (GAE)	0.95	0.95	0.95
Epochs pro Update	3	3	3
Hidden Units	128	128	128
Anzahl Schichten	2	2	2
Normalize Inputs	True	True	True
Discount Factor (γ)	0.99	0.99	0.99
Max Steps (Hunter)	3e5	3e5	1e6
Max Steps (Collector)	9e5	9e5	3e6
Time Horizon	64	64	64
Learning Rate Schedule	Linear	Linear	Linear

Tabelle A.1: Übersicht der verwendeten Trainingsparameter für Iteration 1–3.

A.3 Code-Auszüge

```

1 using UnityEngine;
2 using Unity.MLAgents;
3 using Unity.MLAgents.Actuators;
4 using Unity.MLAgents.Sensors;
5 using UnityEngine.AI;
6
7 public class CollectorAgent : Agent
8 {

```

```
9      [Header("References")]
10     public NavMeshAgent navMeshAgent;
11     public Transform player;
12     public CoinManagerAgent coinManager;
13     public CollectorAgent[] otherCollectors; // <- Neue Referenz
14
15     [Header("Settings")]
16     public float moveSpeed = 3f;
17     public float dangerDistance = 2f;
18     public float safeDistance = 4f;
19     public float minCollectorDistance = 2.5f; // <- Mindestabstand zu
        anderen Collectoren
20
21     private GameObject targetCoin;
22
23     public override void Initialize()
24     {
25         if (navMeshAgent == null)
26             navMeshAgent = GetComponent<NavMeshAgent>();
27     }
28
29     public override void OnEpisodeBegin()
30     {
31         Vector3 spawn = coinManager.GetRandomSpawnPoint();
32         navMeshAgent.Warp(spawn);
33         navMeshAgent.ResetPath();
34     }
35
36     public override void CollectObservations(VectorSensor sensor)
37     {
38         // Eigene Position
39         sensor.AddObservation(transform.localPosition / 10f);
40
41         // Distanz & Richtung zum Hunter
42         Vector3 toPlayer = player.position - transform.position;
43         sensor.AddObservation(toPlayer.normalized);
44         sensor.AddObservation(toPlayer.magnitude / 10f);
45
46         // Nächster Coin
```

```
47     GameObject nearestCoin =
48         coinManager.GetNearestCoin(transform.position);
49     if (nearestCoin != null)
50     {
51         Vector3 toCoin = nearestCoin.transform.position -
52             transform.position;
53         sensor.AddObservation(toCoin.normalized);
54         sensor.AddObservation(toCoin.magnitude / 10f);
55     }
56     else
57     {
58         sensor.AddObservation(Vector3.zero);
59         sensor.AddObservation(1f);
60     }
61
62     // Abstand zu anderen Collectoren (max. 2 berücksichtigen)
63     int count = 0;
64     foreach (var other in otherCollectors)
65     {
66         if (other == null || other == this) continue;
67         Vector3 toOther = other.transform.position -
68             transform.position;
69         sensor.AddObservation(toOther.normalized);
70         sensor.AddObservation(toOther.magnitude / 10f);
71         count++;
72         if (count >= 2) break;
73     }
74
75     // Falls weniger als 2 -> Dummy
76     for (; count < 2; count++)
77     {
78         sensor.AddObservation(Vector3.zero);
79         sensor.AddObservation(1f);
80     }
81
82     public override void OnActionReceived(ActionBuffers actions)
83     {
```

```
82     float moveX = Mathf.Clamp(actions.ContinuousActions[0], -1f,
83                               1f);
84     float moveZ = Mathf.Clamp(actions.ContinuousActions[1], -1f,
85                               1f);
86     Vector3 move = new Vector3(moveX, 0, moveZ).normalized;
87     navMeshAgent.Move(move * moveSpeed * Time.deltaTime);
88
89     // --- Reward Shaping ---
90     float distToHunter = Vector3.Distance(transform.position,
91     player.position);
92
93     // Abstand zum Hunter
94     if (distToHunter > safeDistance)
95         AddReward(+0.01f);
96     else if (distToHunter < dangerDistance)
97         AddReward(-0.005f);
98
99     // Abstand zu anderen Collectoren
100    foreach (var other in otherCollectors)
101    {
102        if (other == null || other == this) continue;
103        float dist = Vector3.Distance(transform.position,
104        other.transform.position);
105        if (dist < minCollectorDistance)
106            AddReward(-0.002f); // zu nah -> Malus
107        else
108            AddReward(+0.001f); // schön verteilt -> Bonus
109    }
110
111    // Kleine Living Penalty
112    AddReward(-0.0005f);
113
114    }
115
116    public override void Heuristic(in ActionBuffers actionsOut)
117    {
118        var c = actionsOut.ContinuousActions;
119        c[0] = Input.GetAxis("Horizontal");
120        c[1] = Input.GetAxis("Vertical");
121    }
```

```
117
118     private void OnTriggerEnter(Collider other)
119     {
120         if (other.CompareTag("Projectile"))
121         {
122             AddReward(-5f);
123             EndEpisode();
124         }
125
126         if (other.CompareTag("Coin"))
127         {
128             float distHunter = Vector3.Distance(transform.position,
129             player.position);
129             if (distHunter < safeDistance)
130                 AddReward(+1f);
131                 AddReward(+3f);
132                 coinManager.RespawnCoin(other.gameObject);
133             }
134         }
135
136     private void OnCollisionStay(Collision collision)
137     {
138         if (collision.gameObject.CompareTag("Wall"))
139             AddReward(-0.002f);
140     }
141 }
```

Listing A.1: CollectorAgent.cs

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using UnityEngine.AI;
5
6 public class CollectorController : MonoBehaviour
7 {
8     [Header("Collector Settings")]
9     public string collectorName = "Collector"; // frei im Inspector
10     // public float moveSpeed = 3f;
11     setzbar
```

```
11     public int coinsCollected = 0;
12
13
14     [Header("Behaviour Settings")]
15     public float dangerDistance = 5f; // wenn Player so nah ist ->
        Ausweichmodus
16     public float safeDistance = 8f;    // Coins in diesem Abstand
        bevorzugt
17
18     // private Rigidbody rb;
19     // private GameObject targetCoin;
20     // private Transform player;
21
22     private NavMeshAgent agent;
23     private GameObject targetCoin;
24     private Transform player;
25
26     void Start()
27     {
28         // rb = GetComponent<Rigidbody>();
29         // rb.freezeRotation = true;
30
31         // player =
            GameObject.FindGameObjectWithTag("Player").transform;
32         // FindNewTarget();
33
34
35         agent = GetComponent<NavMeshAgent>();
36         player =
            GameObject.FindGameObjectWithTag("Player").transform;
37
38         FindNewTarget();
39     }
40
41     void Update()
42     {
43         if ((object)targetCoin == null || !targetCoin.activeSelf)
44         {
45             FindNewTarget();
```

```
46     }
47     else
48     {
49         //MoveToCoin();
50         agent.SetDestination(targetCoin.transform.position);
51     }
52 }
53
54
55 void FindNewTarget()
56 {
57     GameObject[] coins = GameObject.FindGameObjectsWithTag("Coin");
58     GameObject[] collectors =
59         GameObject.FindGameObjectsWithTag("Collector");
60
61     if (coins.Length == 0)
62     {
63         targetCoin = null;
64         return;
65     }
66
67     float playerDistance = player != null ?
68         Vector3.Distance(transform.position, player.position) :
69         Mathf.Infinity;
70
71     GameObject bestCoin = null;
72     float bestValue = Mathf.Infinity;
73
74     foreach (GameObject coin in coins)
75     {
76         if (!coin.activeSelf) continue;
77
78         float distToCollector = Vector3.Distance(transform.position,
79             coin.transform.position);
80         float distToPlayer = player != null ?
81             Vector3.Distance(player.position,
82                 coin.transform.position) : Mathf.Infinity;
83
84         // --- Abstand zu anderen Collectoren berechnen ---
```



```
79     float minDistToOtherCollector = Mathf.Infinity;
80     foreach (GameObject col in collectors)
81     {
82         if (col == this.gameObject) continue;
83         float d = Vector3.Distance(col.transform.position,
84                                     coin.transform.position);
85         if (d < minDistToOtherCollector)
86             minDistToOtherCollector = d;
87     }
88     // Coins, die weiter von anderen Collectoren entfernt sind,
89     sind attraktiver
90     float spacingBonus = minDistToOtherCollector * 0.3f;
91     // Bewertungslogik abhängig von Spiellersituation
92     float value;
93     if (playerDistance < dangerDistance)
94     {
95         // Spieler ist nah -> Coins bevorzugen, die weiter vom
96         Spieler weg sind
97         value = distToCollector - spacingBonus - distToPlayer *
98             0.5f;
99     }
100     else
101     {
102         // Normal -> Coins nehmen, die nah am Collector liegen,
103         aber nicht zu nah am Spieler
104         value = distToCollector - spacingBonus + (distToPlayer <
105             safeDistance ? 5f : 0f);
106     }
107     if (value < bestValue)
108     {
109         bestValue = value;
110         bestCoin = coin;
111     }
112     // Fallback: wenn kein guter Coin gefunden -> irgendeinen nehmen
```

```
112     if (bestCoin == null && coins.Length > 0)
113     {
114         bestCoin = coins[Random.Range(0, coins.Length)];
115     }
116
117     targetCoin = bestCoin;
118 }
119
120 private void OnTriggerEnter(Collider other)
121 {
122     if (other.CompareTag("Coin"))
123     {
124         coinsCollected++;
125         other.gameObject.SetActive(false); // Coin einsammeln
126         FindNewTarget();
127     }
128 }
129
130
131 //Für Editor
132 private void OnDrawGizmos()
133 {
134     // DangerDistance (rot) → Kreis um den Collector
135     Gizmos.color = Color.red;
136     Gizmos.DrawWireSphere(transform.position, dangerDistance);
137
138     // SafeDistance (grün) → Kreis um den Player
139     GameObject player =
140         GameObject.FindGameobjectWithTag("Player");
141     if (player != null)
142     {
143         Gizmos.color = Color.green;
144         Gizmos.DrawWireSphere(player.transform.position,
145             safeDistance);
146     }
147
148     // Linie zum aktuellen Coin (gelb)
149     if (targetCoin != null)
150     {
```

```
149         Gizmos.color = Color.yellow;
150         Gizmos.DrawLine(transform.position,
151             targetCoin.transform.position);
152     }
153 }
154 private void OnDrawGizmosSelected()
155 {
156     // Dieser Code läuft IMMER im SceneView, sobald das
157     // GameObject ausgewählt ist
158
159     // DangerDistance (rot)
160     Gizmos.color = Color.red;
161     Gizmos.DrawWireSphere(transform.position, dangerDistance);
162
163     // SafeDistance (grün)
164     Gizmos.color = Color.green;
165     Gizmos.DrawWireSphere(transform.position, safeDistance);
166 }
```

Listing A.2: CollectorController.cs

```
1 using UnityEngine;
2 using Unity.MLAgents;
3 using Unity.MLAgents.Actuators;
4 using Unity.MLAgents.Sensors;
5 using UnityEngine.AI;
6
7 public class DynamicAgent : Agent
8 {
9     [Header("References")]
10     public NavMeshAgent navMeshAgent;
11     public Transform player;
12     public CoinManagerAgent coinManager;
13
14     [Header("Settings")]
15     public float moveSpeed = 3f;
16     public float dangerDistance = 3f;
17     public float safeDistance = 5f;
```

```
18
19 [Header("Collector Settings")]
20 public string collectorName = "Collector";
21 public int coinsCollected = 0;
22 public override void Initialize()
23 {
24     if (navMeshAgent == null)
25         navMeshAgent = GetComponent<NavMeshAgent>();
26 }
27
28 // In DynamicVersion nicht mehr benötigt
29 public override void OnEpisodeBegin()
30 {
31     // Respawn übernimmt GameManager
32 }
33
34 public override void CollectObservations(VectorSensor sensor)
35 {
36     sensor.AddObservation(transform.localPosition / 10f);
37
38     // Distanz zum Player
39     Vector3 toPlayer = player.position - transform.position;
40     sensor.AddObservation(toPlayer.normalized);
41     sensor.AddObservation(toPlayer.magnitude / 10f);
42
43     // Distanz zum nächsten Coin
44     GameObject nearestCoin =
45         coinManager.GetNearestCoin(transform.position);
46     if (nearestCoin != null)
47     {
48         Vector3 toCoin = nearestCoin.transform.position -
49             transform.position;
50         sensor.AddObservation(toCoin.normalized);
51         sensor.AddObservation(toCoin.magnitude / 10f);
52     }
53     else
54     {
55         sensor.AddObservation(Vector3.zero);
56         sensor.AddObservation(1f);
57     }
58 }
```

```
55     }
56
57     // Nächstes Projektil
58     ProjectileAgent[] projectiles =
59         FindObjectsOfType<ProjectileAgent>();
60     ProjectileAgent nearestProj = null;
61     float bestDist = Mathf.Infinity;
62
63     foreach (var p in projectiles)
64     {
65         float d = Vector3.Distance(transform.position,
66                                     p.transform.position);
67         if (d < bestDist)
68         {
69             bestDist = d;
70             nearestProj = p;
71         }
72     }
73
74     if (nearestProj != null)
75     {
76         Vector3 toProj = nearestProj.transform.position -
77             transform.position;
78         sensor.AddObservation(toProj.normalized);
79         sensor.AddObservation(toProj.magnitude / 10f);
80     }
81     else
82     {
83         sensor.AddObservation(Vector3.zero);
84         sensor.AddObservation(1f);
85     }
86 }
87
88 public override void OnActionReceived(ActionBuffers actions)
89 {
90     float moveX = Mathf.Clamp(actions.ContinuousActions[0], -1f,
91                                1f);
92     float moveZ = Mathf.Clamp(actions.ContinuousActions[1], -1f,
93                                1f);
```

```
89
90     Vector3 move = new Vector3(moveX, 0, moveZ).normalized;
91
92     if (move.sqrMagnitude > 0.01f) // nur wenn wirklich eine
93         Richtung da ist
94     {
95         // Zielpunkt etwas vor dem Agenten
96         Vector3 target = transform.position + move * 2f;
97         navMeshAgent.SetDestination(target);
98     }
99     else
100     {
101         // Kein Input → Agent stehen lassen
102         navMeshAgent.ResetPath();
103     }
104     // navMeshAgent.Move(move * moveSpeed * Time.deltaTime);
105
106     // Rewards im Inference-Modus deaktivieren:
107     /*
108     float distToHunter = Vector3.Distance(transform.position,
109         player.position);
110     if (distToHunter > safeDistance) AddReward(+0.002f);
111     else if (distToHunter < dangerDistance) AddReward(-0.002f);
112     AddReward(-0.0005f);
113     */
114 }
115
116 private void OnTriggerEnter(Collider other)
117 {
118     if (other.CompareTag("Projectile"))
119     {
120         // kein EndEpisode() mehr!
121
122         gameObject.SetActive(false);
123     }
124
125     if (other.CompareTag("Coin"))
126     {
127         // im Spiel Coin despawnen + neuen spawnen
128     }
129 }
```

```
126         coinsCollected++;
127         coinManager.RespawnCoin(other.gameObject);
128     }
129 }
130 }
```

Listing A.3: DynamicAgent.cs

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using UnityEngine.UI;
5 using TMPro;
6 using UnityEngine.SceneManagement;
7
8 public class GameManagerDynamic : MonoBehaviour
9 {
10     [Header("Game Settings")]
11     public float roundTime = 60f;
12     public Vector3 arenaSize = new Vector3(10, 0, 10); // gleiche
        Größe wie im Training
13
14     [Header("UI")]
15     public TMP_Text timerText;
16     public GameObject scoreboardPanel;
17     public TMP_Text scoreboardTitle;
18     public TMP_Text scoreboardText;
19
20     private float timeRemaining;
21     private bool roundActive = true;
22
23     private DynamicAgent[] collectors;
24     private CoinManagerAgent coinManager;
25
26     void Start()
27     {
28         // Referenzen holen
29         collectors = FindObjectsOfType<DynamicAgent>();
30         coinManager = FindObjectOfType<CoinManagerAgent>();
31     }
```

```
32         StartRound();
33     }
34
35     void Update()
36     {
37         if (!roundActive) return;
38
39         timeRemaining -= Time.deltaTime;
40         timerText.text = Mathf.Ceil(timeRemaining).ToString() + "s";
41
42         if (timeRemaining <= 0 || AllCollectorsDown())
43         {
44             EndRound();
45         }
46     }
47
48     void StartRound()
49     {
50         timeRemaining = roundTime;
51         roundActive = true;
52         scoreboardPanel.SetActive(false);
53
54         // Coins resettten
55         if (coinManager != null)
56         {
57             ResetAllCoins();
58         }
59
60         // Collectors neu spawnen
61         foreach (var c in collectors)
62         {
63             Vector3 randomSpawn = new Vector3(
64                 Random.Range(-arenaSize.x / 2, arenaSize.x / 2),
65                 0.5f,
66                 Random.Range(-arenaSize.z / 2, arenaSize.z / 2)
67             );
68             c.navMeshAgent.Warp(randomSpawn);
69             c.coinsCollected = 0;
70             c.gameObject.SetActive(true);
```



```
71     }
72 }
73
74 bool AllCollectorsDown()
75 {
76     foreach (var c in collectors)
77     {
78         if (c.gameObject.activeSelf) return false;
79     }
80     return true;
81 }
82
83 void EndRound()
84 {
85     roundActive = false;
86     scoreboardPanel.SetActive(true);
87
88     Cursor.lockState = CursorLockMode.None;
89     Cursor.visible = true;
90
91     float roundDuration = roundTime - timeRemaining;
92     scoreboardTitle.text = "Scoreboard";
93     string results = "Runde beendet nach " +
94         roundDuration.ToString("F1") + " Sekunden\n\n";
95
96     foreach (var c in collectors)
97     {
98         results += c.collectorName + ": " + c.coinsCollected + "
99             Coins\n";
100     }
101
102     scoreboardText.text = results;
103 }
104
105 // Coins komplett neu spawnen
106 void ResetAllCoins()
107 {
108     // Alle alten Coins zerstören
109     var oldCoins = GameObject.FindGameObjectsWithTag("Coin");
```

```

108     foreach (var coin in oldCoins)
109     {
110         Destroy(coin);
111     }
112
113     // Neue Coins spawnen
114     for (int i = 0; i < coinManager.maxCoins; i++)
115     {
116         coinManager.SendMessage("SpawnCoin");
117     }
118 }
119
120 // UI Buttons
121 public void RestartRound()
122 {
123     SceneManager.LoadScene(SceneManager.GetActiveScene().name);
124 }
125
126 public void BackToMainMenu()
127 {
128     SceneManager.LoadScene("MainMenu");
129 }
130 }

```

Listing A.4: GameManagerDynamic.cs

```

1 using UnityEngine;
2 using Unity.MLAgents;
3 using Unity.MLAgents.Actuators;
4 using Unity.MLAgents.Sensors;
5 using UnityEngine.AI;
6
7 public class HunterAgent : Agent
8 {
9     [Header("References")]
10    public NavMeshAgent navMeshAgent;           // Für Bewegung
11    public Transform projectileSpawnPoint;       // Mündung für Schüsse
12    public GameObject projectilePrefab;         // Projektil Prefab
13    public CoinManagerAgent coinManager;        // Coins im
                                           Spielfeld

```

```
14     public CollectorAgent[] collectors;           // Referenz zu allen
           Collectors
15
16     [Header("Settings")]
17     public float moveSpeed = 4f;
18     public float shootForce = 10f;
19     public float shootCooldown = 1f;
20     public float shootRange = 5f;
21
22     private float lastShootTime = -999f;
23
24     public override void Initialize()
25     {
26         if (navMeshAgent == null)
27             navMeshAgent = GetComponent<NavMeshAgent>();
28
29         if (coinManager == null)
30             coinManager = FindObjectOfType<CoinManagerAgent>();
31     }
32
33     public override void OnEpisodeBegin()
34     {
35         // Hunter zurücksetzen
36         transform.localPosition = coinManager.GetRandomSpawnPoint();
37         navMeshAgent.ResetPath();
38
39         // Reset Cooldowns
40         lastShootTime = -999f;
41     }
42
43     public override void CollectObservations(VectorSensor sensor)
44     {
45         // Hunter Position
46         sensor.AddObservation(transform.localPosition / 10f);
47
48         // Eigene Blickrichtung Iteration 2
49         sensor.AddObservation(transform.forward);
50
51         // Beobachte Collector-Positionen
```

```
52     foreach (var c in collectors)
53     {
54         if (c != null)
55         {
56             Vector3 toCollector = c.transform.position -
57                 transform.position;
58             sensor.AddObservation(toCollector.normalized);
59             sensor.AddObservation(toCollector.magnitude / 10f);
60         }
61         else
62         {
63             // Falls kein Collector → Dummy
64             sensor.AddObservation(Vector3.zero);
65             sensor.AddObservation(1f);
66         }
67     }
68
69     public override void OnActionReceived(ActionBuffers actions)
70     {
71         // Bewegung (Continuous)
72         float moveX = Mathf.Clamp(actions.ContinuousActions[0], -1f,
73             1f);
74         float moveZ = Mathf.Clamp(actions.ContinuousActions[1], -1f,
75             1f);
76         Vector3 move = new Vector3(moveX, 0, moveZ).normalized;
77
78         navMeshAgent.Move(move * moveSpeed * Time.deltaTime);
79
80
81         // Schießen (Discrete: 0 = nichts, 1 = schießen)
82         if (actions.DiscreteActions[0] == 1 && Time.time -
83             lastShootTime > shootCooldown)
84         {
85             Shoot();
86         }
```

```

87      // Reward Shaping
88      // In Schussnähe von Collectors bleiben
89      foreach (var c in collectors)
90      {
91          if (c == null) continue;
92          float dist = Vector3.Distance(transform.position,
93                                         c.transform.position);
94
95          // Nähe belohnen
96          //if (dist < shootRange * 1.5f) AddReward(+0.002f);
97          // Iteration 3
98          if (dist < shootRange * 2f) AddReward(+0.002f);
99          if (dist < shootRange * 1.2f) AddReward(+0.005f);
100
101          // Im Zielkorridor (z. B. Hunter schaut in
102             Collector-Richtung)
103          Vector3 dirToCollector = (c.transform.position -
104                                     transform.position).normalized;
105          float aimDot = Vector3.Dot(transform.forward,
106                                     dirToCollector);
107          // if (aimDot > 0.95f && dist < shootRange * 1.2f)
108             AddReward(+0.01f); // zielt "fast direkt" auf
109             Collector
110
111          // Iteration 3
112          if (aimDot > 0.9f && dist < shootRange * 1.5f)
113             AddReward(+0.005f);
114          if (aimDot > 0.98f && dist < shootRange * 1.2f)
115             AddReward(+0.02f);
116
117      }
118
119      // Kleine Living Penalty
120      AddReward(-0.0005f);
121
122  }
123
124  public override void Heuristic(in ActionBuffers actionsOut)
125  {
126      // Für Tastatur-Test
127      var c = actionsOut.ContinuousActions;

```

```
118     c[0] = Input.GetAxis("Horizontal");
119     c[1] = Input.GetAxis("Vertical");
120
121     var d = actionsOut.DiscreteActions;
122     d[0] = Input.GetKey(KeyCode.Space) ? 1 : 0;
123 }
124
125 /* // Iteration 1
126 private void Shoot()
127 {
128     lastShootTime = Time.time;
129
130     GameObject proj = Instantiate(projectilePrefab,
131         projectileSpawnPoint.position,
132         projectileSpawnPoint.rotation);
133
134     Rigidbody rb = proj.GetComponent<Rigidbody>();
135     rb.AddForce(projectileSpawnPoint.forward * shootForce,
136         ForceMode.VelocityChange);
137
138     // Owner setzen (ProjectileAgent!)
139     ProjectileAgent p = proj.GetComponent<ProjectileAgent>();
140     if (p != null) p.SetOwner(this);
141
142     Destroy(proj, 1f); // Projektil nach 3s löschen
143
144     AddReward(-0.01f);
145 }
146 */
147
148 private void Shoot()
149 {
150     lastShootTime = Time.time;
151
152     GameObject proj = Instantiate(projectilePrefab,
153         projectileSpawnPoint.position,
154         projectileSpawnPoint.rotation);
155
156     Rigidbody rb = proj.GetComponent<Rigidbody>();
157     rb.AddForce(projectileSpawnPoint.forward * shootForce,
158         ForceMode.VelocityChange);
```

```
151
152     ProjectileAgent p = proj.GetComponent<ProjectileAgent>();
153     if (p != null) p.SetOwner(this);
154
155     Destroy(proj, 1f);
156
157     // Strafe fürs Ballern ins Leere (deutlich spürbar machen)
158     AddReward(-0.02f);
159 }
160
161 private void OnTriggerEnter(Collider other)
162 {
163     if (other.CompareTag("Coin"))
164     {
165         // Strafe: Hunter sollte Coins nicht einsammeln , Camping
166         verhindern
167         AddReward(-0.5f);
168     }
169
170 public void OnCollectorHit()
171 {
172     // Wird von Collector aufgerufen, wenn er getroffen wurde
173     AddReward(+10f);
174 }
175 }
```

Listing A.5: HunterAgent.cs

A.4 TensorBoard-Statistiken

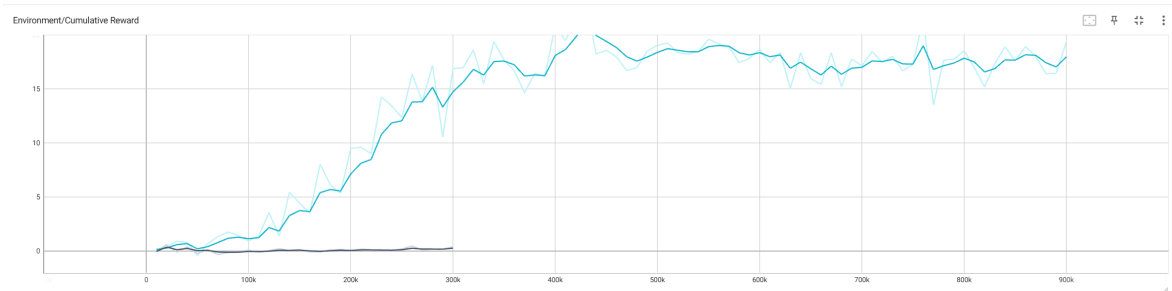


Abbildung A.1: Cumulative Reward – Iteration 1 (Collector und Hunter).

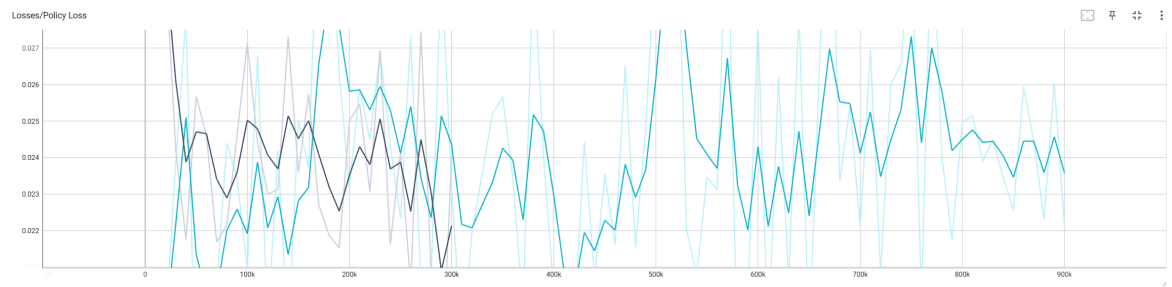


Abbildung A.2: Policy Loss – Iteration 1 (Collector und Hunter).

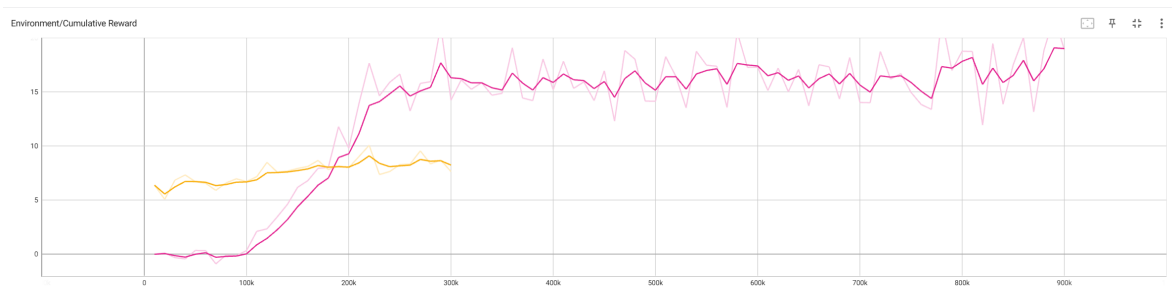


Abbildung A.3: Cumulative Reward – Iteration 2 (Collector und Hunter).

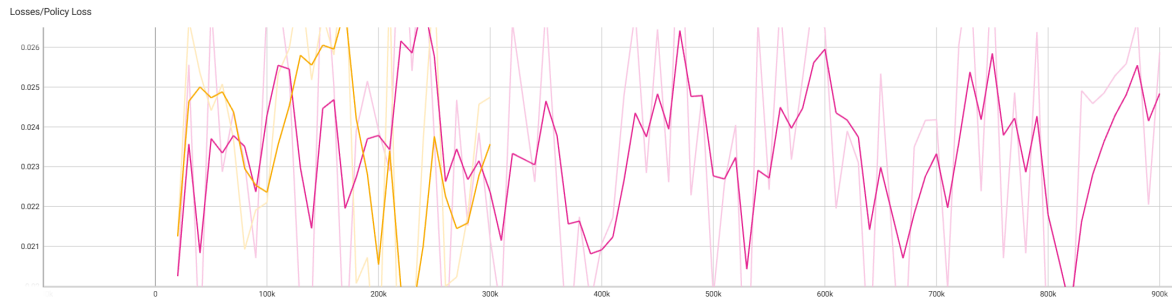


Abbildung A.4: Policy Loss – Iteration 2 (Collector und Hunter).

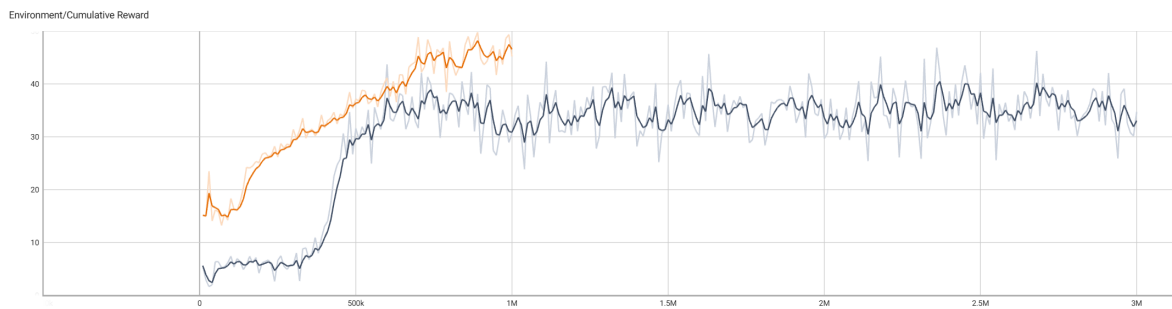


Abbildung A.5: Cumulative Reward – Iteration 3 (Collector und Hunter).

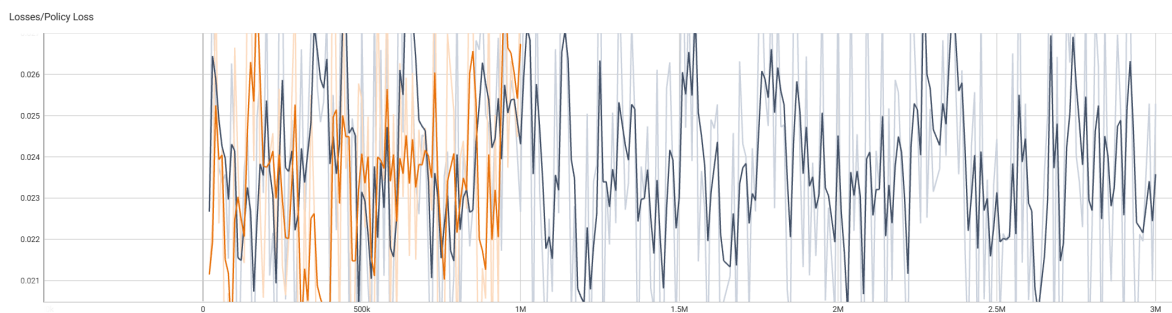


Abbildung A.6: Policy Loss – Iteration 3 (Collector und Hunter).

Gesamtübersicht der Iterationen:

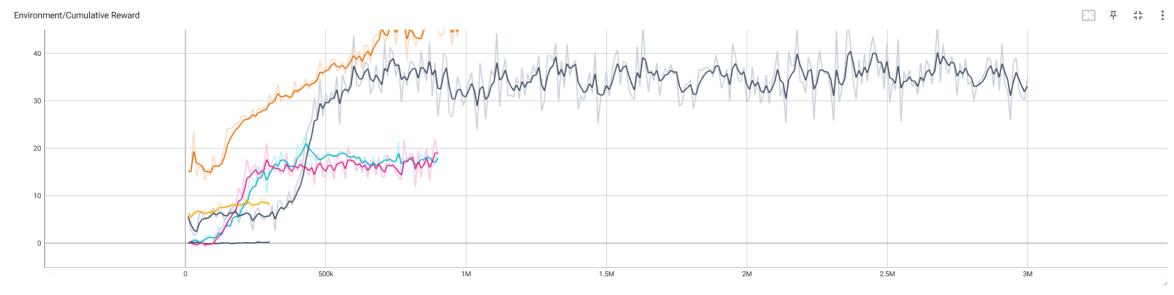


Abbildung A.7: Cumulative Reward (Collector und Hunter).

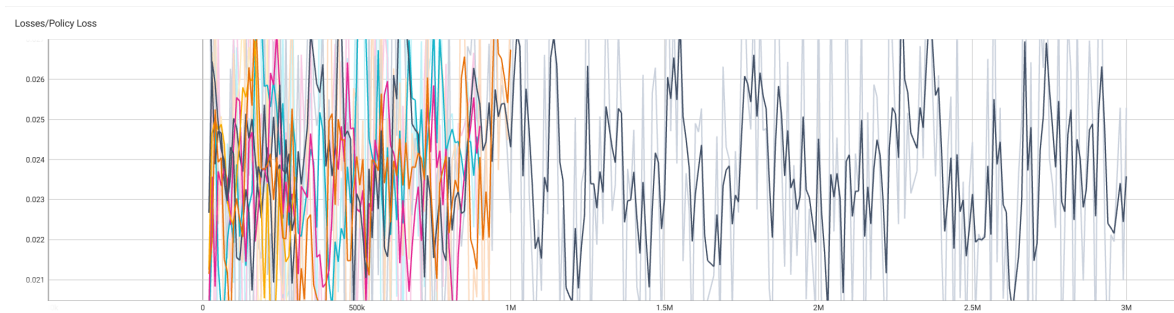


Abbildung A.8: Policy Loss (Collector und Hunter).

Literaturverzeichnis

- [1] J. Almeida, T. Gomes, and R. J. F. Rossetti, “Reinforcement learning applied to ai bots in first-person multiplayer deathmatch shooter,” *Algorithms*, vol. 16, no. 7, p. 323, 2023. [Online]. Available: <https://doi.org/10.3390/a16070323>
- [2] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994. [Online]. Available: <https://doi.org/10.1002/9780470316887>
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>
- [4] C. J. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992. [Online]. Available: <https://doi.org/10.1007/BF00992698>
- [5] L. P. Kaelbling, M. L. Littman, and A. W. Moore, “Reinforcement learning: A survey,” *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996. [Online]. Available: <https://jair.org/index.php/jair/article/view/10166>
- [6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: <https://doi.org/10.1038/nature14236>
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>

- [8] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser *et al.*, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016. [Online]. Available: <https://doi.org/10.1038/nature16961>
- [9] I. Millington and J. Funge, *Artificial Intelligence for Games*, 3rd ed. CRC Press, 2019. [Online]. Available: <https://www.routledge.com/AI-for-Games-Third-Edition/Millington/p/book/9780367670566>
- [10] D. Isla, “Handling complexity in the halo 2 ai,” 2005, gDC 2005 Proceeding. [Online]. Available: <https://www.gamedeveloper.com/programming/gdc-2005-proceeding-handling-complexity-in-the-i-halo-2-i-ai>
- [11] J. Zhou, “Authoring games with tile rewrite rule behavior trees,” *ACM Digital Library*, 2024. [Online]. Available: <https://dl.acm.org/doi/fullHtml/10.1145/3649921.3656979>
- [12] R. Hunicke, “The case for dynamic difficulty adjustment in games,” in *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology*. ACM, 2005. [Online]. Available: <https://doi.org/10.1145/1178477.1178573>
- [13] G. Andrade, G. Ramalho, H. Santana, and V. Corruble, “Extending reinforcement learning to provide dynamic game balancing,” in *Proceedings of the IJCAI-05 Workshop on Reasoning, Representation, and Learning in Computer Games*, D. W. Aha, H. Muñoz-Avila, and M. van Lent, Eds., Edinburgh, Scotland, 2005, pp. 7–12, workshop Proceedings; vollständiger Band, Beitrag auf S. 7–12. [Online]. Available: <https://www.cs.auckland.ac.nz/compsci777s2c/ijcai05.pdf>
- [14] C. Holmgård, M. C. Green, A. Liapis, and J. Togelius, “Automated playtesting with procedural personas through MCTS with evolved heuristics,” *IEEE Transactions on Games*, vol. 11, no. 4, pp. 352–362, 2019. [Online]. Available: <https://doi.org/10.1109/TG.2018.2808198>
- [15] M. Booth, “The ai systems of Left 4 Dead,” in *Proceedings of the Fifth Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE*

- 2009). Stanford, CA: AAAI Press, 2009, keynote/Industry Talk. [Online]. Available: https://steamcdn-a.akamaihd.net/apps/valve/2009/ai_systems_of_l4d_mike_booth.pdf
- [16] T. N. D. Oliveira *et al.*, “Investigating reinforcement learning for dynamic difficulty adjustment in fighting game,” *Proceedings of the ACM Digital Conference*, 2023. [Online]. Available: <https://dl.acm.org/doi/10.1145/3631085.3631229>
- [17] J. Zhang, “Implementation and effect evaluation of dynamic difficulty adjustment based on reinforcement learning in multiplayer online battle arena games,” *Science and Technology of Engineering Chemistry and Environmental Protection*, 2024. [Online]. Available: <https://doi.org/10.61173/r8885t34>
- [18] C. Holmgård, J. Zhu, and S. Ontañón, “Player-centered ai for automatic game personalization: Open problems,” in *Proceedings of the 15th International Conference on the Foundations of Digital Games (FDG)*. ACM, 2021. [Online]. Available: <https://doi.org/10.1145/3402942.3402951>
- [19] C. Zhu, M. Dastani, and S. Wang, “A survey of multi-agent deep reinforcement learning with communication,” *Autonomous Agents and Multi-Agent Systems*, 2024. [Online]. Available: <https://www.ifaamas.org/Proceedings/aamas2024/pdfs/p2845.pdf>
- [20] A. Sestini, A. Kuhnle, and A. D. Bagdanov, “Policy fusion for adaptive and customizable reinforcement learning agents,” *arXiv preprint arXiv:2104.10610*, 2021. [Online]. Available: <https://arxiv.org/abs/2104.10610>
- [21] F. G. Glavin and M. G. Madden, “Skilled experience catalogue: A skill-balancing mechanism for non-player characters using reinforcement learning,” *arXiv preprint arXiv:1806.07637*, 2018. [Online]. Available: <https://arxiv.org/abs/1806.07637>
- [22] U. Technologies, “Ml-agents toolkit documentation,” <https://github.com/Unity-Technologies/ml-agents>, 2023, zugriff am 01. August 2025.
- [23] N. Döring and J. Bortz, *Forschungsmethoden und Evaluation in den Sozial- und Humanwissenschaften*, 5th ed. Berlin / Heidelberg: Springer, 2016. [Online]. Available: <https://doi.org/10.1007/978-3-642-41089-5>

- [24] G. Charness, U. Gneezy, and A. Megginat, “Experimental methods: Between-subject and within-subject designs,” *Journal of Economic Behavior & Organization*, vol. 81, no. 1, pp. 1–8, 2012. [Online]. Available: <https://doi.org/10.1016/j.jebo.2011.08.009>
- [25] P. R. Wurman, S. C. Barrett, K. Kawamoto, J. MacGlashan, K. Subramanian, T. Walsh, R. Capobianco, A. Devlic, F. Eckert, F. Fuchs, L. H. Gilpin, P. Khandelwal, V. J. Kompella, H.-C. Lin, P. MacAlpine, D. D. Oller, H. Aghabozorgi, and H. ... Kitano, “Outracing champion gran turismo drivers with deep reinforcement learning,” *Nature*, vol. 602, no. 7896, pp. 223–228, 2022, sony AI & Polyphony Digital, „GT Sophy“. [Online]. Available: <https://europepmc.org/article/med/35140384>
- [26] J. Romoff and D. O’Connor, “Deep reinforcement learning for efficient player bots – talk at cedec/ubisoft la forge,” <https://www.ubisoft.com/en-us/studio/laforge/news/6Mv4hZqUMJoY1xpfl1yiQP1/ubisoft-la-forge-pushing-stateoftheart-ai-in-games-to-create-the-next-generation-of-npcs>, 2022, ubisoft La Forge – Blog/Research News.